

讲堂 深入拆解 Java 虚拟机 文章详情

开篇词 | 为什么我们要学习 Java虚拟机？

2018-07-16 郑雨迪



开篇词 | 为什么我们要学习Java虚拟机？

朗读人：郑雨迪 07'14" | 3.32M

前不久我参加了一个国外程序员的讲座，讲座的副标题很有趣，叫做：“我如何学会停止恐惧，并且爱上 Java 虚拟机”。

这句话来自一部黑色幽默电影《奇爱博士》，电影描述了冷战时期剑拔弩张的氛围。

程序员之间的语言之争又未尝不是如此。写系统语言的鄙视托管语言低下的执行效率；写托管语言的则取笑系统语言需要手动管理内存；写动态语言的不屑于静态语言那冗余的类型系统；写静态语言的则嘲讽动态语言里面各种光怪陆离的运行时错误。

Java 作为应用最广的语言，自然吸引了不少的攻击，而身为 Java 程序员的你，或许在口水战中落了下风，忿忿于没有足够的知识武装自己；又或许想要深入学习 Java 语言，却又无从下手。甚至是在实践中被 Java 的启动性能、内存耗费所震惊，因此对 Java 语言本身产生了种种的怀疑与顾虑。

别担心，我就是来解答你对 Java 的种种疑虑的。“知其然”也要“知其所以然”，学习 Java 虚拟机的本质，更多是了解 Java 程序是如何被执行且优化的。这样一来，你才可以从内部入手，达到高效编程

的目的。与此同时，你也可以为学习更深层级、更为核心的 Java 技术打好基础。

我相信在不少程序员的观念里，Java 虚拟机是透明的。在大家看来，我们仅需知道 Java 核心类库，以及第三方类库里 API 的用法，便可以专注于实现具体业务，并且依赖 Java 虚拟机自动执行乃至优化我们的应用程序。那么，我们还需要了解 Java 虚拟机吗？

我认为是非常有必要的。如果我们把核心类库的 API 比做数学公式的话，那么 Java 虚拟机的知识就好比公式的推导过程。掌握数学公式固然可以应付考试，但是了解背后的推导过程更加有助于记忆和理解。并且，在遇到那些没法套公式的情况下，我们也能知道如何解决。

具体来说，了解 Java 虚拟机有如下（但不限于）好处。

首先，Java 虚拟机提供了许多配置参数，用于满足不同应用场景下，对程序性能的需求。学习 Java 虚拟机，你可以针对自己的应用，最优化匹配运行参数。（你可以用下面这个例子看一下自己虚拟机的参数列表。）

举例来说，macOS 上的 Java 10 共有近千个配置参数：

```
$ java -XX:+PrintFlagsFinal -XX:+UnlockDiagnosticVMOptions -version | wc -l
java version "10" 2018-03-20
Java(TM) SE Runtime Environment 18.3 (build 10+46)
Java HotSpot(TM) 64-Bit Server VM 18.3 (build 10+46, mixed mode)
812
```

其次，Java 虚拟机本身是一种工程产品，在实现过程中自然存在不少局限性。学习 Java 虚拟机，可以更好地规避它在使用中的 Bug，也可以更快地识别出 Java 虚拟机中的错误，

再次，Java 虚拟机拥有当前最前沿、最成熟的垃圾回收算法实现，以及即时编译器实现。学习 Java 虚拟机，我们可以了解背后的设计决策，今后再遇到其他代码托管技术也能触类旁通。

最后，Java 虚拟机发展到了今天，已经脱离 Java 语言，形成了一套相对独立的、高性能的执行方案。除了 Java 外，Scala、Clojure、Groovy，以及时下热门的 Kotlin，这些语言都可以运行在 Java 虚拟机之上。学习 Java 虚拟机，便可以了解这些语言的通用机制，甚至于让这些语言共享生态系统。

说起写作这个专栏的初心，与我个人的经历是分不开的，我现在是甲骨文实验室的高级研究员，工作主要是负责研究如何通过程序分析技术以及动态编译技术让程序语言跑得更快。明面上，我是 Graal 编译器的核心开发者之一，在为 HotSpot 虚拟机项目拧螺丝。

这里顺便说明一下，Graal 编译器是 Java 10 正式引入的实验性即时编译器，在国内同行口中被戏称

为“甲骨文黑科技”。当然，在我看来，我们的工作同样也是分析应用程序的性能瓶颈，寻找优化空间，只不过我们的优化方式对自动化、通用性有更高的要求。

加入甲骨文之前，我在瑞士卢加诺大学攻读博士学位，研究如何更加精准地监控 Java 程序，以便做出更具针对性的优化。这些研究工作均已发表在程序语言方向的顶级会议上，并获得了不少同行的认可（OOPSLA 2015 最佳论文奖）。

在这 7 年的学习工作生涯中，我拜读过许多大神关于 Java 虚拟机的技术博客。在受益匪浅的同时，我发觉不少文章的门槛都比较高，而且过分注重实现细节，这并不是大多数的开发人员可以受益的调优方案。这么一来，许多原本对 Java 虚拟机感兴趣的同学，也因为过高的门槛，以及短时间内看不到的收益，而放弃了对 Java 虚拟机的学习。

在收到极客时间的邀请后，我决定也挑战一下 Java 虚拟机的科普工作。和其他栏目一样，我会用简单通俗的语言，来介绍 Java 虚拟机的实现。具体到每篇文章，我将采用一个贯穿全文的案例来阐述知识点，并且给出相应的调优建议。在文章的末尾，我还将附上一个动手实践的环节，帮助你巩固对知识点的理解。

整个专栏将分为四大模块。

- 1. 基本原理：剖析 Java 虚拟机的运行机制，逐一介绍 Java 虚拟机的设计决策以及工程实现；
- 2. 高效实现：探索 Java 编译器，以及内嵌于 Java 虚拟机中的即时编译器，帮助你更好地理解 Java 语言特性，继而写出简洁高效的代码；
- 3. 代码优化：介绍如何利用工具定位并解决代码中的问题，以及在已有工具不适用的情况下，如何打造专属轮子；
- 4. 虚拟机黑科技：介绍甲骨文实验室近年来的前沿工作之一 GraalVM。包括如何在 JVM 上高效运行其他语言；如何混搭这些语言，实现 Polyglot；如何将这些语言事前编译（Ahead-Of-Time，AOT）成机器指令，单独运行甚至嵌入至数据库中运行。

我希望借由这四个模块 36 个案例，帮助你理解 Java 虚拟机的运行机制，掌握诊断手法和调优方式。最重要的，是激发你学习 Java 虚拟机乃至其他底层工作、前沿工作的热情。

知识框架图





(点击查看高清大图, iOS 用户可长按保存)

版权归极客邦科技所有, 未经许可不得转载

精选留言



godtrue

JVM

19

1:现在我的理解

1-1:三个英文单词的缩写，中文意思是Java虚拟机，作用是帮助执行Java字节码的，不同的平台有不同的JVM，这样java源代码经过编译为字节码之后就能在各种平台上运行了

1-2:JVM还有内存管理，垃圾回收等底层功能，这样程序员就不用太操心这些事情了

1-3:内存管理主要是将JVM的内存容量划分成不同的模块，使用不同的管理方式，核心目的是为了更少的内存使用更快的内存性能以及恰当的内存回收策略

1-4:垃圾回收之所以存在是因为JVM是运行在内存之中的，它的内存空间是有限的，当加载进内存的对象越来越多的时候，会影响JVM的运行性能，所以，要回收一些内存空间，垃圾回收的关键是识别出垃圾以及使用不影响JVM运行的回收策略

1-5:JVM是Java代码执行的地方，Java程序性能上出现了问题的时候，我们要从代码层面进行分析、定位、优化，但是我们怎么知道那段代码性能差哪？此时要看JVM中代码的执行情况，看看谁慢？为什么慢？这些我还不清楚咋快速定位，所以，我订阅了这个专栏

2我希望我能学到如下内容

2-1:Java源代码怎么变成Java字节码的？

2-2:Java字节码怎么进入JVM的？

2-3:Java 字节码进入JVM后是怎么放置的？

2-4:JVM执行字节码的时候怎么定位的？他怎么知道该执行那句代码了？它怎么知道那句代码是什么意思？

2-5:性能优化，我的理解是让干活快的干活，不让干的慢的干，如果做不到，就让干活快的多干，干的慢的少干？JVM的性能优化可能也类似，哪JVM怎么知道谁干的慢谁干的快？JVM在执行Java字节码的时候都是需要做什么事情呢？它怎么安排自己的工作呢？

2-6:实际开发工作中怎么监控JVM的工作情况哪？怎么定位那些懒蛋哪？定位到了怎么解决他们哪？

2018-07-21



原子

16

JVM很有用，目测会是个不错的专栏，期待！

2018-07-16



水有罔象番石榴

7

看了知识框架图 可以说十分期待了

2018-07-16



木甘

6

是java10吗

2018-07-17

作者回复

介绍的内容不局限于一个版本的。从8到11都会涉及到。

2018-07-17

- 

浮生老莫

期待老师后续的内容，刚想学JVM，就来了，再打磨打磨自己的技术

2018-07-16

3
- 

沙漏人生

已购，看看作者大能如何把复杂的东西简单化。

2018-07-16

2
- 

zhenTomcat

期待

2018-07-16

2
- 

Desperado

沙发期待中

2018-07-18

1
- 

为了吃方便面

下面这个讲课的目录很给力

2018-07-18

0
- 

lynd

能够对java虚拟机做分块的详细介绍不，最好能附上简短的代码介绍，谢谢！

2018-07-17

0

作者回复

这个专栏会避免介绍具体的虚拟机代码。但我会在最后列举一些学习资料。

2018-07-17
- 

刘晓宇

希望不虚此行，希望有趣。

2018-07-16

0
- 

三木子

很期待

2018-07-16

0
- 

苏忆

看了下目录，介绍的比较笼统，希望讲解的时候比较深入并提供相关资料提供学习。谢谢，郑大，一起加油！

2018-07-16

8
- 

云学

提一个建议: 让读者看懂是第一位的。只要读者会java语法，就应该能让他看懂，谢谢

2018-07-17

5



小宝儿
Android用户也可以长按保存

2018-07-17

4



小猫也是老虎
希望大佬能在以后更新快点，毕竟秋招要来了💎💎

2018-07-19

3



夏雪冬日
单纯根据目录看，介绍的还是比较泛，希望在文章中对核心点能进行深入的分析，期待精彩干货。

2018-07-16

3



Knuthie
与openjdk open jvm的对比可以讲讲么？

2018-07-19

2



吴双
已购买，期待后续文章啊

2018-07-17

2



李小龙
看目录还是比较全面的，非常期待

2018-07-27

1

讲堂 深入拆解 Java 虚拟机 文章详情

01 | Java代码是怎么运行的？

2018-07-20 郑雨迪



01 | Java代码是怎么运行的？

朗读人：郑雨迪 10'36" | 6.09M

我们学院的一位教授之前去美国开会，入境的时候海关官员就问他：既然你会计算机，那你谈谈你用的都是什么语言吧？

教授随口就答了个 Java。海关一看是懂行的，也就放行了，边敲章还边说他们上学那会学的是 C+。我还特意去查了下，真有叫 C+ 的语言，但是这里海关官员应该指的是 C++。

事后教授告诉我们，他当时差点就问海关，是否知道 Java 和 C++ 在运行方式上的区别。但是又担心海关官员拿他的问题来考别人，也就没问出口。那么，下次你去美国，不幸地被海关官员问这个问题，你懂得如何回答吗？

作为一名 Java 程序员，你应该知道，Java 代码有很多种不同的运行方式。比如说可以在开发工具中运行，可以双击执行 jar 文件运行，也可以在命令行中运行，甚至可以在网页中运行。当然，这些执行方式都离不开 JRE，也就是 Java 运行时环境。

实际上，JRE 仅包含运行 Java 程序的必需组件，包括 Java 虚拟机以及 Java 核心类库等。我们 Java

程序员经常接触到的 JDK（Java 开发工具包）同样包含了 JRE，并且还附带了一系列开发、诊断工具。

然而，运行 C++ 代码则无需额外的运行时。我们往往把这些代码直接编译成 CPU 所能理解的代码格式，也就是机器码。

比如下图的中间列，就是用 C 语言写的 Helloworld 程序的编译结果。可以看到，C 程序编译而成的机器码就是一个一个的字节，它们是给机器读的。那么为了让开发人员也能够理解，我们可以用反汇编器将其转换成汇编代码（如下图的最右列所示）。

```

; 最左列是偏移; 中间列是给机器读的机器码; 最右列是给人读的汇编代码
0x00:  55                push   rbp
0x01:  48 89 e5           mov    rbp, rsp
0x04:  48 83 ec 10        sub    rsp, 0x10
0x08:  48 8d 3d 3b 00 00 00 lea     rdi, [rip+0x3b]
                                ; 加载 "Hello, World!\n"
0x0f:  c7 45 fc 00 00 00 00 mov    DWORD PTR [rbp-0x4], 0x0
0x16:  b0 00              mov    al, 0x0
0x18:  e8 0d 00 00 00     call   0x12
                                ; 调用 printf 方法
0x1d:  31 c9              xor     ecx, ecx
0x1f:  89 45 f8           mov    DWORD PTR [rbp-0x8], eax
0x22:  89 c8              mov    eax, ecx
0x24:  48 83 c4 10        add    rsp, 0x10
0x28:  5d                pop    rbp
0x29:  c3                ret
```

既然 C++ 的运行方式如此成熟，那么你有没有想过，为什么 Java 要在虚拟机中运行呢，Java 虚拟机具体又是怎样运行 Java 代码的呢，它的运行效率又如何呢？

今天我便从这几个问题入手，和你探讨一下，Java 执行系统的主流实现以及设计决策。

为什么 Java 要在虚拟机里运行？

Java 作为一门高级程序语言，它的语法非常复杂，抽象程度也很高。因此，直接在硬件上运行这种复杂的程序并不现实。所以呢，在运行 Java 程序之前，我们需要对其进行一番转换。

这个转换具体是怎么操作的呢？当前的主流思路是这样子的，设计一个面向 Java 语言特性的虚拟机，并通过编译器将 Java 程序转换成该虚拟机所能识别的指令序列，也称 Java 字节码。这里顺便

说一句，之所以这么取名，是因为 Java 字节码指令的操作码（opcode）被固定为一个字节。

举例来说，下图的中间列，正是用 Java 写的 Helloworld 程序编译而成的字节码。可以看到，它与 C 版本的编译结果一样，都是由一个个字节组成的。

并且，我们同样可以将其反汇编为人类可读的代码格式（如下图的最右列所示）。不同的是，Java 版本的编译结果相对精简一些。这是因为 Java 虚拟机相对于物理机而言，抽象程度更高。

```
# 最左列是偏移；中间列是给虚拟机读的机器码；最右列是给人读的代码

0x00:  b2 00 02      getstatic java.lang.System.out
0x03:  12 03          ldc "Hello, World!"
0x05:  b6 00 04      invokevirtual java.io.PrintStream.println
0x08:  b1              return
```

Java 虚拟机可以由硬件实现 [1]，但更为常见的是在各个现有平台（如 Windows_x64、Linux_aarch64）上提供软件实现。这么做的意义在于，一旦一个程序被转换成 Java 字节码，那么它便可以在不同平台上的虚拟机实现里运行。这也就是我们经常说的“一次编写，到处运行”。

虚拟机的另外一个好处是它带来了一个托管环境（Managed Runtime）。这个托管环境能够代替我们处理一些代码中冗长而且容易出错的部分。其中最广为人知的当属自动内存管理与垃圾回收，这部分内容甚至催生了一波垃圾回收调优的业务。

除此之外，托管环境还提供了诸如数组越界、动态类型、安全权限等等的动态检测，使我们免于书写这些无关业务逻辑的代码。

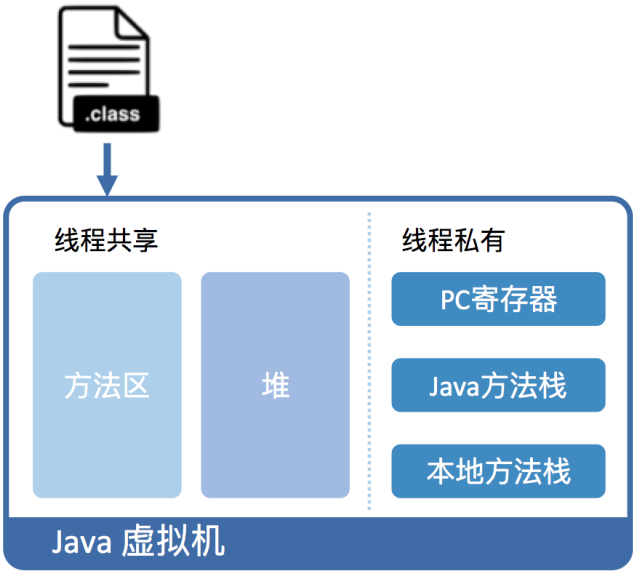
Java 虚拟机具体是怎样运行 Java 字节码的？

下面我将以标准 JDK 中的 HotSpot 虚拟机为例，从虚拟机以及底层硬件两个角度，给你讲一讲 Java 虚拟机具体是怎么运行 Java 字节码的。

从虚拟机视角来看，执行 Java 代码首先需要将它编译而成的 class 文件加载到 Java 虚拟机中。加载后的 Java 类会被存放于方法区（Method Area）中。实际运行时，虚拟机会执行方法区内的代码。

如果你熟悉 X86 的话，你会发现这和段式内存管理中的代码段类似。而且，Java 虚拟机同样也在内存中划分出堆和栈来存储运行时数据。

不同的是，Java 虚拟机会将栈细分为面向 Java 方法的 Java 方法栈，面向本地方法（用 C++ 写的 native 方法）的本地方法栈，以及存放各个线程执行位置的 PC 寄存器。

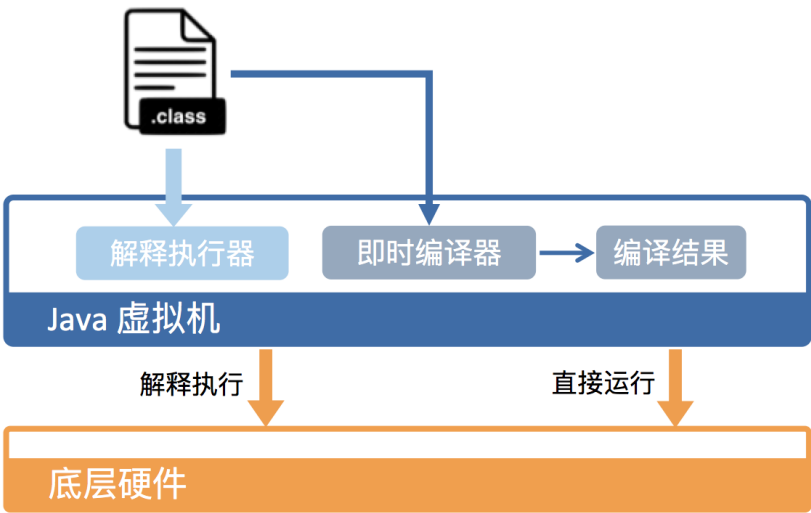


在运行过程中，每当调用进入一个 Java 方法，Java 虚拟机会在当前线程的 Java 方法栈中生成一个栈帧，用以存放局部变量以及字节码的操作数。这个栈帧的大小是提前计算好的，而且 Java 虚拟机不要求栈帧在内存空间里连续分布。

当退出当前执行的方法时，不管是正常返回还是异常返回，Java 虚拟机均会弹出当前线程的当前栈帧，并将之舍弃。

从硬件视角来看，Java 字节码无法直接执行。因此，Java 虚拟机需要将字节码翻译成机器码。

在 HotSpot 里面，上述翻译过程有两种形式：第一种是解释执行，即逐条将字节码翻译成机器码并执行；第二种是即时编译（Just-In-Time compilation，JIT），即将一个方法中包含的所有字节码编译成机器码后再执行。



前者的优势在于无需等待编译，而后者的优势在于实际运行速度更快。HotSpot 默认采用混合模式，综合了解释执行和即时编译两者的优点。它会先解释执行字节码，而后将其中反复执行的热点代码，以方法为单位进行即时编译。

Java 虚拟机的运行效率究竟是怎么样的？

HotSpot 采用了多种技术来提升启动性能以及峰值性能，刚刚提到的即时编译便是其中最重要的技术之一。

即时编译建立在程序符合二八定律的假设上，也就是百分之二十的代码占据了百分之八十的计算资源。

对于占据大部分的不常用的代码，我们无需耗费时间将其编译成机器码，而是采取解释执行的方式运行；另一方面，对于仅占据小部分的热点代码，我们则可以将其编译成机器码，以达到理想的运行速度。

理论上讲，即时编译后的 Java 程序的执行效率，是可能超过 C++ 程序的。这是因为与静态编译相比，即时编译拥有程序的运行时信息，并且能够根据这个信息做出相应的优化。

举个例子，我们知道虚方法是用来实现面向对象语言多态性的。对于一个虚方法调用，尽管它有很多个目标方法，但在实际运行过程中它可能只调用其中的一个。

这个信息便可以被子时编译器所利用，来规避虚方法调用的开销，从而达到比静态编译的 C++ 程序更高的性能。

为了满足不同用户场景的需要，HotSpot 内置了多个即时编译器：C1、C2 和 Graal。Graal 是 Java

10 正式引入的实验性即时编译器，在专栏的第四部分我会详细介绍，这里暂不做讨论。

之所以引入多个即时编译器，是为了在编译时间和生成代码的执行效率之间进行取舍。C1 又叫做 Client 编译器，面向的是对启动性能有要求的客户端 GUI 程序，采用的优化手段相对简单，因此编译时间较短。

C2 又叫做 Server 编译器，面向的是对峰值性能有要求的服务器端程序，采用的优化手段相对复杂，因此编译时间较长，但同时生成代码的执行效率较高。

从 Java 7 开始，HotSpot 默认采用分层编译的方式：热点方法首先会被 C1 编译，而后热点方法中的热点会进一步被 C2 编译。

为了不干扰应用的正常运行，HotSpot 的即时编译是放在额外的编译线程中进行的。HotSpot 会根据 CPU 的数量设置编译线程的数目，并且按 1:2 的比例配置给 C1 及 C2 编译器。

在计算资源充足的情况下，字节码的解释执行和即时编译可同时进行。编译完成后的机器码会在下次调用该方法时启用，以替换原本的解释执行。

总结与实践

今天我简单介绍了 Java 代码为何在虚拟机中运行，以及如何在虚拟机中运行。

之所以要在虚拟机中运行，是因为它提供了可移植性。一旦 Java 代码被编译为 Java 字节码，便可以在不同平台上的 Java 虚拟机实现上运行。此外，虚拟机还提供了一个代码托管的环境，代替我们处理部分冗长而且容易出错的事务，例如内存管理。

Java 虚拟机将运行时内存区域划分为五个部分，分别为方法区、堆、PC 寄存器、Java 方法栈和本地方法栈。Java 程序编译而成的 class 文件，需要先加载至方法区中，方能在 Java 虚拟机中运行。

为了提高运行效率，标准 JDK 中的 HotSpot 虚拟机采用的是一种混合执行的策略。

它会解释执行 Java 字节码，然后将其中反复执行的热点代码，以方法为单位进行即时编译，翻译成机器码后直接运行在底层硬件之上。

HotSpot 装载了多个不同的即时编译器，以便在编译时间和生成代码的执行效率之间做取舍。

下面我给你留一个小作业，通过观察两个条件判断语句的运行结果，来思考 Java 语言和 Java 虚拟机看待 boolean 类型的方式是否不同。

下载 `asmtools.jar` [2]，并在命令行中运行下述指令（不包含提示符 \$）：

```
$ echo '  
public class Foo {
```



```
public static void main(String[] args) {  
    boolean flag = true;  
    if (flag) System.out.println("Hello, Java!");  
    if (flag == true) System.out.println("Hello, JVM!");  
}  
' > Foo.java  
$ javac Foo.java  
$ java Foo  
$ java -cp /path/to/asmttools.jar org.openjdk.asmttools.jdis.Main Foo.class > Foo.jasm.1  
$ awk 'NR==1,/iconst_1/{sub(/iconst_1/, "iconst_2")} 1' Foo.jasm.1 > Foo.jasm  
$ java -cp /path/to/asmttools.jar org.openjdk.asmttools.jasm.Main Foo.jasm  
$ java Foo
```

[1]: https://en.wikipedia.org/wiki/Java_processor

[2]: <https://wiki.openjdk.java.net/display/CodeTools/asmttools>



版权归极客邦科技所有，未经许可不得转载

精选留言



小名叫大明

受益匪浅，多谢老师。

请教老师一个问题，网上我没有搜到。

服务器线程数爆满，使用jstack打印线程堆栈信息，想知道是哪类线程数太多，但是堆栈里全是一样的信息且没有任何关键信息，是哪个方法创建的，以及哪个线程池的都看不到。

如何更改打印线程堆栈信息的代码（动态）让其打印线程池信息呢？

2018-07-26



东方

114

jvm把boolean当做int来处理

Bag = iconst_1 = true

awk把stackframe中的Bag改为iconst_2

if (Bag) 比较时ifeq指令做是否为零判断，常数2仍为true，打印输出

if (true == Bag) 比较时if_cmpne做整数比较，iconst_1是否等于Bag，比较失败，不再打印输出

2018-07-20

作者回复

字节码高手！

2018-07-20



かつこいすぎる 郑一凡

33

我想问下，JVM是怎么区别出热点代码和非热点代码的？

2018-07-20



novembersky

32

文中提到虚拟机会把部分热点代码编译成机器码，我有个疑问，为什么不把java代码全部编译成机器码？很多服务端应用发布频率不会太频繁，但是对运行时的性能和吞吐量要求较高。如果发布或启动时多花点时间编译，能够带来运行时的持久性能收益，不是很合适么？

2018-07-20

作者回复

问得好！事实上JVM确实有考虑做AOT (ahead of time compilation) 这种事情。AOT能够在线下将Java字节码编译成机器码，主要是用来解决启动性能不好的问题。

对于这种发布频率不频繁(也就是长时间运行吧？)的程序，其实选择线下编译和即时编译都一样，因为至多一两个小时该即时编译的都已经编译完成了。另外，即时编译器因为有程序的运行时信息，优化效果更好，也就是说峰值性能更好。

2018-07-20

醉人

25



解释执行 执行时才翻译成机器指令，无需保存不占内存。但即时编译类似预编译，编译之后的指令需要保存在内存中，这种方式吃内存，按照二八原则这种混合模式最恰当的，热点代码编译之后放入内存避免重复编译，而其他运行次数较少代码则解释执行，避免占用过多内存

2018-07-20



Ryan-Hou

15

在为什么Java要在虚拟机里执行这一节您提到，java语法复杂，抽象度高，直接通过硬件来执行不现实，但是同样作为高级语言为什么C++就可以呢？这个理由作为引入虚拟机这个中间层的原因不是很充分吧

2018-07-20

作者回复

多谢指出！这里的直接运行指的是不经过任何转换(编译)，直接在硬件上跑。即便是C++，也不可以直接运行。

C++的策略是直接编译成目标架构的机器码，Java的策略是编译成一个虚拟架构的机器码。这个虚拟架构可以有物理实现(可以搜Java processor)，也可以是软件实现，也就是我们经常接触到的JRE

。

2018-07-20



陈晨

12

作业终于做出来~\(\ge \nabla \le)/~喜大普奔

asmtools下载地址：

<https://adopt-openjdk.ci.cloudbees.com/view/OpenJDK/job/asmtools/lastSuccessfulBuild/artifact/asmtools-6.0.tar.gz>；

先是在window环境里，awk不能使用，看<https://zh.wikipedia.org/wiki/Awk>，AWK是一种优良的文本处理工具，Linux及Unix环境中现有的功能最强大的数据处理引擎之一,于是转战Linux，

```
[root@localhost cqj]# javac Foo.java
```

```
[root@localhost cqj]# java Foo
```

```
Hello,Java
```

```
Hello,JVM
```

```
[root@localhost cqj]# java -cp /cqj/asmtools.jar org.openjdk.asmtools.jdis.Main Foo.class>Foo.jasm.1
```

```
[root@localhost cqj]# ls
```

```
asmtools.jar Foo.class Foo.jasm.1 Foo.java
```

```
[root@localhost cqj]# vi Foo.jasm.1
```

```
[root@localhost cqj]# awk 'NR==1,/iconst_1/{sub(/iconst_1/,"iconst_2")}' 1' Foo.jasm.1>Foo.jasm
```

```
[root@localhost cqj]# java -cp /cqj/asmtools.jar org.openjdk.asmtools.jasm.Main Foo.jasm
```

```
[root@localhost cqj]# java Foo
```

```
Hello,Java
```

结果为啥是这个看点赞第一的高手；

另外asmtools使用方式还可以这样子：

```
java -jar asmtools.jar jdis Foo.class>Foo.jasm.1
java -jar asmtools.jar jasm Foo.jasm
```

2018-07-25



掌心童话

10

我只想问，你就没有教授的担忧？万一我拿今天的知识点去问面试者，答不上来咋办？

2018-07-21



godtrue

9

1:为什么使用JVM?

1-1:可以轻松实现Java代码的跨平台执行

1-2:JVM提供了一个托管平台，提供内存管理、垃圾回收、编译时动态校验等功能

1-3:使用JVM能够让我们的编程工作更轻松、高效节省公司成本，提示社会化的整体快发效率，我们只关注和业务相关的程序逻辑的编写，其他业务无关但对于编程同样重要的事情交给JVM来处理

2:听完此节的课程的疑惑（之前就没太明白，原期待听完后不再疑惑的）

2-1:Java源代码怎么就经过编译变成了Java字节码？

2-2:JVM怎么就把Java字节码加载进JVM内了？先加载那个类的字节码？它怎么定位的？拿到后怎么解析的？不会整个文件放到一个地方吧？使用的时候又是怎么找到的呢？这些感觉还是黑盒

2-3:JVM将内存区域分成堆和栈，然后将栈分成pc寄存器、本地方法栈、Java方法栈，有些内存空间是线程可共享的，有些是线程私有的。现在也了解不同的内存区块有不同的用处，不过他们是怎么被划分的哪？为什么是他们，不能再多几种或少几种了吗？共享的内存区和私有的又是怎么控制的哪？

2018-07-24

作者回复

总结得非常细致！

2-1 其实是这样的，JVM接收字节码，要运行在JVM上只能选择转化为字节码。要是不想在JVM上跑，可以选择直接转化为机器码。

2-2 类加载会在第三篇详细介绍。

2-3 具体的划分都是实现细节。你也可以选择全部冗杂在一起。但是这样子做性能较高，因为线程私有的可以不用同步。

2018-07-24



踏雪无痕

8

您好，我现在所在的项目经常堆外内存占用非常多，超过总内存的70%，请问一下有没有什么方法能观察一下堆外内存有什么内容？

2018-07-20

作者回复

堆外内存的话，就把JVM当成普通进程来查找内存泄漏。可以看下Google Performance Tools相关资料

2018-07-20



欲风

7

方法区和元空间是一个概念吧，能不能统一说法到jdk8之后的版本 ~

2018-07-20

不出腹肌
不换头像

志文

5

Java 作为一门高级程序语言，它的语法非常复杂，抽象程度非常高，所以不能直接在硬件上执行。所以要引入JAVA 虚拟机。

我觉得理由不充分，JAVA 为什么不能像c++一样直接转成机器码呢？从理论上是可以编译器来实现这个的功能的。问题在于直接像c++那样编译成机器码，就实现不了跨平台了。那么是不是跨平台才是引入JAVA 虚拟机的重要原因呢。请老师解答

2018-07-22



陈树义

5

asmtools.jar 是在哪里下载的，怎么在给的链接页面没找到。

2018-07-21



曾泽浩

5

解释执行和即时执行这里听得有点蒙

2018-07-20



周仕林

4

看到有人说热点代码的区别，在git里面涉及到的热点代码有两种算法，基于采样的热点探测和基于计数器的热点探测。一般采用的都是基于计数器的热点探测，两者的优缺点百度一下就知道了。基于计数器的热点探测又有两个计数器，方法调用计数器，回边计数器，他们在C1和C2又有不同的阈值。????

2018-07-23

作者回复

谢谢！

2018-07-24



临风

4

我跟楼上的novembersky同学一样疑惑，对于性能要求高的web应用，为什么不直接使用即时编译器在启动时全部编译成机器码呢？虽然启动耗时，但是也是可以接受的

2018-07-20

作者回复

通常，对于长时间运行的程序来说，大部分即时编译就发生在前几个小时。

再之后的即时编译主要是一些非热点代码，以及即时编译器中的bug造成的反复去优化重新编译。

2018-07-20



Fyypumpkin

4

老师，问一下这个asmtools是做什么用的

2018-07-20

作者回复

就是Java字节码的反汇编器和汇编器。

2018-07-20



xianhai

3

即时编译生成的代码是只保存在内存中吗？会不会写到磁盘上？如果我怀疑优化后的代码有bug，有办法debug吗？

2018-07-21



那我懂你意思了

3

老师，那个pc寄存器，本地方法栈，以及方法栈，java方法栈这三个组成的就是我们常统称的栈吧，然后也叫栈帧？

2018-07-20

作者回复

JVM里的栈指的应该是Java方法栈和本地方法栈。每个方法调用会在栈上划出一块作为栈帧(stack frame)。栈是由多个栈帧构成的，就好比电影是由一个个帧构成的。

2018-07-20



祁颜·恩和

3

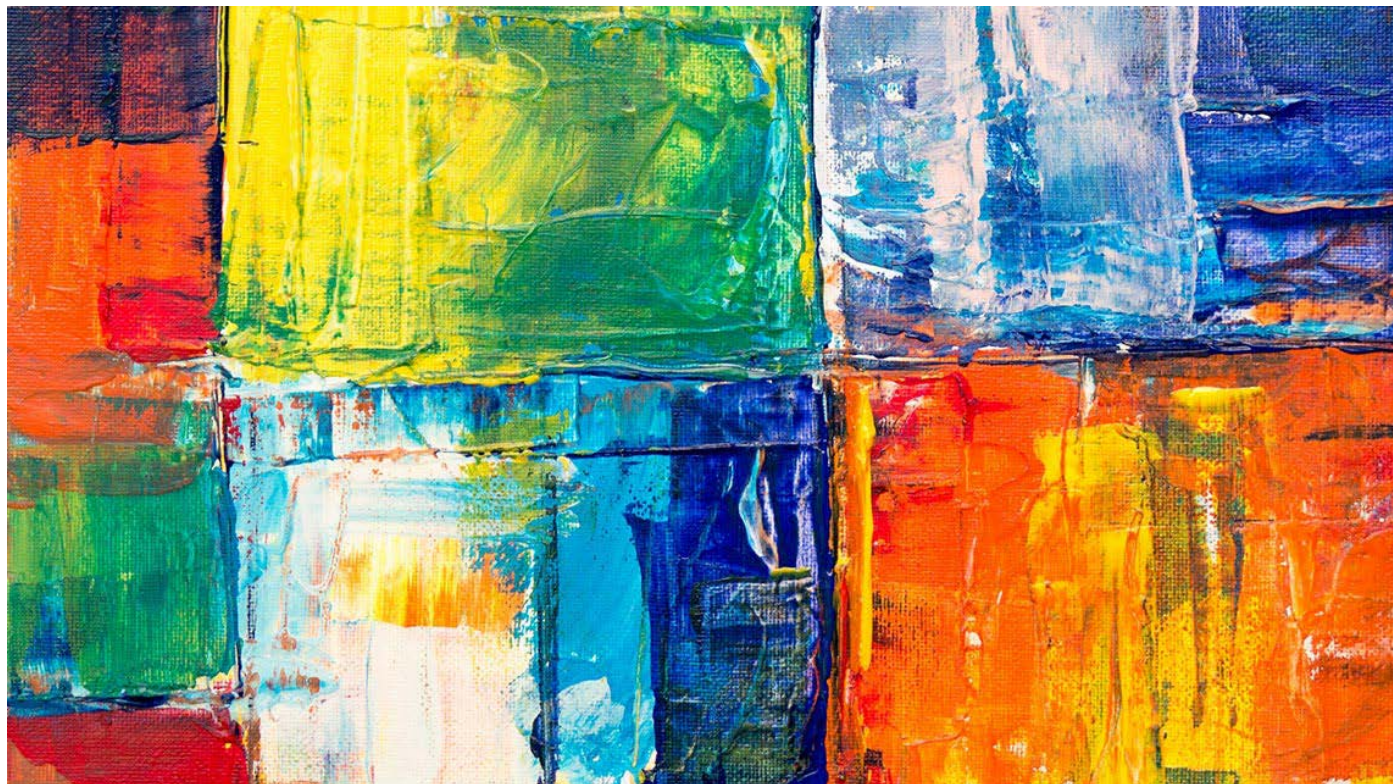
java10的javafx能ppapi吗？我网上没查到该资料。告诉我呗

2018-07-20

讲堂 深入拆解 Java 虚拟机 文章详情

02 | Java的基本类型

2018-07-23 郑雨迪



02 | Java的基本类型

朗读人：郑雨迪 13'26" | 6.16M

如果你了解面向对象语言的发展史，那你可能听说过 Smalltalk 这门语言。它的影响力之大，以至于之后诞生的面向对象语言，或多或少都借鉴了它的设计和实现。

在 Smalltalk 中，所有的值都是对象。因此，许多人认为它是一门纯粹的面向对象语言。

Java 则不同，它引进了八个基本类型，来支持数值计算。Java 这么做的原因主要是工程上的考虑，因为使用基本类型能够在执行效率以及内存使用两方面提升软件性能。

今天，我们就来了解一下基本类型在 Java 虚拟机中的实现。

```
public class Foo {  
    public static void main(String[] args) {  
        boolean 吃过饭没 = 2; // 直接编译的话 javac 会报错  
        if (吃过饭没) System.out.println(" 吃了 ");  
    }  
}
```

```
    if (true == 吃过饭没) System.out.println(" 真吃了 ");  
  }  
}
```

在上一篇结尾的小作业里，我构造了这么一段代码，它将一个 `boolean` 类型的局部变量赋值为 2。为了方便记忆，我们给这个变量起个名字，就叫“吃过饭没”。

赋值语句后边我设置了两个看似一样的 `if` 语句。第一个 `if` 语句，也就是直接判断“吃过饭没”，在它成立的情况下，代码会打印“吃了”。

第二个 `if` 语句，也就是判断“吃过饭没”和 `true` 是否相等，在它成立的情况下，代码会打印“真吃了”。

当然，直接编译这段代码，编译器是会报错的。所以，我迂回了一下，采用一个 Java 字节码的汇编工具，直接对字节码进行更改。

那么问题就来了：当一个 `boolean` 变量的值是 2 时，它究竟是 `true` 还是 `false` 呢？

如果你跑过这段代码，你会发现，问虚拟机“吃过饭没”，它会回答“吃了”，而问虚拟机“真（==）吃过饭没”，虚拟机则不会回答“真吃了”。

那么虚拟机到底吃过没，下面我们来一起分析一下这背后的细节。

Java 虚拟机的 `boolean` 类型

首先，我们来看看 Java 语言规范以及 Java 虚拟机规范是怎么定义 `boolean` 类型的。

在 Java 语言规范中，`boolean` 类型的值只有两种可能，它们分别用符号“`true`”和“`false`”来表示。显然，这两个符号是不能被虚拟机直接使用的。

在 Java 虚拟机规范中，`boolean` 类型则被映射成 `int` 类型。具体来说，“`true`”被映射为整数 1，而“`false`”被映射为整数 0。这个编码规则约束了 Java 字节码的具体实现。

举个例子，对于存储 `boolean` 数组的字节码，Java 虚拟机需保证实际存入的值是整数 1 或者 0。

Java 虚拟机规范同时也要求 Java 编译器遵守这个编码规则，并且用整数相关的字节码来实现逻辑运算，以及基于 `boolean` 类型的条件跳转。这样一来，在编译而成的 `class` 文件中，除了字段和传入参数外，基本看不出 `boolean` 类型的痕迹了。

```
# Foo.main 编译后的字节码  
  
0: iconst_2      // 我们用 AsmTools 更改了这一指令  
  
1: istore_1
```

```
2: iload_1
3: ifeq 14      // 第一个 if 语句, 即操作数栈上数值为 0 时跳转
6: getstatic java.lang.System.out
9: ldc " 吃了 "
11: invokevirtual java.io.PrintStream.println
14: iload_1
15: iconst_1
16: if_icmpne 27 // 第二个 if 语句, 即操作数栈上两个数值不相同同时跳转
19: getstatic java.lang.System.out
22: ldc " 真吃了 "
24: invokevirtual java.io.PrintStream.println
27: return
```

在前面的例子中, 第一个 if 语句会被编译成条件跳转字节码 ifeq, 翻译成成人话就是说, 如果局部变量“吃过饭没”的值为 0, 那么跳过打印“吃了”的语句。

而第二个 if 语句则会被编译成条件跳转字节码 if_icmpne, 也就是说, 如果局部变量的值和整数 1 不相等, 那么跳过打印“真吃了”的语句。

可以看到, Java 编译器的确遵守了相同的编码规则。当然, 这个约束很容易绕开。除了我们小作业中用到的汇编工具 AsmTools 外, 还有许多可以修改字节码的 Java 库, 比如说 ASM [\[1\]](#) 等。

对于 Java 虚拟机来说, 它看到的 boolean 类型, 早已被映射为整数类型。因此, 将原本声明为 boolean 类型的局部变量, 赋值为除了 0、1 之外的整数值, 在 Java 虚拟机看来是“合法”的。

在我们的例子中, 经过编译器编译之后, Java 虚拟机看到的不是在问“吃过饭没”, 而是在问“吃过几碗饭”。也就是说, 第一个 if 语句变成: 你不会一碗饭都没吃吧。第二个 if 语句则变成: 你吃过一碗饭了吗。

如果我们约定俗成, 每人每顿只吃一碗, 那么第二个 if 语句还是有意义的。但如果我们打破常规, 吃了两碗, 那么较真的 Java 虚拟机就会将第二个 if 语句判定为假了。

Java 的基本类型

除了上面提到的 boolean 类型外, Java 的基本类型还包括整数类型 byte、short、char、int 和 long, 以及浮点类型 float 和 double。

类型	值域	默认值	虚拟机内部符号
boolean	{false, true}	false	Z
byte	[-128, 127]	0	B
short	[-32768, 32767]	0	S
char	[0, 65535]	'\u0000'	C
int	$[-2^{31}, 2^{31}-1]$	0	I
long	$[-2^{63}, 2^{63}-1]$	0L	J
float	$\sim[-3.4E38, 3.4E38]$	+0.0F	F
double	$\sim[-1.8E308, 1.8E308]$	+0.0D	D

Java 的基本类型都有对应的值域和默认值。可以看到，byte、short、int、long、float 以及 double 的值域依次扩大，而且前面的值域被后面的值域所包含。因此，从前面的基本类型转换至后面的基本类型，无需强制转换。另外一点值得注意的是，尽管他们的默认值看起来不一样，但在内存中都是 0。

在这些基本类型中，boolean 和 char 是唯二的无符号类型。在不考虑违反规范的情况下，boolean 类型的取值范围是 0 或者 1。char 类型的取值范围则是 [0, 65535]。通常我们可以认定 char 类型的值为非负数。这种特性十分有用，比如说作为数组索引等。

在前面的例子中，我们能够将整数 2 存储到一个声明为 boolean 类型的局部变量中。那么，声明为 byte、char 以及 short 的局部变量，是否也能够存储超出它们取值范围的数值呢？

答案是可以的。而且，这些超出取值范围的数值同样会带来一些麻烦。比如说，声明为 char 类型的局部变量实际上有可能为负数。当然，在正常使用 Java 编译器的情况下，生成的字节码会遵守 Java 虚拟机规范对编译器的约束，因此你无须过分担心局部变量会超出它们的取值范围。

Java 的浮点类型采用 IEEE 754 浮点数格式。以 float 为例，浮点类型通常有两个 0，+0.0F 以及 -0.0F。

前者在 Java 里是 0，后者是符号位为 1、其他位均为 0 的浮点数，在内存中等同于十六进制整数 0x80000000（即 -0.0F 可通过 Float.intBitsToFloat(0x80000000) 求得）。尽管它们的内存数值不同，但是在 Java 中 +0.0F == -0.0F 会返回真。

在有了 +0.0F 和 -0.0F 这两个定义后，我们便可以定义浮点数中的正无穷及负无穷。正无穷就是任意正浮点数（不包括 +0.0F）除以 +0.0F 得到的值，而负无穷是任意正浮点数除以 -0.0F 得到的值。在 Java 中，正无穷和负无穷是有确切的值，在内存中分别等同于十六进制整数 0x7F800000 和 0xFF800000。

你也许会好奇，既然整数 0x7F800000 等同于正无穷，那么 0x7F800001 又对应什么浮点数呢？

这个数字对应的浮点数是 NaN (Not-a-Number) 。

不仅如此, [0x7F800001, 0x7FFFFFFF] 和 [0xFF800001, 0xFFFFFFFF] 对应的都是 NaN。当然, 一般我们计算得出的 NaN, 比如说通过 $+0.0F/+0.0F$, 在内存中应为 0x7FC00000。这个数值, 我们称之为标准的 NaN, 而其他的我们称之为不标准的 NaN。

NaN 有一个有趣的特性: 除了“!=”始终返回 true 之外, 所有其他比较结果都会返回 false。

举例来说, “NaN<1.0F”返回 false, 而“NaN>=1.0F”同样返回 false。对于任意浮点数 f, 不管它是 0 还是 NaN, “f!=NaN”始终会返回 true, 而“f==NaN”始终会返回 false。

因此, 我们在程序里做浮点数比较的时候, 需要考虑上述特性。在本专栏的第二部分, 我会介绍这个特性给向量化比较带来什么麻烦。

Java 基本类型的大小

在第一篇中我曾经提到, Java 虚拟机每调用一个 Java 方法, 便会创建一个栈帧。为了方便理解, 这里我只讨论供解释器使用的解释栈帧 (interpreted frame) 。

这种栈帧有两个主要的组成部分, 分别是局部变量区, 以及字节码的操作数栈。这里的局部变量是广义的, 除了普遍意义下的局部变量之外, 它还包含实例方法的“this 指针”以及方法所接收的参数。

在 Java 虚拟机规范中, 局部变量区等价于一个数组, 并且可以用正整数来索引。除了 long、double 值需要用两个数组单元来存储之外, 其他基本类型以及引用类型的值均占用一个数组单元。

也就是说, boolean、byte、char、short 这四种类型, 在栈上占用的空间和 int 是一样的, 和引用类型也是一样的。因此, 在 32 位的 HotSpot 中, 这些类型在栈上将占用 4 个字节; 而在 64 位的 HotSpot 中, 他们将占 8 个字节。

当然, 这种情况仅存在于局部变量, 而并不会出现在存储于堆中的字段或者数组元素上。对于 byte、char 以及 short 这三种类型的字段或者数组单元, 它们在堆上占用的空间分别为一字节、两字节, 以及两字节, 也就是说, 跟这些类型的值域相吻合。

因此, 当我们将一个 int 类型的值, 存储到这些类型的字段或数组时, 相当于做了一次隐式的掩码操作。举例来说, 当我们把 0xFFFFFFFF (-1) 存储到一个声明为 char 类型的字段里时, 由于该字段仅占两字节, 所以高两位的字节便会被截取掉, 最终存入“\uFFFF”。

boolean 字段和 boolean 数组则比较特殊。在 HotSpot 中, boolean 字段占用一字节, 而 boolean 数组则直接用 byte 数组来实现。为了保证堆中的 boolean 值是合法的, HotSpot 在存储时显式地进行掩码操作, 也就是说, 只取最后一位的值存入 boolean 字段或数组中。

讲完了存储，现在我来讲讲加载。Java 虚拟机的算数运算几乎全部依赖于操作数栈。也就是说，我们需要将堆中的 boolean、byte、char 以及 short 加载到操作数栈上，而后将栈上的值当成 int 类型来运算。

对于 boolean、char 这两个无符号类型来说，加载伴随着零扩展。举个例子，char 的大小为两个字节。在加载时 char 的值会被复制到 int 类型的低二字节，而高二字节则会用 0 来填充。

对于 byte、short 这两个类型来说，加载伴随着符号扩展。举个例子，short 的大小为两个字节。在加载时 short 的值同样会被复制到 int 类型的低二字节。如果该 short 值为非负数，即最高位为 0，那么该 int 类型的值的高二字节会用 0 来填充，否则用 1 来填充。

总结与实践

今天我介绍了 Java 里的基本类型。

其中，boolean 类型在 Java 虚拟机中被映射为整数类型：“true”被映射为 1，而“false”被映射为 0。Java 代码中的逻辑运算以及条件跳转，都是用整数相关的字节码来实现的。

除 boolean 类型之外，Java 还有另外 7 个基本类型。它们拥有不同的值域，但默认值在内存中均为 0。这些基本类型之中，浮点类型比较特殊。基于它的运算或比较，需要考虑 +0.0F、-0.0F 以及 NaN 的情况。

除 long 和 double 外，其他基本类型与引用类型在解释执行的方法栈帧中占用的大小是一致的，但它们在堆中占用的大小确不同。在将 boolean、byte、char 以及 short 的值存入字段或者数组单元时，Java 虚拟机会进行掩码操作。在读取时，Java 虚拟机则会将其扩展为 int 类型。

今天的动手环节，你可以观测一下，将 boolean 类型的值存入字段中时，Java 虚拟机所做的掩码操作。

你可以将下面代码中 boolValue = true 里的 true 换为 2 或者 3，看看打印结果与你的猜测是否符合。

熟悉 Unsafe 的同学，可以使用 Unsafe.putBoolean 和 Unsafe.putByte 方法，看看还会不会做掩码操作。

```
public class Foo {
    static boolean boolValue;

    public static void main(String[] args) {
        boolValue = true; // 将这个 true 替换为 2 或者 3，再看看打印结果

        if (boolValue) System.out.println("Hello, Java!");

        if (boolValue == true) System.out.println("Hello, JVM!");
    }
}
```

```
}  
}
```



版权归极客邦科技所有，未经许可不得转载

精选留言



落...小莫

36

当替换为2的时候无输出

当替换为3的时候打印HelloJava及HelloJVM

猜测是因为将boolean 保存在静态域中,指定了其类型为'Z',当修改为2时取低位最后一位为0,当修改为3时取低位最后一位为1

则说明boolean的掩码处理是取低位的最后一位

2018-07-23

作者回复

对的!

2018-07-24



金蝉子

18

老师可以讲下ASM、Unsafe和CAS的底层原理吗? 这块儿一直是个拦路虎, 谢谢!

2018-07-23

作者回复

ASM你指的是那个字节码工程包吗? 是的话那就是一个提供了字节码抽象的工具, 允许用Java代码来生成或者更改字节码。JDK里也会用到ASM, 用来生成一些适配器什么的。我印象中代码覆盖工具JaCoCo也是用ASM来实现的。

Unsafe就是一些不被虚拟机控制的内存操作的合集。具体想要了解哪个API?

CAS可以理解为原子性的写操作，这个概念来自于底层CPU指令。Unsafe提供了一些cas的Java接口，在即时编译器中我们会将对这些接口的调用替换成具体的CPU指令。

2018-07-24



东方

13

Unsafe.putBoolean和Unsafe.putByte是native实现

putBoolean和putByte也是通过宏SET_FIELD模板出的函数

```
#define SET_FIELD(obj, offset, type_name, x) \
oop p = JNIHandles::resolve(obj); \
*(type_name*)index_oop_from_peld_offset_long(p, offset) = truncate_##type_name(x)
```

unsafe.cpp中定义宏做truncate

```
#define truncate_jboolean(x) ((x) & 1)
#define truncate_jbyte(x) (x)
#define truncate_jshort(x) (x)
#define truncate_jchar(x) (x)
#define truncate_jint(x) (x)
#define truncate_jlong(x) (x)
#define truncate_jfloat(x) (x)
#define truncate_jdouble(x) (x)
```

综上：unsafe.Put*不会对值做修改

getBoolean和getBytes也是通过宏GET_FIELD模板出的函数

```
#define GET_FIELD(obj, offset, type_name, v) \
oop p = JNIHandles::resolve(obj); \
type_name v = *(type_name*)index_oop_from_peld_offset_long(p, offset)
```

综上，unsafe.Get*不会对值做修改

验证：

```
unsafe.putByte(foo, addr, (byte)2); // 设置为: 2
System.out.println(unsafe.getBytes(foo, addr)); // 打印getBytes: 2
System.out.println(unsafe.getBoolean(foo, addr)); // 打印getBoolean: true
```

```
unsafe.putByte(foo, addr, (byte)1); // 设置为: 1
System.out.println(unsafe.getBytes(foo, addr)); // 打印getBytes: 1
System.out.println(unsafe.getBoolean(foo, addr)); // 打印getBoolean: true
```

疑问：
if(foo.βag)判断，使用getPeld Field βag:"Z"，执行逻辑等于： 0 ! = βag
if(foo.getFlag())判断，使用invokevirtual Method getFlag:"()Z"，执行逻辑等于： 0 != ((βag) & 1)

求大神帮忙解答

附getFlag jasm码：
public Method getFlag:"()Z"
stack 1 locals 1
{
 aload_0;
 getPeld Field βag:"Z";
 ireturn;
}

<https://gist.github.com/qudongfang/49635d86882c03e49cff2b0f7d833805>

2018-07-23

作者回复

研究得非常深入！

Unsafe.putBoolean会做掩码，另外方法返回也会对boolean byte char short进行掩码

2018-07-24



Kyle

9

老师，文中看你说到：
“也就是说，boolean、byte、char、short 这四种类型，在栈上占用的空间和 int 是一样的，和引用类型也是一样的。因此，在 32 位的 HotSpot 中，这些类型在栈上将占用 4 个字节；而在 64 位的 HotSpot 中，他们将占 8 个字节。”

但是我记得boolean在内存中占1字节，char占2字节，这里是什么个意思？

2018-07-23

作者回复

你说的是在堆里的情况。在解释器栈上是不一样的。至于原因吗，主要是变长数组不好控制，所以就选择浪费一些空间，以便访问时直接通过下标来计算地址。

2018-07-24



Geek_dde3ac

7

你好，在内存中都是0，那么是如何区别是哪种类型数据的呢？

2018-07-23

作者回复

内存中是不做区分的。Java程序想要把它解读成什么类型，它就是什么类型。

2018-07-24



杨春鹏

3

局部变量中基本数据类型存储在栈中，变量的变量名(引用符号)和变量值(字面量)都存储在栈中。局部变量中引用数据类型的引用地址存储在栈中，对象的实例数据存储在堆中，类型数据存储在方法区

全局变量的基本数据类型和引用数据类型，都存储在堆中。

不知理解的是否正确

2018-07-26



吴天

2

char在什么情况下或那些语言下是占用一个字节？英文字符？似乎还分什么内码外码 这是什么？

2018-07-24



志远

2

NaN 有一个有趣的特性：除了“!= 始终返回 true”之外，所有其他比较结果都会返回 false。这句话好拗口啊，双引号的标点符号有问题吧

2018-07-23

作者回复

多谢指出！应该是 “!=“

2018-07-24



取个好名字

2

下载asm.jar那个，点开右边下载链接是个jenkins页面，已经显示有构建好的包，点击就直接下了

2018-07-23

作者回复

谢谢！

2018-07-24



Ennis LM

1

因此，当我们将一个 int 类型的值，存储到这些类型的字段或数组时，相当于做了一次隐式的掩码操作。

为了保证堆中的 boolean 值是合法的，HotSpot 在存储时显式地进行掩码操作，也就是说，只取最后一位的值存入 boolean 字段或数组中。

为什么一个是隐式一个是显式

2018-07-28



梧桐树

1

如果64位处理器，long 和double和其它类型一样都是8个字节码在栈中

2018-07-26



黄小建儿??

基本类型存在栈里，包装类存储在堆上吧？

2018-07-24

1



志远

老师，文章提到局部变量会超出它们的取值范围，为什么无须担心啊，虚拟机会如何处理超过范围的值啊，为什么无须担心？

2018-07-24

1



落叶飞逝的恋

其实那个boolean的true虚拟机里面为1，也就是if(true==吃了没)其实可以替换成if(1==2)这样理解吧

2018-07-24

1

作者回复

对的！

2018-07-24



Eid

<https://adopt-openjdk.ci.cloudbees.com/view/OpenJDK/job/asmtools/lastSuccessfulBuild/artifact/asmtools-6.0.tar.gz>

2018-07-23

1

作者回复

多谢！

2018-07-24



羊飞

老师，我昨天那个好像试的不对。。。我是直接bool=true,抱歉。。。忽略我之前的话??

2018-07-23

1

作者回复

赞！抱歉回复得不及时

2018-07-24



NEO??

老师 上节课说的asmtool没有找到在哪里下载

2018-07-23

1



北风一叶

想问下，基本类型会存储到堆中么？什么情况会存呢？

2018-08-15

0



流水光阴

为啥我这里编译都不通过啊～

0

2018-08-14



And

后面讲boolean以外的类型我都看不懂了

0

2018-08-11



Page

因为刚开始接触jvm理解起来比较吃力，有没有配合的资料推荐，更好的理解现在讲的内容。

0

2018-08-10



QIDoors

```
Float sn1 = 0.0f / 0.0f;  
Float f1 = 0.0f;  
Float sn2 = f1 / f1;  
System.out.println("sn1 hex = " + Integer.toHexString(Float.floatToIntBits(sn1)));  
System.out.println("sn2 hex = " + Integer.toHexString(Float.floatToIntBits(sn2)));
```

0

老师，请问这段代码为何输出是

sn1 hex = 7fc00000

sn2 hex = ffc00000

我的运行环境是idea jdk8

2018-08-09

作者回复

这个我之前还真不知道。

看了一下字节码和即时编译后的代码，sn1是由Java编译器给出的，也就是说Java编译器认为0/0是0x7FC00000。sn2的值是CPU用DIVSS指令计算出来的。至于为什么算出这个值，应该跟CPU的实现有关。

由于都是NaN，理论上用哪个值都是正确的。

2018-08-10



一个坏人

老师好，请教一个问题。文中提到“对于 byte、short 这两个类型来说，加载伴随着符号扩展。举个例子，short 的大小为两个字节。在加载时 short 的值同样会被复制到 int 类型的低二字节。如果该 short 值为非负数，即最高位为 0，那么该 int 类型的值的高二字节会用 0 来填充，否则用 1 来填充。”在我的认知里，计算机内部是用补码存储的，所以short如果是负数，不应该是简单的被复制到int的低二字节，最高位是1，其余高位字节是0吧？？

0

2018-08-08



猫和芝士蛋糕

0

0x8000000 (即 -0.0F 可通过 Float.intBitsToFloat(0x8000000) 求得

这里的两个十六进制数是不是都少了1位啊，怎么查都是只有7位

2018-08-03



Bale

0

时间原因才学到第二节，@东方，大佬的笔记受益甚多，最后getFlag 因为方法返回值是boolean 因此 从结果看 方法返回结果进一步做了一次掩码，也就是 永远返回最后一位 所以结果和Bag&1是一样的

2018-08-01



Ennis LM

0

前者在 Java 里是 0，后者是符号位为 1、其他位均为 0 的浮点数，在内存中等同于十六进制整数 0x8000000 (即 -0.0F 可通过 Float.intBitsToFloat(0x8000000) 求得)。

这里少写了一个0吧

2018-07-28



李阳

0

```
Float res = 0x7F800001;  
System.out.println(res);  
int resInt = 0x7F800001;  
System.out.println(resInt);
```

输出：

2.13909504E9

2139095041

也没有出现NaN呀

2018-07-28



dong

0

感觉"吃饭了"例子，弄得有点饶了。也有些地方语句的起承转合不是很通顺，个人理解。

2018-07-27

作者回复

谢谢建议

2018-07-27



rainbow

0

最后例子里，true改为2或3是直接源代码里改吗？编译报类型错误啊

2018-07-27

作者回复

不能在源代码更好，需要借助ASMTTools，可以参考1的课后习题

2018-07-27



阿康

0

通常我们可以认定 char 类型的值为非负数。这种特性十分有用，比如说作为数组索引等。

这句话不是很理解？

2018-07-26



倪有才

0

快点更新呀

2018-07-26



jerry

0

老师，我用javap命令看编译出来的class文件，方法里面的本地变量是用槽（slot）来标示的，double两个槽，int一个槽，boolean也是一个槽。这个槽就是4字节的结构吧？boolean, short都占一个槽，所以在栈空间中就是4字节？

2018-07-26



沉淀的梦想

0

换了台电脑就可以复现你们的结果了，在我自己的电脑上死活复现不出来，我怀疑是不是有的硬件不支持1位的掩码？

2018-07-26



沉淀的梦想

0

有个疑问，文中说，在64位的HotSpot的栈上，一个byte, char, short, int都占8个字节，而一个double, long将占16个字节。但是在64位虚拟机上面的int的取值范围也还是只有4字节的取值范围，long也还是只有8字节的取值范围，那为什么不直接把long, double和int它们统一都只占数组的一个单元(即8字节)，而非要让double和long占两个数组单元呢？这么实现的意义是什么呢？

2018-07-26



张鑫

0

基本数据类型内存不是直接分配到栈内存空间吗？boolean类型占一个字节，char类型占两个字节，怎么我看上面回复还有分配在堆内存空间？

2018-07-26



沉淀的梦想

0

为什么我替换成2和3，但是现象和第一讲的时候一样呢？都只能打印出一个Hello Java，我反复检查也没发现什么问题，这有可能是是什么原因呢？

2018-07-25



Z_Winner

0

所以是不是boolean其实和c语言的boolean差不多，但是java故意限制了boolean与其他基本数据类型互相转换

2018-07-25



等待

0

请教问题，比如在一个方法中声明了一个引用类型，这个引用类型会指向一个堆里的一个对象，假如我声明了一个`int i = 1`，那这个1是从某个地方拷贝到局部变量表，还是说有其他的机制产生一个1，还有就是类的属性是基本数据类型的情况和方法中的局部变量是基本数据类型的赋值细节是一样的吗？

2018-07-25



落叶飞逝的恋

0

作者你好，能单独讲下指令码吗？谢谢

2018-07-24



无泪

0

栈的地方如果用图形表示一下就好理解了

2018-07-24



别处

0

以下两个引至本文内容：

- 1、在 Java 虚拟机规范中，boolean 类型则被映射成 int 类型。
- 2、在 HotSpot 中，boolean 字段占用一字节，

问题：一个是int类型，一个是一个字节(32位系统的话就是byte类型)，是没讲透还是错误？

2018-07-24

作者回复

多谢指出。严格来说，前者指的是计算的时候被映射成int类型，后者指的是存储到堆中是一个字节。

2018-07-24



渺渺兮于怀

0

“第二个 if 语句则会被编译成条件跳转字节码 `if_icmpne`，也就是说，如果局部变量的值和整数 1 不相等，那么跳过打印“真吃了”的语句。”

不相等，应该是不打印“真吃了”的语句，这里是不是想表达，阐述字节码执行顺序过程呀？

2018-07-24

作者回复

对的，字节码里是跳转。

2018-07-24



Living as a naturalist

0

老师，百度没有找到AsmTools这个工具，是你们特用的吗

2018-07-24

作者回复

是openjdk一个项目里的工具，开源的

2018-07-24



Oswwww

0

char -> character -> /'kerə/

2018-07-24

作者回复

多谢指出！其实我在录音前还特地去查了下，选了个看起来用得多的。

2018-07-24



羊飞

0

老师，我昨天试的时候两个都打印出来了

2018-07-23



乔伟

0

Double long 占用两个数组单位 64位的数组单位是8个字节 是不是说在解释栈上面 64位的情况下 double占用16个字节？

2018-07-23

作者回复

是的。占用但并没有用高八字节的空间。这个属于HotSpot的实现细节，偏向于快速访问而牺牲了耗费空间。

2018-07-24



godtrue

0

使用基本类型能够在执行效率以及内存使用两方面提升软件性能？具体是什么原理呢？占的空间更小，不需要类型转换吗？

2018-07-23

作者回复

对的！

2018-07-24

讲堂 深入拆解 Java 虚拟机 文章详情

03 | Java虚拟机是如何加载 Java类的？

2018-07-25 郑雨迪



03 | Java虚拟机是如何加载Java类的？

朗读者：郑雨迪 12'38" | 5.80M

听我的意大利同事说，他们那边有个习俗，就是父亲要帮儿子盖栋房子。

这事要放在以前还挺简单，亲朋好友搭把手，盖个小砖房就可以住人了。现在呢，整个过程要耗费好久的时间。首先你要请建筑师出个方案，然后去市政部门报备、验证，通过后才可以开始盖房子。盖好房子还要装修，之后才能住人。

盖房子这个事，和 Java 虚拟机中的类加载还是挺像的。从 class 文件到内存中的类，按先后顺序需要经过加载、链接以及初始化三大步骤。其中，链接过程中同样需要验证；而内存中的类没有经过初始化，同样不能使用。那么，是否所有的 Java 类都需要经过这几步呢？

我们知道 Java 语言的类型可以分为两大类：基本类型（primitive types）和引用类型（reference types）。在上一篇中，我已经详细介绍过了 Java 的基本类型，它们是由 Java 虚拟机预先定义好的。

至于另一大类引用类型，Java 将其细分为四种：类、接口、数组类和泛型参数。由于泛型参数会在

编译过程中被擦除（我会在专栏的第二部分详细介绍），因此 Java 虚拟机实际上只有前三种。在类、接口和数组类中，数组类是由 Java 虚拟机直接生成的，其他两种则有对应的字节流。

说到字节流，最常见的形式要属由 Java 编译器生成的 class 文件。除此之外，我们也可以在程序内部直接生成，或者从网络中获取（例如网页中内嵌的小程序 Java applet）字节流。这些不同形式的字节流，都会被加载到 Java 虚拟机中，成为类或接口。为了叙述方便，下面我就用“类”来统称它们。

无论是直接生成的数组类，还是加载的类，Java 虚拟机都需要对其进行链接和初始化。接下来，我会详细给你介绍一下每个步骤具体都在干些什么。

加载

加载，是指查找字节流，并且据此创建类的过程。前面提到，对于数组类来说，它并没有对应的字节流，而是由 Java 虚拟机直接生成的。对于其他的类来说，Java 虚拟机则需要借助类加载器来完成查找字节流的过程。

以盖房子为例，村里的 Tony 要盖个房子，那么按照流程他得先找个建筑师，跟他说想要设计一个房型，比如说“一房、一厅、四卫”。你或许已经听出来了，这里的房型相当于类，而建筑师，就相当于类加载器。

村里有许多建筑师，他们等级森严，但有着共同的祖师爷，叫启动类加载器（boot class loader）。启动类加载器是由 C++ 实现的，没有对应的 Java 对象，因此在 Java 中只能用 null 来指代。换句话说，祖师爷不喜欢像 Tony 这样的小角色来打扰他，所以谁也没有祖师爷的联系方式。

除了启动类加载器之外，其他的类加载器都是 `java.lang.ClassLoader` 的子类，因此有对应的 Java 对象。这些类加载器需要先由另一个类加载器，比如说启动类加载器，加载至 Java 虚拟机中，方能执行类加载。

村里的建筑师有一个潜规则，就是接到单子自己不能着手干，得先给师傅过过目。师傅不接手的情况下，才能自己来。在 Java 虚拟机中，这个潜规则有个特别的名字，叫双亲委派模型。每当一个类加载器接收到加载请求时，它会先将请求转发给父类加载器。在父类加载器没有找到所请求的类的情况下，该类加载器才会尝试去加载。

在 Java 9 之前，启动类加载器负责加载最为基础、最为重要的类，比如存放在 JRE 的 lib 目录下 jar 包中的类（以及由虚拟机参数 `-Xbootclasspath` 指定的类）。除了启动类加载器之外，另外两个重要的类加载器是扩展类加载器（extension class loader）和应用类加载器（application class loader），均由 Java 核心类库提供。

扩展类加载器的父类加载器是启动类加载器。它负责加载相对次要、但又通用的类，比如存放在

JRE 的 lib/ext 目录下 jar 包中的类（以及由系统变量 java.ext.dirs 指定的类）。

应用类加载器的父类加载器则是扩展类加载器。它负责加载应用程序路径下的类。（这里的应用程序路径，便是指虚拟机参数 -cp/-classpath、系统变量 java.class.path 或环境变量 CLASSPATH 所指定的路径。）默认情况下，应用程序中包含的类便是由应用类加载器加载的。

Java 9 引入了模块系统，并且略微更改了上述的类加载器¹。扩展类加载器被改名为平台类加载器（platform class loader）。Java SE 中除了少数几个关键模块，比如说 java.base 是由启动类加载器加载之外，其他的模块均由平台类加载器所加载。

除了由 Java 核心类库提供的类加载器外，我们还可以加入自定义的类加载器，来实现特殊的加载方式。举例来说，我们可以对 class 文件进行加密，加载时再利用自定义的类加载器对其解密。

除了加载功能之外，类加载器还提供了命名空间的作用。这个很好理解，打个比方，咱们这个村不讲究版权，如果你剽窃了另一个建筑师的设计作品，那么只要你标上自己的名字，这两个房型就是不同的。

在 Java 虚拟机中，类的唯一性是由类加载器实例以及类的全名一同确定的。即便是同一串字节流，经由不同的类加载器加载，也会得到两个不同的类。在大型应用中，我们往往借助这一特性，来运行同一个类的不同版本。

链接

链接，是指将创建成的类合并至 Java 虚拟机中，使之能够执行的过程。它可分为验证、准备以及解析三个阶段。

验证阶段的目的是，在于确保被加载类能够满足 Java 虚拟机的约束条件。这就好比 Tony 需要将设计好的房型提交给市政部门审核。只有当审核通过，才能继续下面的建造工作。

通常而言，Java 编译器生成的类文件必然满足 Java 虚拟机的约束条件。因此，这部分我留到讲解字节码注入时再详细介绍。

准备阶段的目的，则是为被加载类的静态字段分配内存。Java 代码中对静态字段的具体初始化，则会在稍后的初始化阶段中进行。过了这个阶段，咱们算是盖好了毛坯房。虽然结构已经完整，但是在没有装修之前是不能住人的。

除了分配内存外，部分 Java 虚拟机还会在此阶段构造其他跟类层次相关的数据结构，比如说用来实现虚方法的动态绑定的方法表。

在 class 文件被加载至 Java 虚拟机之前，这个类无法知道其他类及其方法、字段所对应的具体地址，甚至不知道自己方法、字段的地址。因此，每当需要引用这些成员时，Java 编译器会生成一个符号引用。在运行阶段，这个符号引用一般都能够无歧义地定位到具体目标上。

举例来说，对于一个方法调用，编译器会生成一个包含目标方法所在类的名字、目标方法的名字、接收参数类型以及返回值类型的符号引用，来指代所要调用的方法。

解析阶段的目的，正是将这些符号引用解析成为实际引用。如果符号引用指向一个未被加载的类，或者未被加载类的字段或方法，那么解析将触发这个类的加载（但未必触发这个类的链接以及初始化。）

如果将这段话放在盖房子的语境下，那么符号引用就好比“Tony 的房子”这种说法，不管它存在不存在，我们都可以用这种说法来指代 Tony 的房子。实际引用则好比实际的通讯地址，如果我们想要与 Tony 通信，则需要启动盖房子的过程。

Java 虚拟机规范并没有要求在链接过程中完成解析。它仅规定了：如果某些字节码使用了符号引用，那么在执行这些字节码之前，需要完成对这些符号引用的解析。

初始化

在 Java 代码中，如果要初始化一个静态字段，我们可以在声明时直接赋值，也可以在静态代码块中对其赋值。

如果直接赋值的静态字段被 `final` 所修饰，并且它的类型是基本类型或字符串时，那么该字段便会被 Java 编译器标记成常量值（`ConstantValue`），其初始化直接由 Java 虚拟机完成。除此之外的直接赋值操作，以及所有静态代码块中的代码，则会被 Java 编译器置于同一方法中，并把它命名为 `<clinit>`。

类加载的最后一步是初始化，便是为标记为常量值的字段赋值，以及执行 `<clinit>` 方法的过程。Java 虚拟机会通过加锁来确保类的 `<clinit>` 方法仅被执行一次。

只有当初始化完成之后，类才正式成为可执行的状态。这放在我们盖房子的例子中就是，只有当房子装修过后，Tony 才能真正地住进去。

那么，类的初始化何时会被触发呢？JVM 规范枚举了下述多种触发情况：

1. 当虚拟机启动时，初始化用户指定的主类；
2. 当遇到用以新建目标类实例的 `new` 指令时，初始化 `new` 指令的目标类；
3. 当遇到调用静态方法的指令时，初始化该静态方法所在的类；
4. 当遇到访问静态字段的指令时，初始化该静态字段所在的类；
5. 子类的初始化会触发父类的初始化；
6. 如果一个接口定义了 `default` 方法，那么直接实现或者间接实现该接口的类的初始化，会触发该接口的初始化；
7. 使用反射 API 对某个类进行反射调用时，初始化这个类；
8. 当初次调用 `MethodHandle` 实例时，初始化该 `MethodHandle` 指向的方法所在的类。


```
public class Singleton {  
    private Singleton() {}  
  
    private static class LazyHolder {  
        static final Singleton INSTANCE = new Singleton();  
    }  
  
    public static Singleton getInstance() {  
        return LazyHolder.INSTANCE;  
    }  
}
```

我在文章中贴了一段代码，这段代码是在著名的单例延迟初始化例子中²，只有当调用 `Singleton.getInstance` 时，程序才会访问 `LazyHolder.INSTANCE`，才会触发对 `LazyHolder` 的初始化（对应第 4 种情况），继而新建一个 `Singleton` 的实例。

由于类初始化是线程安全的，并且仅被执行一次，因此程序可以确保多线程环境下有且仅有一个 `Singleton` 实例。

总结与实践

今天我介绍了 Java 虚拟机将字节流转化为 Java 类的过程。这个过程可分为加载、链接以及初始化三大步骤。

加载是指查找字节流，并且据此创建类的过程。加载需要借助类加载器，在 Java 虚拟机中，类加载器使用了双亲委派模型，即接收到加载请求时，会先将请求转发给父类加载器。

链接，是指将创建成的类合并至 Java 虚拟机中，使之能够执行的过程。链接还分验证、准备和解析三个阶段。其中，解析阶段为非必须的。

初始化，则是为标记为常量值的字段赋值，以及执行 `< clinit >` 方法的过程。类的初始化仅会被执行一次，这个特性被用来实现单例的延迟初始化。

今天的实践环节，你可以来验证一下本篇中的理论知识。

通过 JVM 参数 `-verbose:class` 来打印类加载的先后顺序，并且在 `LazyHolder` 的初始化方法中打印特定字样。在命令行中运行下述指令（不包含提示符 `$`）：

```
$ echo '  
public class Singleton {  
    private Singleton() {}
```

```
private static class LazyHolder {  
    static final Singleton INSTANCE = new Singleton();  
  
    static {  
        System.out.println("LazyHolder.<clinit>");  
    }  
}  
  
public static Object getInstance(boolean flag) {  
    if (flag) return new LazyHolder[2];  
    return LazyHolder.INSTANCE;  
}  
  
public static void main(String[] args) {  
    getInstance(true);  
    System.out.println("----");  
    getInstance(false);  
}  
}' > Singleton.java  
$ javac Singleton.java  
$ java -verbose:class Singleton
```

问题 1：新建数组（第 11 行）会导致 LazyHolder 的加载吗？会导致它的初始化吗？

在命令行中运行下述指令（不包含提示符 \$）：

```
$ java -cp /path/to/asmttools.jar org.openjdk.asmttools.jdis.Main Singleton\LazyHolder.class >  
Singleton\LazyHolder.jasm.1  
$ awk 'NR==1,/stack 1/{sub(/stack 1/, "stack 0")}' 1' Singleton\LazyHolder.jasm.1 >  
Singleton\LazyHolder.jasm  
$ java -cp /path/to/asmttools.jar org.openjdk.asmttools.jasm.Main Singleton\LazyHolder.jasm  
$ java -verbose:class Singleton
```

问题 2：新建数组会导致 LazyHolder 的链接吗？



版权归极客邦科技所有，未经许可不得转载

精选留言



东方

24

1. 新建数组会加载元素类LazyHolder；不会初始化元素类
2. 新建数组不会链接元素类LazyHolder；在getInstance(false)时才真正链接和初始化

链接的第一步：验证字节码，awk把字节码改为不符合jvm规范
初始化调用<clinit>(即class init)

PS:好像二个问题包含了第一个问题的答案

2018-07-25

作者回复

多谢指出！

2018-07-25



mover

11

到目前为止，讲解的内容没有超出周志明老师的 深入理解JAVA 虚拟机这本书的内容，老师可以讲解的更深入一点吗？可以介绍一下类加载后在meta区的大概布局吗？class类对象与meta区的类数据结构是什么关系？当我们创建类，使用类时，类实例，类对象，meta区类数据结构是如何交互的？

2018-07-25

作者回复

谢谢你的建议！前几章不好搞太难，希望后面能够满足你的需求

2018-07-25



ruyin

-XX: +TraceClassLoading 可以看到类加载过程

2018-07-27

3



吴天

每次new一个类都是一次初始化吧？加载和链接以后生成的是什么样的数据结构？存储在什么地方？

2018-07-25

作者回复

类的初始化只会发生一次，你可能指的是实例的初始化？

3

JVM并不会直接使用.class文件，类加载链接的目的就是在JVM中创建相应的类结构，会存储在元空间(我之前用的老说法“方法区”，感谢某同学指出)。

2018-07-25



L.B.Q.Y

从大的方面讲，类加载的结果是把一段字节流变换成Class结构并写方法区，实际写方法区具体是发生在加载、链接、初始化的哪个环节呢？

2018-07-25

作者回复

在加载阶段就已经生成class结构了，所以我认为应该已经写入了方法区，只是被标记为未链接而暂不能使用。

2018-07-25

3



Geek_ecc6af

老师，你说可以通过不同的类加载器加载同一个类得到类的不同版本，我有个疑问，类是通过包名加类名来使用的，那怎么样区分不同的类加载器加载的类呢？

2018-07-25

作者回复

你指的是在写代码的时候如何区分对吧？我认为没法区分。如果你有一个类的两个不同版本，而且它们不兼容，那么编译时指向哪个，就按哪个来编译。也就是说，如果要同时使用两个版本，那么你需要分开编译。

2018-07-25

3



迈克播

学习了！可以的话希望老师能附上一些图解，便于更理解

2018-07-29

2



Geek_dde3ac

请问有什么办法或者工具可以看到类加载的这些过程呢？

2

2018-07-25



志远

2

请问，关于双亲委派类加载器的情况，与周志明的深入理解java虚拟机矛盾啊，到底听哪个呢？周志明书籍中lib/ext是由扩展类加载器加载，你这里是启动类加载器加载，到底哪个是正确的呢？

2018-07-25

作者回复

lib/ext是由扩展类加载器加载的，我文中应该也是这样讲的。

2018-07-25



仲伦

1

忍着瞌睡把内容看完了。

全是复习了一遍。

作者对 类加载中的 链接(验证、准备、解析)讲解不太到位吧？

另外，对一个的初始化发生在第一次主动使用该类时，作者列出的几种情况都属于主动使用类。感觉应该有被动使用的举例，并告知大家这样做是不会执行初始化的。

2018-07-27

作者回复

多谢建议！

2018-07-27



小蛋壳

1

加载阶段都加载哪些类呢，那么多类，全部加载吗？

2018-07-26

作者回复

加载阶段是针对单个类的，一般用到的类才会被加载。大部分情况下，不同类的加载阶段是不同的

。

2018-07-27



scutware

1

您在评论回复说.class在加载后已经写入方法区（元空间），但是我理解在方法区里类代码的方法调用应该是实际的调用地址吧？而取得实际调用地址不是在链接阶段吗？这里不太理解，求解答～

2018-07-26

作者回复

链接时取得的不是被加载类的地址，而且被加载类所调用的其它方法的地址

2018-07-27



L.B.Q.Y

1

如果class结构是加载阶段写入方法区的，那么如果后面的链接阶段验证失败的话，还需要把已经写入方法区的class擦除是吗

2018-07-25



熊猫酒仙

1

有几个疑问，请老师指点迷津。

- 1.扩展类加载器的父类，是启动类加载器，而后者是C++实现的，java继承C++的类？不大能理解。
- 2.虚方法的概念在C++中有了解过，java中的虚方法该如何定义呢？以前没接触过java虚方法的概念
- 3.我以前的理解是，有一个零值(0/null)初始化，针对于类的静态成员变量，如果是final修饰的静态成员变量，也就是常量，是初始化为代码中指定的值比如10。非final修饰的静态成员变量，在clinit执行过程中赋值为代码中指定的值，请问老师是这样的吗？

2018-07-25

作者回复

- 1. 可能我翻译得有点瑕疵，导致了你的误解。这里我指的是扩展器类的 父-类加载器，而不是父类-(加载器)。
- 2. Java中所有的非私有实例方法，都算是虚方法。调用这些方法的指令，也区分直接调用和虚调用。下一篇我会讲到。
- 3. 赞一个。被final修饰的静态成员变量，如果不是基本类型或者字符串，也会放在clinit 来做。

2018-07-25



左岸

1

“除此之外的直接赋值操作，以及所有静态代码块中的代码，则会被 Java 编译器置于同一方法中，并把它命名为”。请问一下这里是命名是什么？

2018-07-25

作者回复

小于号 clinit 大于号

我手机上看没问题，会不会哪个浏览器给识别成标签了？

2018-07-25



猿工匠

0

学习了，由浅入深

2018-08-16



一个坏人

0

老师好，请教一个问题。“在加载阶段就已经生成class结构了，所以我认为应该已经写入了方法区，只是被标记为未链接而暂不能使用。” 如果验证失败怎么办？

2018-08-09

作者回复

验证失败虚拟机会报错退出。

2018-08-10



一个坏人

0

老师好，请假一个问题。“Java 虚拟机规范并没有要求在链接过程中完成解析。它仅规定了：如果某些字节码使用了符号引用，那么在执行这些字节码之前，需要完成对这些符号引用的解析。” 这个 解析符合引用的具体过程是？当前阶段还是没法知道方法具体的信息，感觉还是只能是符号引

用？谢谢老师。

2018-08-09



王刚

0

你好，老师，我想问一个问题～类加载的过程可以总结为：

- 1、加载class字节码文件到虚拟机中
- 2、验证
- 3、准备
- 4、解析
- 5、初始化

我想问的是在加载class文件时，在没有到达虚拟机方法区中时，是不是已经完成了验证？只有验证通过的class字节码文件才会加载到方法区中？

2018-08-04



|

0

最近碰到一个问题，想请教下作者

tomcat是自定义类加载器加载应用类的，我用的是8.5版本，但是使用并行流的时候，获取并行流中的类加载器返回的是null，这个是因为什么啊

2018-08-03

讲堂 深入拆解 Java 虚拟机 文章详情

04 | JVM是如何执行方法调用的？（上）

2018-07-27 郑雨迪



04 | JVM是如何执行方法调用的？（上）

朗读人：郑雨迪 13'37" | 6.25M

前不久在写代码的时候，我不小心踩到一个可变长参数的坑。你或许已经猜到了，它正是可变长参数方法的重载造成的。（注：官方文档建议避免重载可变长参数方法，见 [1] 的最后一段。）

我把踩坑的过程放在了文稿里，你可以点击查看。

```
void invoke(Object obj, Object... args) { ... }  
void invoke(String s, Object obj, Object... args) { ... }  
  
invoke(null, 1);    // 调用第二个 invoke 方法  
invoke(null, 1, 2); // 调用第二个 invoke 方法  
invoke(null, new Object[]{1}); // 只有手动绕开可变长参数的语法糖，  
                                // 才能调用第一个 invoke 方法
```

当时情况是这样子的，某个 API 定义了两个同名的重载方法。其中，第一个接收一个 Object，以及声明为 Object...的变长参数；而第二个则接收一个 String、一个 Object，以及声明为 Object...的变长参数。

这里我想调用第一个方法，传入的参数为 (null, 1)。也就是说，声明为 Object 的形式参数所对应的实际参数为 null，而变长参数则对应 1。

通常来说，之所以不提倡可变长参数方法的重载，是因为 Java 编译器可能无法决定应该调用哪个目标方法。

在这种情况下，编译器会报错，并且提示这个方法调用有二义性。然而，Java 编译器直接将我的方法调用识别为调用第二个方法，这究竟是因为什么呢？

带着这个问题，我们来看一看 Java 虚拟机是怎么识别目标方法的。

重载与重写

在 Java 程序里，如果同一个类中出现多个名字相同，并且参数类型相同的方法，那么它无法通过编译。也就是说，在正常情况下，如果我们想要在同一个类中定义名字相同的方法，那么它们的参数类型必须不同。这些方法之间的关系，我们称之为重载。

小知识：这个限制可以通过字节码工具绕开。也就是说，在编译完成之后，我们可以再向 class 文件中添加方法名和参数类型相同，而返回类型不同的方法。当这种包括多个方法名相同、参数类型相同，而返回类型不同的方法的类，出现在 Java 编译器的用户类路径上时，它是怎么确定需要调用哪个方法的呢？当前版本的 Java 编译器会直接选取第一个方法名以及参数类型匹配的方法。并且，它会根据所选取方法的返回类型来决定可不可以通过编译，以及需不需要进行值转换等。

重载的方法在编译过程中即可完成识别。具体到每一个方法调用，Java 编译器会根据所传入参数的声明类型（注意与实际类型区分）来选取重载方法。选取的过程共分为三个阶段：

1. 在不考虑对基本类型自动装拆箱（auto-boxing, auto-unboxing），以及可变长参数的情况下选取重载方法；
2. 如果在第 1 个阶段中没有找到适配的方法，那么在允许自动装拆箱，但不允许可变长参数的情况下选取重载方法；
3. 如果在第 2 个阶段中没有找到适配的方法，那么在允许自动装拆箱以及可变长参数的情况下选取重载方法。

如果 Java 编译器在同一个阶段中找到了多个适配的方法，那么它会在其中选择一个最为贴切的，而决定贴切程度的一个关键就是形式参数类型的继承关系。

在开头的例子中，当传入 `null` 时，它既可以匹配第一个方法中声明为 `Object` 的形式参数，也可以匹配第二个方法中声明为 `String` 的形式参数。由于 `String` 是 `Object` 的子类，因此 Java 编译器会认为第二个方法更为贴切。

除了同一个类中的方法，重载也可以作用于这个类所继承而来的方法。也就是说，如果子类定义了与父类中非私有方法同名的方法，而且这两个方法的参数类型不同，那么在子类中，这两个方法同样构成了重载。

那么，如果子类定义了与父类中非私有方法同名的方法，而且这两个方法的参数类型相同，那么这两个方法之间又是什么关系呢？

如果这两个方法都是静态的，那么子类中的方法隐藏了父类中的方法。如果这两个方法都不是静态的，且都不是私有的，那么子类的方法重写了父类中的方法。

众所周知，Java 是一门面向对象的编程语言，它的一个重要特性便是多态。而方法重写，正是多态最重要的一种体现方式：它允许子类在继承父类部分功能的同时，拥有自己独特的行为。

打个比方，如果你经常漫游，那么你可能知道，拨打 10086 会根据你当前所在地，连接到当地的客服。重写调用也是如此：它会根据调用者的动态类型，来选取实际的目标方法。

JVM 的静态绑定和动态绑定

接下来，我们来看看 Java 虚拟机是怎么识别方法的。

Java 虚拟机识别方法的关键在于类名、方法名以及方法描述符（method descriptor）。前面两个就不做过多的解释了。至于方法描述符，它是由方法的参数类型以及返回类型所构成。在同一个类中，如果同时出现多个名字相同且描述符也相同的方法，那么 Java 虚拟机会在类的验证阶段报错。

可以看到，Java 虚拟机与 Java 语言不同，它并不限制名字与参数类型相同，但返回类型不同的方法出现在同一个类中，对于调用这些方法的字节码来说，由于字节码所附带的方法描述符包含了返回类型，因此 Java 虚拟机能够准确地识别目标方法。

Java 虚拟机中关于方法重写的判定同样基于方法描述符。也就是说，如果子类定义了与父类中非私有、非静态方法同名的方法，那么只有当这两个方法的参数类型以及返回类型一致，Java 虚拟机才会判定为重写。

对于 Java 语言中重写而 Java 虚拟机中非重写的情况，编译器会通过生成桥接方法 [2] 来实现 Java 中的重写语义。

由于对重载方法的区分在编译阶段已经完成，我们可以认为 Java 虚拟机不存在重载这一概念。因此，在某些文章中，重载也被称为静态绑定（static binding），或者编译时多态（compile-time polymorphism）；而重写则被称为动态绑定（dynamic binding）。

这个说法在 Java 虚拟机语境下并非完全正确。这是因为某个类中的重载方法可能被它的子类所重写，因此 Java 编译器会将所有对非私有实例方法的调用编译为需要动态绑定的类型。

确切地说，Java 虚拟机中的静态绑定指的是在解析时便能够直接识别目标方法的情况，而动态绑定则指的是需要在运行过程中根据调用者的动态类型来识别目标方法的情况。

具体来说，Java 字节码中与调用相关的指令共有五种。

1. invokestatic：用于调用静态方法。
2. invokespecial：用于调用私有实例方法、构造器，以及使用 super 关键字调用父类的实例方法或构造器，和所实现接口的默认方法。
3. invokevirtual：用于调用非私有实例方法。
4. invokeinterface：用于调用接口方法。
5. invokedynamic：用于调用动态方法。

由于 invokedynamic 指令较为复杂，我将在后面的篇章中单独介绍。这里我们只讨论前四种。

我在文章中贴了一段代码，展示了编译生成这四种调用指令的情况。

```
interface 客户 {
    boolean isVIP();
}

class 商户 {
    public double 折后价格 (double 原价, 客户 某客户) {
        return 原价 * 0.8d;
    }
}

class 奸商 extends 商户 {
    @Override
    public double 折后价格 (double 原价, 客户 某客户) {
        if (某客户.isVIP()) { // invokeinterface
            return 原价 * 价格歧视 (); // invokestatic
        } else {
            return super. 折后价格 (原价, 某客户); // invokespecial
        }
    }

    public static double 价格歧视 () {
```

```
// 咱们的杀熟算法太粗暴了，应该将客户城市作为随机数生成器的种子。  
  
return new Random()                // invokespecial  
    .nextDouble()                  // invokevirtual  
    + 0.8d;  
  
}  
  
}
```

在代码中，“商户”类定义了一个成员方法，叫做“折后价格”，它将接收一个 double 类型的参数，以及一个“客户”类型的参数。这里“客户”是一个接口，它定义了一个接口方法，叫“isVIP”。

我们还定义了另一个叫做“奸商”的类，它继承了“商户”类，并且重写了“折后价格”这个方法。如果客户是 VIP，那么它会被给到一个更低的折扣。

在这个方法中，我们首先会调用“客户”接口的“isVIP”方法。该调用会被编译为 invokeinterface 指令。

如果客户是 VIP，那么我们会调用奸商类的一个名叫“价格歧视”的静态方法。该调用会被编译为 invokestatic 指令。如果客户不是 VIP，那么我们会通过 super 关键字调用父类的“折后价格”方法。该调用会被编译为 invokespecial 指令。

在静态方法“价格歧视”中，我们会调用 Random 类的构造器。该调用会被编译为 invokespecial 指令。然后我们会以这个新建的 Random 对象为调用者，调用 Random 类中的 nextDouble 方法。该调用会被编译为 invokevirtual 指令。

对于 invokestatic 以及 invokespecial 而言，Java 虚拟机能够直接识别具体的目标方法。

而对于 invokevirtual 以及 invokeinterface 而言，在绝大部分情况下，虚拟机需要在执行过程中，根据调用者的动态类型，来确定具体的目标方法。

唯一的例外在于，如果虚拟机能够确定目标方法有且仅有一个，比如说目标方法被标记为 final [3] [4]，那么它可以不通过动态类型，直接确定目标方法。

调用指令的符号引用

在编译过程中，我们并不知道目标方法的具体内存地址。因此，Java 编译器会暂时用符号引用来表示该目标方法。这一符号引用包括目标方法所在的类或接口的名字，以及目标方法的方法名和方法描述符。

符号引用存储在 class 文件的常量池之中。根据目标方法是否为接口方法，这些引用可分为接口符号引用和非接口符号引用。我在文章中贴了一个例子，利用“javap -v”打印某个类的常量池，如果你感兴趣的话可以到文章中查看。

```
// 在奸商.class 的常量池中, #16 为接口符号引用, 指向接口方法 " 客户.isVIP()". 而 #22 为非接口符号引用, 指向静态方法 " 奸商. 价格歧视 ()".

$ javap -v 奸商.class ...

Constant pool:

...
    #16 = InterfaceMethodref #27.#29          // 客户.isVIP():()Z
...
    #22 = Methodref          #1.#33          // 奸商. 价格歧视:()D
...
```

上一篇中我曾提到过, 在执行使用了符号引用的字节码前, Java 虚拟机需要解析这些符号引用, 并替换为实际引用。

对于非接口符号引用, 假定该符号引用所指向的类为 C, 则 Java 虚拟机会按照如下步骤进行查找。

1. 在 C 中查找符合名字及描述符的方法。
2. 如果没有找到, 在 C 的父类中继续搜索, 直至 Object 类。
3. 如果没有找到, 在 C 所直接实现或间接实现的接口中搜索, 这一步搜索得到的目标方法必须是非私有、非静态的。并且, 如果目标方法在间接实现的接口中, 则需满足 C 与该接口之间没有其他符合条件的目标方法。如果有多个符合条件的目标方法, 则任意返回其中一个。

从这个解析算法可以看出, 静态方法也可以通过子类来调用。此外, 子类的静态方法会隐藏（注意与重写区分）父类中的同名、同描述符的静态方法。

对于接口符号引用, 假定该符号引用所指向的接口为 I, 则 Java 虚拟机会按照如下步骤进行查找。

1. 在 I 中查找符合名字及描述符的方法。
2. 如果没有找到, 在 Object 类中的公有实例方法中搜索。
3. 如果没有找到, 则在 I 的超接口中搜索。这一步的搜索结果的要求与非接口符号引用步骤 3 的要求一致。

经过上述的解析步骤之后, 符号引用会被解析成实际引用。对于可以静态绑定的方法调用而言, 实际引用是一个指向方法的指针。对于需要动态绑定的方法调用而言, 实际引用则是一个方法表的索引。具体什么是方法表, 我会在下一篇中做出解答。

总结与实践

今天我介绍了 Java 以及 Java 虚拟机是如何识别目标方法的。

在 Java 中, 方法存在重载以及重写的概念, 重载指的是方法名相同而参数类型不相同的方法之间的

关系，重写指的是方法名相同并且参数类型也相同的方法之间的关系。

Java 虚拟机识别方法的方式略有不同，除了方法名和参数类型之外，它还会考虑返回类型。

在 Java 虚拟机中，静态绑定指的是在解析时便能够直接识别目标方法的情况，而动态绑定则指的是需要在运行过程中根据调用者的动态类型来识别目标方法的情况。由于 Java 编译器已经区分了重载的方法，因此可以认为 Java 虚拟机中不存在重载。

在 class 文件中，Java 编译器会用符号引用指代目标方法。在执行调用指令前，它所附带的符号引用需要被解析成实际引用。对于可以静态绑定的方法调用而言，实际引用为目标方法的指针。对于需要动态绑定的方法调用而言，实际引用为辅助动态绑定的信息。

在文中我曾提到，Java 的重写与 Java 虚拟机中的重写并不一致，但是编译器会通过生成桥接方法来弥补。今天的实践环节，我们来看一下两个生成桥接方法的例子。你可以通过“javap -v”来查看 class 文件所包含的方法。

```
interface Customer {
    boolean isVIP();
}

class Merchant {
    public Number actionPrice(double price, Customer customer) {
        ...
    }
}

class NaiveMerchant extends Merchant {
    @Override
    public Double actionPrice(double price, Customer customer) {
        ...
    }
}

class Merchant<T extends Customer> {
    public double actionPrice(double price, T customer) {
        ...
    }
}
```



```
class VIPOnlyMerchant extends Merchant<VIP> {  
    @Override  
    public double actionPrice(double price, VIP customer) {  
        ...  
    }  
}
```

[1] <https://docs.oracle.com/javase/8/docs/technotes/guides/language/varargs.html>

[2]

<https://docs.oracle.com/javase/tutorial/java/generics/bridgeMethods.html>

[3]

<https://wiki.openjdk.java.net/display/HotSpot/VirtualCalls>

[4]

<https://wiki.openjdk.java.net/display/HotSpot/InterfaceCalls>



版权归极客邦科技所有，未经许可不得转载

精选留言



Thomas

看明白了.....这篇真好

2018-07-27

6



永烁星光

写的有点晦涩难懂，看了好几遍，还是有点迷糊

36

2018-07-29



godtrue

8

1:方法重载

方法名相同、方法参数类型不同（其中包括参数的个数、类型、次序，三者之中只要有一个不同就行）。以前的理解方法重载是在一个类内，今天读后感觉类间有继承关系也是存在方法重载的，需要验证一下？

2:方法重写

方法名相同、方法参数类型相同、方法返回值相同，类之间有继承关系，便构成方法重写。这个概念和之前一样，不过老师强调了父类中的方法是非静态的，这个有待验证一下？

3:JVM定位目标方法的关键是类名+方法名+方法参数类型+方法返回值类型，于是就出现了两种JVM找目标方法的方式，静态绑定、动态绑定

4:静态绑定

在解析时JVM便知道该调用那个目标方法

5:动态绑定

在运行时JVM需要根据对应的类类型来具体定位应该调用那个目标方法。对于方法重写，对应的类会拥有一个方法表（一个二维数组表，给方法标上序号，重写的方法序号一致）

听了几遍，也看了几遍，感觉对具体细节还是不清楚，比如：

1:静态绑定具体咋实现的？

2:方法表在那里？啥时候创建的？咋和具体的类关联起来的呢？

可能篇幅有限啊！总体老师讲的很好，有些细节没讲到，我的感觉！

2018-07-30



小贝_No_7

4

如果这两个方法都是静态的，那么子类中的方法隐藏了父类中的方法。

这句没太明白，这个(隐藏)是否有更深一层的意义？

2018-07-28



三木子

3

需要看三遍

2018-07-27



永烁星光

2

唯一的例外在于，如果虚拟机能够确定目标方法有且仅有一个，比如说目标方法被标记为 `final` [3][4]

请问这里的`final`[3][4] 是什么意思

2018-07-29



Askerlve

任督二脉就靠这个系列打通了 ~ 💎💎

2018-07-28

2



L.B.Q.Y

开篇例子中的invoke(null,1)之所以选择第二个invoke(), 按照重载方法选择三步骤的布骤二, 不考虑变长参数但是考虑基本类型的拆装箱, 正好匹配第二个invoke.

而invoke(null,1,2)按照重载方法选择三步骤的步骤三, 两个invoke方法都匹配, 考虑类型的继承关系, 第二个invoke更恰当。

2018-07-27

2



蒙奇•D•273°

没完全理解。有个问题, 接口符号指向接口方法, 但是接口是没有实现的, 他的实现在其实现类里面, 我理解最终应该指向接口的实现类

2018-08-15

1



lxz

接口的符号引用这部分没看懂, 字节码确实是能看到invokeinterface指向了接口的方法, 但是实际执行的时候, 是怎么做的呢, 应该执行具体实现类的字节码啊。

另外, 一个小问题, 方法字节码code段内, 有些类似行号的数字, 在每行开头, 后面跟着一个冒号, 是什么意思, 是行号吗?

2018-08-02

1



安小依

对 null 强转类型至 Object 或者 String , 可以灵活控制调用第一个还是第二个函数。

2018-07-29

1



东方

好顶赞💎💎

2018-07-28

1



陈树义

文章开头的例子, 我在JDK8环境下写了个例子测试, 发现貌似和文中所说的不一致。不知道是不是因为JDK版本问题, 还是我例子有问题?

2018-07-27

作者回复

我在java8和java10里测得的都一样

2018-07-27

1



瑜斌

沙发!

2018-07-27

1



Tom

0

对于继承关系中的隐藏和重写还是不太理解，到底在哪里他们是有区别的呢？因为在调用中根本无法清楚隐藏到底是个什么概念，是在字节码文件中还是指令中有些许区别呢？感受不到区别

2018-08-17



皮卡皮卡丘

0

开篇的例子测试结果正好与老师的相反，jdk8

2018-08-15



包子

0

对于 Java 语言中重写而 Java 虚拟机中非重写的情况，编译器会通过生成桥接方法 [2] 来实现 Java 中的重写语义。

您好老师，文章中多次提到的Java语言和Java虚拟机。对这块一直很模糊。

例如上述的Java语言重写我理解是编译阶段，也就是java代码到class文件阶段。

而Java虚拟机重写是在什么阶段判定的？或者说Java虚拟机识别方法实在什么时候识别的。

Java代码从写到执行我理解的步骤是：

1. 通过javac命令从代码到class文件阶段
2. 加载class文件到虚拟机阶段（加载，链接，初始化）
3. 字节码到机器码阶段
4. 执行机器码

2018-08-13



包子

0

Java识别方法是在java代码——>字节流class编译阶段的。

Jvm识别方法是在字节码class——>机器码阶段的。也就是加载，链接，初始化

2018-08-13



东方

0

Merchant类中actionPrice方法返回值类型为Number

NaiveMerchant类中actionPrice方法返回值类型为Double

NaiveMerchant类生成的字节码中有两个参数类型相同返回值类型不同的actionPrice方法

Method actionPrice:(DLCustomer;)Ljava/lang/Double;

Method actionPrice:(DLCustomer;)Ljava/lang/Number; // 桥接到返回值为double的方法
Flags: ACC_PUBLIC, ACC_BRIDGE, ACC_SYNTHETIC

方法返回值不同为何也要产生桥接方法呢？ 为了保证重写语义？

不知为何javac在编译

NaiveMerchant naiveMerchant = new NaiveMerchant();

Number number = naiveMerchant.actionPrice(1d, null) // 特意要求Number类型的返回值(方法描

述符)

时，总invokevirtual到Method NaiveMerchant.actionPrice:(DLCustomer;)Ljava/lang/Double，这又是为什么呢？

附jdk版本

```
java version "1.8.0_172"  
Java(TM) SE Runtime Environment (build 1.8.0_172-b11)  
GraalVM 1.0.0-rc5 (build 25.71-b01-internal-jvmci-0.46, mixed mode)
```

2018-08-11

作者回复

- 1 对的，为了保证重写语义。
- 2 生成的桥接方法还有一个acc_synthetic标记，代表对程序不可见。因此javac不能直接选取那个方法。

2018-08-15



陈勇明

0

如果这两个方法都是静态的，那么子类中的方法隐藏了父类中的方法，这里的隐藏是什么意思？父类的方法调用不到了吗？类C及子类C1都有静态方法m,那么C.m()与C1.m(), 以及这两个类的实例调用该方法应该都互不影响吧

2018-08-09

作者回复

就是C1.m()调用不到C.m()了。如果C1没有定义m(), 那么用C1.m()可以调用C.m()

2018-08-10

讲堂 深入拆解 Java 虚拟机 文章详情

05 | JVM是如何执行方法调用的？（下）

2018-07-30 郑雨迪



05 | JVM是如何执行方法调用的？（下）

朗读人：郑雨迪 13'01" | 5.97M

我在读博士的时候，最怕的事情就是被问有没有新的 Idea。有一次我被老板问急了，就随口说了一个。

这个 Idea 究竟是什么呢，我们知道，设计模式大量使用了虚方法来实现多态。但是虚方法的性能效率并不高，所以我就说，是否能够在此基础上写篇文章，评估每一种设计模式因为虚方法调用而造成的性能开销，并且在文章中强烈谴责一下？

当时呢，我老板教的是一门高级程序设计的课，其中有好几节课刚好在讲设计模式的种种好处。所以，我说完这个 Idea，就看到老板的神色略有不悦了，脸上写满了“小郑啊，你这是舍本逐末啊”，于是，我就连忙挽尊，说我是开玩笑的。

在这里呢，我犯的错误其实有两个。第一，我不应该因为虚方法的性能效率，而放弃良好的设计。第二，通常来说，Java 虚拟机中虚方法调用的性能开销并不大，有些时候甚至可以完全消除。第一个错误是原则上的，这里就不展开了。至于第二个错误，我们今天便来聊一聊 Java 虚拟机中虚方法

调用的具体实现。

首先，我们来看一个模拟出国边检的小例子。

```
abstract class 乘客 {  
    abstract void 出境 ();  
  
    @Override  
    public String toString() { ... }  
}  
  
class 外国人 extends 乘客 {  
    @Override  
    void 出境 () { /* 进外国人通道 */ }  
}  
  
class 中国人 extends 乘客 {  
    @Override  
    void 出境 () { /* 进中国人通道 */ }  
    void 买买买 () { /* 逛免税店 */ }  
}  
  
乘客 某乘客 = ...  
某乘客. 出境 ();
```

这里我定义了一个抽象类，叫做“乘客”，这个类中有一个名为“出境”的抽象方法，以及重写自 Object 类的 toString 方法。

然后，我将“乘客”粗暴地分为两种：“中国人”以及“外国人”。这两个类分别实现了“出境”这个方法，具体来说，就是中国人走中国人通道，外国人走外国人通道。由于咱们储蓄较多，所以我在“中国人”这个类中，还特意添加了一个叫做“买买买”的方法。

那么在实际运行过程中，Java 虚拟机是如何高效地确定每个“乘客”实例应该去哪条通道的呢？我们一起来看一下。

1. 虚方法调用

在上一篇中我曾经提到，Java 里所有非私有实例方法调用都会被编译成 invokevirtual 指令，而接口方法调用都会被编译成 invokeinterface 指令。这两种指令，均属于 Java 虚拟机中的虚方法调用。

在绝大多数情况下，Java 虚拟机需要根据调用者的动态类型，来确定虚方法调用的目标方法。这个

过程我们称之为动态绑定。那么，相对于静态绑定的非虚方法调用来说，虚方法调用更加耗时。

在 Java 虚拟机中，静态绑定包括用于调用静态方法的 `invokestatic` 指令，和用于调用构造器、私有实例方法以及超类非私有实例方法的 `invokespecial` 指令。如果虚方法调用指向一个标记为 `final` 的方法，那么 Java 虚拟机也可以静态绑定该虚方法调用的目标方法。

Java 虚拟机中采取了一种用空间换取时间的策略来实现动态绑定。它为每个类生成一张方法表，用以快速定位目标方法。那么方法表具体是怎样实现的呢？

2. 方法表

在介绍那篇类加载机制的链接部分中，我曾提到类加载的准备阶段，它除了为静态字段分配内存之外，还会构造与该类相关联的方法表。

这个数据结构，便是 Java 虚拟机实现动态绑定的关键所在。下面我将以 `invokevirtual` 所使用的虚方法表（virtual method table, `vtable`）为例介绍方法表的用法。`invokeinterface` 所使用的接口方法表（interface method table, `itable`）稍微复杂些，但是原理其实是类似的。

方法表本质上是一个数组，每个数组元素指向一个当前类及其祖先类中非私有的实例方法。

这些方法可能是具体的、可执行的方法，也可能是没有相应字节码的抽象方法。方法表满足两个特质：其一，子类方法表中包含父类方法表中的所有方法；其二，子类方法在方法表中的索引值，与它所重写的父类方法的索引值相同。

我们知道，方法调用指令中的符号引用会在执行之前解析成实际引用。对于静态绑定的方法调用而言，实际引用将指向具体的目标方法。对于动态绑定的方法调用而言，实际引用则是方法表的索引值（实际上并不仅是索引值）。

在执行过程中，Java 虚拟机将获取调用者的实际类型，并在该实际类型的虚方法表中，根据索引值获得目标方法。这个过程便是动态绑定。

乘客的方法表

0	乘客.toString()
1	乘客.出境() (备注：抽象方法，不可执行)

外国人的方法表

0	乘客.toString()
1	外国人.出境()

中国人的方法表

0	乘客.toString()
1	中国人.出境()
2	中国人.买买买()

在我们的例子中，“乘客”类的方法表包括两个方法：“toString”以及“出境”，分别对应 0 号和 1 号。之所以方法表调换了“toString”方法和“出境”方法的位置，是因为“toString”方法的索引值需要与 Object 类中同名方法的索引值一致。为了保持简洁，这里我就不考虑 Object 类中的其他方法。

“外国人”的方法表同样有两行。其中，0 号方法指向继承而来的“乘客”类的“toString”方法。1 号方法则指向自己重写的“出境”方法。

“中国人”的方法表则包括三个方法，除了继承而来的“乘客”类的“toString”方法，自己重写的“出境”方法之外，还包括独有的“买买买”方法。

```
乘客 某乘客 = ...
某乘客. 出境 ();
```

这里，Java 虚拟机的工作可以想象为导航员。每当来了一个乘客需要出境，导航员会先问是中国人还是外国人（获取动态类型），然后翻出中国人 / 外国人对应的小册子（获取动态类型的方法表），小册子的第 1 页便写着应该到哪条通道办理出境手续（用 1 作为索引来查找方法表所对应的目标方法）。

实际上，使用了方法表的动态绑定与静态绑定相比，仅仅多出几个内存解引用操作：访问栈上的调用者，读取调用者的动态类型，读取该方法类型的方法表，读取方法表中某个索引值所对应的目标方法。相对于创建并初始化 Java 栈帧来说，这几个内存解引用操作的开销简直可以忽略不计。

那么我们是否可以认为虚方法调用对性能没有太大影响呢？

其实是不能的，上述优化的效果看上去十分美好，但实际上仅存在于解释执行中，或者即时编译代码的最坏情况中。这是因为即时编译还拥有另外两种性能更好的优化手段：内联缓存（inlining cache）和方法内联（method inlining）。下面我便来介绍第一种内联缓存。

3. 内联缓存

内联缓存是一种加快动态绑定的优化技术。它能够缓存虚方法调用中调用者的动态类型，以及该类型所对应的目标方法。在之后的执行过程中，如果碰到已缓存的类型，内联缓存便会直接调用该类型所对应的目标方法。如果没有碰到已缓存的类型，内联缓存则会退化至使用基于方法表的动态绑定。

在我们的例子中，这相当于导航员记住了上一个出境乘客的国籍和对应的通道，例如中国人，走了左边通道出境。那么下一个乘客想要出境的时候，导航员会先问是不是中国人，是的话就走左边通道。如果不是的话，只好拿出外国人的小册子，翻到第 1 页，再告知查询结果：右边。

在针对多态的优化手段中，我们通常会提及以下三个术语。

1. 单态（monomorphic）指的是仅有一种状态的情况。
2. 多态（polymorphic）指的是有限数量种状态的情况。二态（bimorphic）是多态的其中一种。
3. 超多态（megamorphic）指的是更多种状态的情况。通常我们用一个具体数值来区分多态和超多态。在这个数值之下，我们称之为多态。否则，我们称之为超多态。

对于内联缓存来说，我们也有对应的单态内联缓存、多态内联缓存和超多态内联缓存。单态内联缓存，顾名思义，便是只缓存了一种动态类型以及它所对应的目标方法。它的实现非常简单：比较所缓存的动态类型，如果命中，则直接调用对应的目标方法。

多态内联缓存则缓存了多个动态类型及其目标方法。它需要逐个将所缓存的动态类型与当前动态类型进行比较，如果命中，则调用对应的目标方法。

一般来说，我们会将更加热门的动态类型放在前面。在实践中，大部分的虚方法调用均是单态的，也就是只有一种动态类型。为了节省内存空间，Java 虚拟机只采用单态内联缓存。

前面提到，当内联缓存没有命中的情况下，Java 虚拟机需要重新使用方法表进行动态绑定。对于内联缓存中的内容，我们有两种选择。一是替换单态内联缓存中的纪录。这种做法就好比 CPU 中的数据缓存，它对数据的局部性有要求，即在替换内联缓存之后的一段时间内，方法调用的调用者的动态类型应当保持一致，从而能够有效地利用内联缓存。

因此，在最坏情况下，我们用两种不同类型的调用者，轮流执行该方法调用，那么每次进行方法调用都将替换内联缓存。也就是说，只有写缓存的额外开销，而没有用缓存的性能提升。

另外一种选择则是劣化为超多态状态。这也是 Java 虚拟机的具体实现方式。处于这种状态下的内联

缓存，实际上放弃了优化的机会。它将直接访问方法表，来动态绑定目标方法。与替换内联缓存纪录的做法相比，它牺牲了优化的机会，但是节省了写缓存的额外开销。

具体到我们的例子，如果来了一队乘客，其中外国人和中国人依次隔开，那么在重复使用的单态内联缓存中，导航员需要反复记住上个出境的乘客，而且记住的信息在处理下一乘客时又会被替换掉。因此，倒不如一直不记，以此来节省脑细胞。

虽然内联缓存附带内联二字，但是它并没有内联目标方法。这里需要明确的是，任何方法调用除非被内联，否则都会有固定开销。这些开销来源于保存程序在该方法中的执行位置，以及新建、压入和弹出新方法所使用的栈帧。

对于极其简单的方法而言，比如说 getter/setter，这部分固定开销占据的 CPU 时间甚至超过了方法本身。此外，在即时编译中，方法内联不仅仅能够消除方法调用的固定开销，而且还增加了进一步优化的可能性，我们会在专栏的第二部分详细介绍方法内联的内容。

总结与实践

今天我介绍了虚方法调用在 Java 虚拟机中的实现方式。

虚方法调用包括 `invokevirtual` 指令和 `invokeinterface` 指令。如果这两种指令所声明的目标方法被标记为 `final`，那么 Java 虚拟机会采用静态绑定。否则，Java 虚拟机将采用动态绑定，在运行过程中根据调用者的动态类型，来决定具体的目标方法。

Java 虚拟机的动态绑定是通过方法表这一数据结构来实现的。方法表中每一个重写方法的索引值，与父类方法表中被重写的方法的索引值一致。在解析虚方法调用时，Java 虚拟机会纪录下所声明的目标方法的索引值，并且在运行过程中根据这个索引值查找具体的目标方法。

Java 虚拟机中的即时编译器会使用内联缓存来加速动态绑定。Java 虚拟机所采用的单态内联缓存将纪录调用者的动态类型，以及它所对应的目标方法。

当碰到新的调用者时，如果其动态类型与缓存中的类型匹配，则直接调用缓存的目标方法。否则，Java 虚拟机将该内联缓存劣化为超多态内联缓存，在今后的执行过程中直接使用方法表进行动态绑定。

在今天的实践环节，我们来观测一下单态内联缓存和超多态内联缓存的性能差距。为了消除方法内联的影响，请使用如下的命令。

```
// Run with: java -XX:CompileCommand='dontinline,*. 出境' 乘客
public abstract class 乘客 {
    abstract void 出境 ();
    public static void main(String[] args) {
```

```
乘客 a = new 中国人 ();
乘客 b = new 外国人 ();

long current = System.currentTimeMillis();
for (int i = 1; i <= 2_000_000_000; i++) {
    if (i % 100_000_000 == 0) {
        long temp = System.currentTimeMillis();
        System.out.println(temp - current);
        current = temp;
    }
    乘客 c = (i < 1_000_000_000) ? a : b;
    c.出境 ();
}
}

class 中国人 extends 乘客 { @Override void 出境 () {} }
class 外国人 extends 乘客 { @Override void 出境 () {} }
```



版权归极客邦科技所有，未经许可不得转载

精选留言



啊一大狗
这套课很好，谢谢！

2018-07-31



Tony

22

同提建议，代码使用英文。刚学java基础时，有老师为了便于理解用中文命名。现在都来学jvm，对java很熟悉了，看到中文不仅不会觉得通俗易懂，反而特别别扭。

2018-07-30

作者回复

多谢建议！

原本是英文的，录音的时候觉得老要切换，就给换了。。

2018-07-30



Jchriss

9

提个小建议，能否在代码中都使用英文？毕竟使用中文作对象名不值得提倡

2018-07-30

作者回复

谢谢建议！

2018-07-30



陈晨

5

没用过中文写代码，居然认为中文会编译错误T____T

老师是为了课件方便这样写，自己写作业就改下呗，又没规定要每个字照抄

```
[root@localhost cqg]# javac Passenger.java
```

```
[root@localhost cqg]# java Passenger
```

```
cost time : 1167
```

```
cost time : 3156
```

```
[root@localhost cqg]# java -XX:CompileCommand='dontinline,*.exit' Passenger
```

```
CompilerOracle: dontinline *.exit
```

```
cost time : 3709
```

```
cost time : 7557
```

2018-07-30

作者回复

哈，我以前也认为无法编译，直到有一次我看到一个俄语的方法名。。

另外，如果你用javap -v查看常量池的话，你会发现类名方法名以及方法描述符都是用UTF8来存的

。

2018-07-30



杨春鹏

4

关于单态内联缓存中的记录，hotspot采用了超多态。也就是如果该调用者的动态类型不是缓存中的类型的话，直接通过基于方法表来找到具体的目标方法。那么内联缓存中的类型是永久不变，一直是第一次缓存的那个调用者类型吗？

2018-07-31

2



godtrue

1:虚方法

方法重写的方法，可认为就是虚方法

2:JVM怎么执行虚方法

通过方法表，一个二维表结构，标示出类的类型、虚方法的序号。当调用虚方法的时候，先确定类型，再根据类型找方法

2018-07-30



吾是锋子

1

郑老师，您好。有个具体的问题想请教下，String类里面indexOf(String str)调用的是自己类里面indexOf(String str, int fromIndex)方法，但我自己在测试的时候却发现两个方法的速度有很明显的差异，看字节码也没有发现什么特殊。

不知道是不是我忽略了什么，希望您能抽空点拨下，感谢！

2018-08-14

作者回复

HotSpot里有String.indexOf intrinsic，用了很多向量化指令，所以性能会快很多的。

关于intrinsic的概念，你可以理解为HotSpot识别指定方法后，将其替代为语意等价的高效实现。

2018-08-15



左岸

1

为什么调用超类非私有实例方法会属于静态绑定呢？

2018-07-30

作者回复

通过super关键字来调用父类方法，本意就是想要调用父类的特定方法，而不是根据具体类型决定目标方法。

2018-07-31



乔毅

0

感觉还是没理解。虚函数调用开销主要是查表带来的，文中又说查表只是内存解引用成本可以忽略不计。但从课后的例子看有没有查表的缓存，差距有小一倍，所以结论是查表的影响还是挺大的？

2018-08-17



杨军

0

一直不太理解一个问题：“Java的动态类型运行期才可知”，在编译期代码写完之后应该就已经确定了吧，比如A是B的子类，“B b = new B(); b = new A()”这种情况下b的动态类型是A，Java编译器在编译阶段就可以确定啊，为什么说动态类型直到运行期才可知？

诚心求老师解惑，这个问题对我理解Java的动态绑定机制很关键-.-

2018-08-13



东方

0

清晰易懂，赞

2018-08-11



猎户星空

0

单态的用内联缓存，超多态的劣化为方法表+索引，那多态的呢？

2018-08-11

作者回复

这里指的是HotSpot的情况，它不存在多态内联缓存。

2018-08-15



Bale

0

老师，我只有第一次能成功出现，关掉inline 功能会变的慢一些。
如果我不清除缓存的，执行命令和不带命令的时间几乎相等。
但是我清除内存，就会重现结果。
Jvm会缓存 我执行过的java类嘛？

2018-08-06



沉淀的梦想

0

为什么在我电脑上运行实验的结果是这样
java 乘客

打印:

961

1740

java -XX:CompileCommand="dontinline, *.*" 乘客

打印:

3164

3506

禁止方法内联之后，反而差别不大了

2018-08-05



一只智障

0

内联缓存再碰到没有缓存的目标类以及目标方法，那么会去方法表中寻找，那为什么内联表不把这个没有命中的方法缓存下来供下次使用呢？

2018-08-05



丝竹悠扬

0

挺好的，好想可以听懂点

2018-08-05

蠢蠢欲动的腹肌

0



老师，请问下，在准备阶段生成的方法表存储在哪里？

2018-08-03



Grubby??

0

内联缓存是针对每个类的吗？就是说两个不同父类的子类，在jvm里有两个内联缓存？

2018-08-02



lxz

0

建议结合java代码及其对应的字节码来讲解，比如常量池，方法表在字节码中对应的位置，干讲一点印象也没有

2018-08-02



熊猫酒仙

0

想问下老师，方法作为参数传递，传递的是方法地址吗？这种情况，执行时是否不存在查找方法的过程？

另外假如两个类的方法互相调用，互相需要对方的方法信息，(排除其他依赖的情况)那么这两个类的加载是并行还是串行呢？

2018-08-02

讲堂 深入拆解 Java 虚拟机 文章详情

06 | JVM是如何处理异常的？

2018-08-01 郑雨迪



06 | JVM是如何处理异常的？

朗读人：郑雨迪 11'26" | 5.24M

今天我们来讲讲 Java 虚拟机的异常处理。首先提醒你一下，本篇文章代码较多，你可以[点击文稿查看具体代码](#)。

众所周知，异常处理的两大组成要素是抛出异常和捕获异常。这两大要素共同实现程序控制流的非正常转移。

抛出异常可分为显式和隐式两种。显式抛异常的主体是应用程序，它指的是在程序中使用“throw”关键字，手动将异常实例抛出。

隐式抛异常的主体则是 Java 虚拟机，它指的是 Java 虚拟机在执行过程中，碰到无法继续执行的异常状态，自动抛出异常。举例来说，Java 虚拟机在执行读取数组操作时，发现输入的索引值是负数，故而抛出数组索引越界异常（`ArrayIndexOutOfBoundsException`）。

捕获异常则涉及了如下三种代码块。

1. try 代码块：用来标记需要进行异常监控的代码。
2. catch 代码块：跟在 try 代码块之后，用来捕获在 try 代码块中触发的某种指定类型的异常。除了声明所捕获异常的类型之外，catch 代码块还定义了针对该异常类型的异常处理器。在 Java 中，try 代码块后面可以跟着多个 catch 代码块，来捕获不同类型的异常。Java 虚拟机会从上至下匹配异常处理器。因此，前面的 catch 代码块所捕获的异常类型不能覆盖后边的，否则编译器会报错。
3. finally 代码块：跟在 try 代码块和 catch 代码块之后，用来声明一段必定运行的代码。它的设计初衷是为了避免跳过某些关键的清理代码，例如关闭已打开的系统资源。

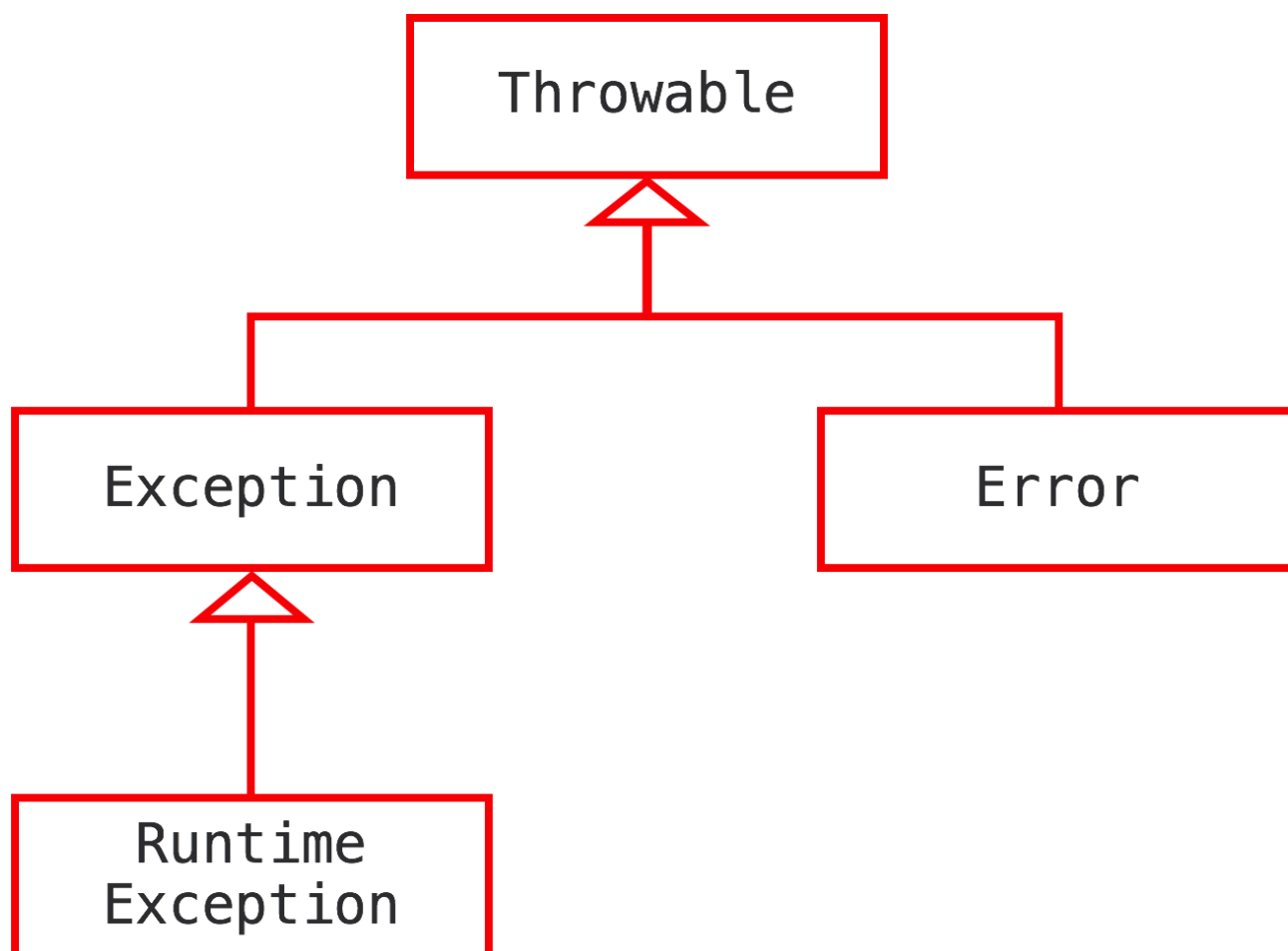
在程序正常执行的情况下，这段代码会在 try 代码块之后运行。否则，也就是 try 代码块触发异常的情况下，如果该异常没有被捕获，finally 代码块会直接运行，并且在运行之后重新抛出该异常。

如果该异常被 catch 代码块捕获，finally 代码块则在 catch 代码块之后运行。在某些不幸的情况下，catch 代码块也触发了异常，那么 finally 代码块同样会运行，并会抛出 catch 代码块触发的异常。在某些极端不幸的情况下，finally 代码块也触发了异常，那么只好中断当前 finally 代码块的执行，并往外抛异常。

上面这段听起来有点绕，但是等我讲完 Java 虚拟机的异常处理机制之后，你便会明白这其中的道理。

异常的基本概念

在 Java 语言规范中，所有异常都是 Throwable 类或者其子类的实例。Throwable 有两大直接子类。第一个是 Error，涵盖程序不应捕获的异常。当程序触发 Error 时，它的执行状态已经无法恢复，需要中止线程甚至是中止虚拟机。第二子类则是 Exception，涵盖程序可能需要捕获并且处理的异常。



Exception 有一个特殊的子类 RuntimeException，用来表示“程序虽然无法继续执行，但是还能抢救一下”的情况。前边提到的数组索引越界便是其中的一种。

RuntimeException 和 Error 属于 Java 里的非检查异常（unchecked exception）。其他异常则属于检查异常（checked exception）。在 Java 语法中，所有的检查异常都需要程序显式地捕获，或者在方法声明中用 throws 关键字标注。通常情况下，程序中自定义的异常应为检查异常，以便最大化利用 Java 编译器的编译时检查。

异常实例的构造十分昂贵。这是由于在构造异常实例时，Java 虚拟机便需要生成该异常的栈轨迹（stack trace）。该操作会逐一访问当前线程的 Java 栈帧，并且记录下各种调试信息，包括栈帧所指向方法的名字，方法所在的类名、文件名，以及在代码中的第几行触发该异常。

当然，在生成栈轨迹时，Java 虚拟机会忽略掉异常构造器以及填充栈帧的 Java 方法（Throwable.printStackTrace），直接从新建异常位置开始算起。此外，Java 虚拟机还会忽略标记为不可见的 Java 方法栈帧。我们在介绍 Lambda 的时候会看到具体的例子。

既然异常实例的构造十分昂贵，我们是否可以缓存异常实例，在需要用的时候直接抛出呢？从语法角度上来看，这是允许的。然而，该异常对应的栈轨迹并非 throw 语句的位置，而是新建异常的位置。

因此，这种做法可能会误导开发人员，使其定位到错误的位置。这也是为什么在实践中，我们往往选择抛出新建异常实例的原因。

Java 虚拟机是如何捕获异常的？

在编译生成的字节码中，每个方法都附带一个异常表。异常表中的每一个条目代表一个异常处理器，并且由 from 指针、to 指针、target 指针以及所捕获的异常类型构成。这些指针的值是字节码索引（bytecode index, bci），用以定位字节码。

其中，from 指针和 to 指针标示了该异常处理器所监控的范围，例如 try 代码块所覆盖的范围。target 指针则指向异常处理器的起始位置，例如 catch 代码块的起始位置。

```
public static void main(String[] args) {
    try {
        mayThrowException();
    } catch (Exception e) {
        e.printStackTrace();
    }
}

// 对应的 Java 字节码
public static void main(java.lang.String[]);
Code:
    0: invokestatic mayThrowException:()V
    3: goto 11
    6: astore_1
    7: aload_1
    8: invokevirtual java.lang.Exception.printStackTrace
   11: return
Exception table:
    from   to target type
    0      3   6   Class java/lang/Exception  // 异常表条目
```

举个例子，在上图的 main 方法中，我定义了一段 try-catch 代码。其中，catch 代码块所捕获的异常类型为 Exception。

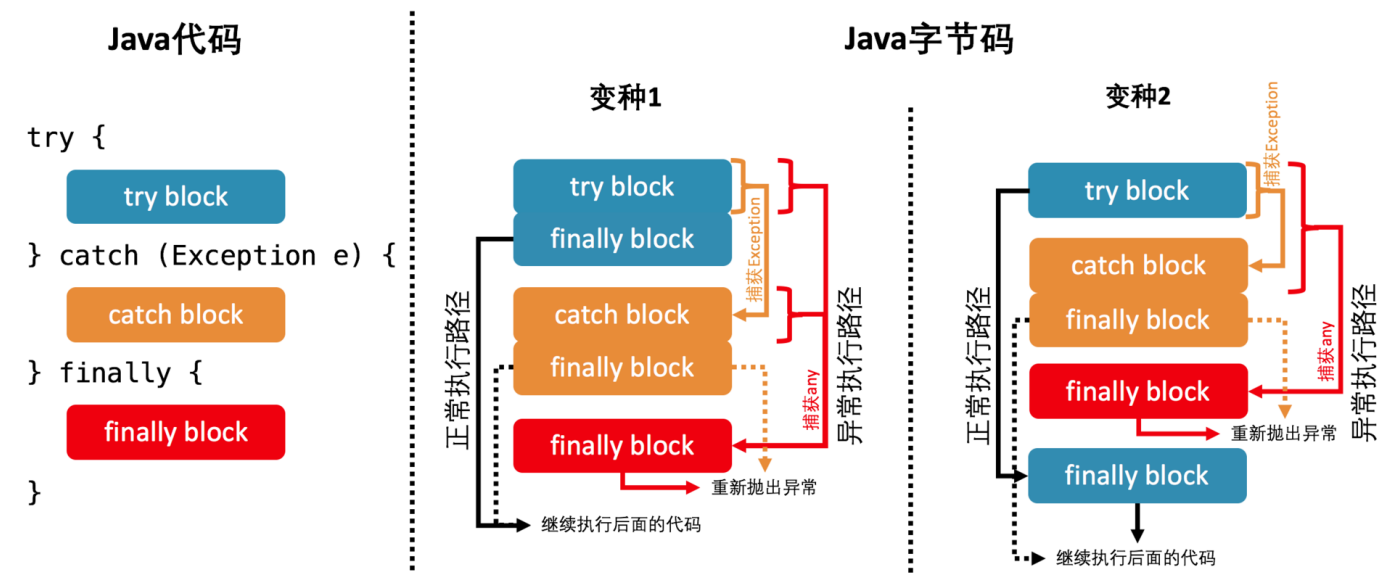
编译过后，该方法的异常表拥有一个条目。其 from 指针和 to 指针分别为 0 和 3，代表它的监控范

围从索引为 0 的字节码开始，到索引为 3 的字节码结束（不包括 3）。该条目的 target 指针是 6，代表这个异常处理器从索引为 6 的字节码开始。条目的最后一列，代表该异常处理器所捕获的异常类型正是 Exception。

当程序触发异常时，Java 虚拟机会从上至下遍历异常表中的所有条目。当触发异常的字节码的索引值在某个异常表条目的监控范围内，Java 虚拟机会判断所抛出的异常和该条目想要捕获的异常是否匹配。如果匹配，Java 虚拟机会将控制流转移至该条目 target 指针指向的字节码。

如果遍历完所有异常表条目，Java 虚拟机仍未匹配到异常处理器，那么它会弹出当前方法对应的 Java 栈帧，并且在调用者（caller）中重复上述操作。在最坏情况下，Java 虚拟机需要遍历当前线程 Java 栈上所有方法的异常表。

pnally 代码块的编译比较复杂。当前版本 Java 编译器的做法，是复制 pnally 代码块的内容，分别放在 try-catch 代码块所有正常执行路径以及异常执行路径的出口中。



针对异常执行路径，Java 编译器会生成一个或多个异常表条目，监控整个 try-catch 代码块，并且捕获所有种类的异常（在 javap 中以 any 指代）。这些异常表条目的 target 指针将指向另一份复制的 pnally 代码块。并且，在这个 pnally 代码块的最后，Java 编译器会重新抛出所捕获的异常。

如果你感兴趣的话，可以用 javap 工具来查看下面这段包含了 try-catch-pnally 代码块的编译结果。为了更好地区分每个代码块，我定义了四个实例字段：tryBlock、catchBlock、pnallyBlock、以及 methodExit，并且仅在对应的代码块中访问这些字段。

```
public class Foo {  
    private int tryBlock;  
    private int catchBlock;  
    private int finallyBlock;  
    private int methodExit;  
}
```



```
public void test() {  
    try {  
        tryBlock = 0;  
    } catch (Exception e) {  
        catchBlock = 1;  
    } finally {  
        finallyBlock = 2;  
    }  
    methodExit = 3;  
}  
}
```

```
$ javap -c Foo
```

```
...
```

```
public void test();
```

```
Code:
```

```
0: aload_0  
1: iconst_0  
2: putfield      #20          // Field tryBlock:I  
5: goto         30  
8: astore_1  
9: aload_0  
10: iconst_1  
11: putfield      #22          // Field catchBlock:I  
14: aload_0  
15: iconst_2  
16: putfield      #24          // Field finallyBlock:I  
19: goto         35  
22: astore_2  
23: aload_0  
24: iconst_2  
25: putfield      #24          // Field finallyBlock:I  
28: aload_2  
29: athrow  
30: aload_0
```

```
31: iconst_2
32: putfield      #24          // Field finallyBlock:I
35: aload_0
36: iconst_3
37: putfield      #26          // Field methodExit:I
40: return

Exception table:
   from    to  target type
    0       5     8   Class java/lang/Exception
    0      14    22    any

...
```

可以看到，编译结果包含三份 `finally` 代码块。其中，前两份分别位于 `try` 代码块和 `catch` 代码块的正常执行路径出口。最后一份则作为异常处理器，监控 `try` 代码块以及 `catch` 代码块。它将捕获 `try` 代码块触发的、未被 `catch` 代码块捕获的异常，以及 `catch` 代码块触发的异常。

这里有一个小问题，如果 `catch` 代码块捕获了异常，并且触发了另一个异常，那么 `finally` 捕获并且重抛的异常是哪一个呢？答案是后者。也就是说原本的异常便会被忽略掉，这对于代码调试来说十分不利。

Java 7 的 `Suppressed` 异常以及语法糖

Java 7 引入了 `Suppressed` 异常来解决这个问题。这个新特性允许开发人员将一个异常附于另一个异常之上。因此，抛出的异常可以附带多个异常的信息。

然而，Java 层面的 `finally` 代码块缺少指向所捕获异常的引用，所以这个新特性使用起来非常繁琐。

为此，Java 7 专门构造了一个名为 `try-with-resources` 的语法糖，在字节码层面自动使用 `Suppressed` 异常。当然，该语法糖的主要目的并不是使用 `Suppressed` 异常，而是精简资源打开关闭的用法。

在 Java 7 之前，对于打开的资源，我们需要定义一个 `finally` 代码块，来确保该资源在正常或者异常执行状况下都能关闭。

资源的关闭操作本身容易触发异常。因此，如果同时打开多个资源，那么每一个资源都要对应一个独立的 `try-finally` 代码块，以保证每个资源都能够关闭。这样一来，代码将会变得十分繁琐。

```
FileInputStream in0 = null;
```

```

FileInputStream in1 = null;
FileInputStream in2 = null;

...
try {
    in0 = new FileInputStream(new File("in0.txt"));
    ...
    try {
        in1 = new FileInputStream(new File("in1.txt"));
        ...
        try {
            in2 = new FileInputStream(new File("in2.txt"));
            ...
        } finally {
            if (in2 != null) in2.close();
        }
    } finally {
        if (in1 != null) in1.close();
    }
} finally {
    if (in0 != null) in0.close();
}

```

Java 7 的 try-with-resources 语法糖，极大地简化了上述代码。程序可以在 try 关键字后声明并实例化实现了 AutoCloseable 接口的类，编译器将自动添加对应的 close() 操作。在声明多个 AutoCloseable 实例的情况下，编译生成的字节码类似于上面手工编写代码的编译结果。与手工代码相比，try-with-resources 还会使用 Supressed 异常的功能，来避免原异常“被消失”。

```

public class Foo implements AutoCloseable {
    private final String name;

    public Foo(String name) { this.name = name; }

    @Override
    public void close() {
        throw new RuntimeException(name);
    }
}

```

```
public static void main(String[] args) {  
    try (Foo foo0 = new Foo("Foo0")); // try-with-resources  
        Foo foo1 = new Foo("Foo1");  
        Foo foo2 = new Foo("Foo2")) {  
        throw new RuntimeException("Initial");  
    }  
}  
}
```

// 运行结果:

```
Exception in thread "main" java.lang.RuntimeException: Initial  
    at Foo.main(Foo.java:18)  
    Suppressed: java.lang.RuntimeException: Foo2  
        at Foo.close(Foo.java:13)  
        at Foo.main(Foo.java:19)  
    Suppressed: java.lang.RuntimeException: Foo1  
        at Foo.close(Foo.java:13)  
        at Foo.main(Foo.java:19)  
    Suppressed: java.lang.RuntimeException: Foo0  
        at Foo.close(Foo.java:13)  
        at Foo.main(Foo.java:19)
```

除了 try-with-resources 语法糖之外，Java 7 还支持在同一 catch 代码块中捕获多种异常。实际实现非常简单，生成多个异常表条目即可。

```
// 在同一 catch 代码块中捕获多种异常  
try {  
    ...  
} catch (SomeException | OtherException e) {  
    ...  
}
```

总结与实践

今天我介绍了 Java 虚拟机的异常处理机制。

Java 的异常分为 Exception 和 Error 两种，而 Exception 又分为 RuntimeException 和其他类型。RuntimeException 和 Error 属于非检查异常。其他的 Exception 皆属于检查异常，在触发时需要显式捕获，或者在方法头用 throws 关键字声明。

Java 字节码中，每个方法对应一个异常表。当程序触发异常时，Java 虚拟机将查找异常表，并依此决定需要将控制流转移至哪个异常处理器之中。Java 代码中的 catch 代码块和 finally 代码块都会生成异常表条目。

Java 7 引入了 Suppressed 异常、try-with-resources，以及多异常捕获。后两者属于语法糖，能够极大地精简我们的代码。

那么今天的实践环节，你可以看看其他控制流语句与 finally 代码块之间的协作。

```
// 编译并用 javap -c 查看编译后的字节码

public class Foo {

    private int tryBlock;

    private int catchBlock;

    private int finallyBlock;

    private int methodExit;

    public void test() {

        for (int i = 0; i < 100; i++) {

            try {

                tryBlock = 0;

                if (i < 50) {

                    continue;

                } else if (i < 80) {

                    break;

                } else {

                    return;

                }

            } catch (Exception e) {

                catchBlock = 1;

            } finally {

                finallyBlock = 2;

            }

        }

    }

}
```

```
methodExit = 3;  
  
}  
  
}
```



版权归极客邦科技所有，未经许可不得转载

精选留言



godtrue

感谢雨迪！

这节让我终于搞清楚了两个疑惑！

13

1:使用异常捕获的代码为什么比较耗费性能？

因为构造异常的实例比较耗性能。这从代码层面很难理解，不过站在JVM的角度来看就简单了，因为JVM在构造异常实例时需要生成该异常的栈轨迹。这个操作会逐一访问当前线程的栈帧，并且记录下各种调试信息，包括栈帧所指向方法的名字，方法所在的类名、文件名，以及在代码中的第几行触发该异常等信息。

虽然具体不清楚JVM的实现细节，但是看描述这件事情也是比较费时费力的。

2:Pnally是怎么实现无论异常与否都能被执行的？

这个事情是由编译器来实现的，现在的做法是这样的，编译器在编译Java代码时，会复制Pnally代码块的内容，然后分别放在try-catch代码块所有的正常执行路径及异常执行路径的出口中。

2018-08-02

王小臭

3



辛苦老师了，这么早更新

2018-08-01



3.27。

2

如果在业务层的代码中使用Assert来判断参数是否有问题，然后在调用方捕捉异常，这样会不会耗性能

2018-08-01

作者回复

首先走抛出异常捕获异常的异常执行路径的话，性能肯定是很慢的，因此最好在参数出现问题的概率很小的情况下使用这种方式。

另外，你说的Assert是某个库的工具类，还是assert语句？后者的话，一般只在开发环境中启用吧

。

2018-08-02



Ennis LM

1

Java 虚拟机会忽略掉异常构造器以及填充栈帧的 Java 方法（Throwable.printStackTrace），直接从新建异常位置开始算起。

Java 虚拟机还会忽略标记为不可见的 Java 方法栈帧。

请问老师，填充栈帧的 Java 方法和不可见的 Java 方法栈帧，是什么

2018-08-01

作者回复

前者指Throwable.printStackTrace 以及异常的构造器，后者为Java虚拟机不想让用户看到的栈帧，比如说方法句柄的适配器类中的方法。之后讲Lambda时会有具体的例子。

2018-08-02



三木子

1

看完今天的文章有几个疑问

1方法的异常表是包含RuntimeException这种非check类型的异常吧？如果是那么每个方法都有异常表，那么是不是每个异常表中都有像ArrayIndexOutOfBoundsException这类型异常了。这类公共异常是私有还是共享呢

2像catch自定义异常，也会添加的当前方法的异常表里吗？

3 我们常常看到的异常调用栈，这里方法调用信息其实就是弹出方法栈帧吗？

2018-08-01

作者回复

1 检查异常这个概念只在源代码中出现。异常表不是声明这段代码所有有可能抛出的异常，而是声明会被捕获的异常。

2 会的

3 栈轨迹 跟 弹出方法栈帧 是两个概念。你可以直接新建一个异常，然后不抛出，直接打印调用栈。这个时候是不会弹出当前栈帧的。

2018-08-02



南城风戈
沙发

1

2018-08-01



东方

0

throw exception性能差Pllstacktrace除了遍历堆栈以外，如果有inline 代码消除等编译优化发生，是不是要先“去优化”完了再Pll？要不然可能出现错误堆栈和代码对不上的情况

throw exception估计也会影响jit的优化，进而影响整体性能

2018-08-11

作者回复

思考得很深，赞一个！

即时编译器生成的代码会保存原始的栈信息，以便去优化时能够复原。PllStackTrace 也会读取这些信息的，所以不用先去优化再Pll。

抛异常本身带来了额外的执行路径。通常如果能够将异常处理器也编译进去，那么不会有太大影响。

2018-08-15



贾智文

0

当触发异常的字节码的索引值在某个异常表条目的监控范围内，Java 虚拟机会判断所抛出的异常和该条目想要捕获的异常是否匹配。

这里有点没懂，每层方法的监控范围有可能会重叠吧，只用索引判断不会出现多个情况都满足的情况吗？

2018-08-10

作者回复

会依照异常表中的前后(上下)顺序来查找，然后被第一个满足条件的异常处理器捕获

2018-08-10



Krloy

0

关于try catch的疑问

如果for里面中写 try catch 一百条数据中有1条数据异常 程序正常执行 会返回99条数据

如果for里面不写 try catch写外面 程序正常执行 但是数据返回0

try catch 异常实例构造非常昂贵，因为虚拟机会生成改异常的栈轨迹，改操作会逐一访问改线程栈帧，并记录下各种调试信息。

那么如果我在for中写try catch 的话 会不会每次循环都生成一个异常实例？
上面两种写try catch的方法 哪种要更好点

2018-08-08



慢悠悠

0

Android上是throw之后，标记异常，然后去Pnd catch块，如果找到就清楚异常，否则返回

2018-08-03



廖志勤

0

异常表是有try catch的方法才会生成的吧？

2018-08-03



贾智文

0

老师，请教个问题，java可以通过反射的方式为属性设置注解吗？公司自研的框架需要每个属性都加一次注解，想反射直接全部设置上去。查了好久没有找到方式，要么是修改注解值要么查值...

2018-08-02



杨春鹏

0

Try catch Pnally的字节码组成:

如果try正常执行，则直接跳转到第三个Pnally代码块；如果try中有异常抛出并且被catch捕获，则执行第一个代码块；如果try中有异常抛出并且该异常不能被catch捕获或者catch代码块抛出的异常则异常处理器从第二个Pnally代码块开始执行。

2018-08-02



YIFENG

0

老师，在讲复制Pnally部分的图中，复制到catch部分的Pnally右边的黄色虚线指向重新抛出异常，哪种情况会走到这条黄线路径呢？

2018-08-01

作者回复

在catch代码块中用throw语句重新抛出异常

2018-08-02



吴伟

0

检查异常和非检查异常也就是其他书籍中说的编译期异常和运行时异常？

2018-08-01

作者回复

编译期异常和运行时异常这种划分有点奇怪。

检查异常也会在运行过程中抛出。但是它会要求编译器检查代码有没有显式地处理该异常。非检查异常包括Error和RuntimeException(会不会那本书直译为“运行时异常”?), 这两个则不要求编译器显式处理。

2018-08-02

讲堂 深入拆解 Java 虚拟机 文章详情

07 | JVM是如何实现反射的？

2018-08-06 郑雨迪



07 | JVM是如何实现反射的？

朗读人：郑雨迪 12'26" | 5.71M

今天我们来聊聊 Java 里的反射机制。

反射是 Java 语言中一个相当重要的特性，它允许正在运行的 Java 程序观测，甚至是修改程序的动态行为。

举例来说，我们可以通过 Class 对象枚举该类中的所有方法，我们还可以通过 Method.setAccessible（位于 java.lang.reflect 包，该方法继承自 AccessibleObject）绕过 Java 语言的访问权限，在私有方法所在类之外的地方调用该方法。

反射在 Java 中的应用十分广泛。开发人员日常接触到的 Java 集成开发环境（IDE）便运用了这一功能：每当我们敲入点号时，IDE 便会根据点号前的内容，动态展示可以访问的字段或者方法。

另一个日常应用则是 Java 调试器，它能够在调试过程中枚举某一对象所有字段的值。

```
Foo foo = new Foo();
```

```
foo.
```

- bField : boolean - Foo
- iField : int - Foo
- bar() : void - Foo
- ◆ clone() : Object - Object
- equals(Object obj) : boolean - Object
- ◆ finalize() : void - Object
- foo() : void - Foo
- getClass() : Class<?> - Object
- hashCode() : int - Object
- notify() : void - Object
- notifyAll() : void - Object

Press '^Space' to show Template Proposals

(图中 eclipse 的自动提示使用了反射)

在 Web 开发中，我们经常能够接触到各种可配置的通用框架。为了保证框架的可扩展性，它们往往借助 Java 的反射机制，根据配置文件来加载不同的类。举例来说，Spring 框架的依赖反转 (IoC)，便是依赖于反射机制。

然而，我相信不少开发人员都嫌弃反射机制比较慢。甚至是甲骨文关于反射的教学网页 [1]，也强调了反射性能开销大的缺点。

今天我们便来了解一下反射的实现机制，以及它性能糟糕的原因。如果你对反射 API 不是特别熟悉的话，你可以查阅我放在文稿末尾的附录。

反射调用的实现

首先，我们来看看方法的反射调用，也就是 Method.invoke，是怎么实现的。

```
public final class Method extends Executable {
    ...
    public Object invoke(Object obj, Object... args) throws ... {
        ... // 权限检查
        MethodAccessor ma = methodAccessor;
        if (ma == null) {
```

```

        ma = acquireMethodAccessor();
    }
    return ma.invoke(obj, args);
}
}

```

如果你查阅 `Method.invoke` 的源代码，那么你会发现，它实际上委派给 `MethodAccessor` 来处理。`MethodAccessor` 是一个接口，它有两个已有的具体实现：一个通过本地方法来实现反射调用，另一个则使用了委派模式。为了方便记忆，我使用“本地实现”和“委派实现”来指代这两者。

每个 `Method` 实例的第一次反射调用都会生成一个委派实现，它所委派的具体实现便是一个本地实现。本地实现非常容易理解。当进入了 Java 虚拟机内部之后，我们便拥有了 `Method` 实例所指向方法的具体地址。这时候，反射调用无非就是将传入的参数准备好，然后调用进入目标方法。

```

// v0 版本
import java.lang.reflect.Method;

public class Test {
    public static void target(int i) {
        new Exception("#" + i).printStackTrace();
    }

    public static void main(String[] args) throws Exception {
        Class<?> klass = Class.forName("Test");
        Method method = klass.getMethod("target", int.class);
        method.invoke(null, 0);
    }
}

# 不同版本的输出略有不同，这里我使用了 Java 10。
$ java Test
java.lang.Exception: #0
    at Test.target(Test.java:5)
    at java.base/jdk.internal.reflect.NativeMethodAccessorImpl.invoke0(Native Method)
    at java.base/jdk.internal.reflect.NativeMethodAccessorImpl.invoke(NativeMethodAccessorImpl.java:62)

```

```
t      java.base/jdk.internal.reflect.DelegatingMethodAccessorImpl.i
.invoke(DelegatingMethodAccessorImpl.java:43)
      java.base/java.lang.reflect.Method.invoke(Method.java:564)
t      Test.main(Test.java:131
```

为了方便理解，我们可以打印一下反射调用到目标方法时的栈轨迹。在上面的 v0 版本代码中，我们获取了一个指向 Test.target 方法的 Method 对象，并且用它来进行反射调用。在 Test.target 中，我会打印出栈轨迹。

可以看到，反射调用先是调用了 Method.invoke，然后进入委派实现（DelegatingMethodAccessorImpl），再然后进入本地实现（NativeMethodAccessorImpl），最后到达目标方法。

这里你可能会疑问，为什么反射调用还要采取委派实现作为中间层？直接交给本地实现不可以么？

其实，Java 的反射调用机制还设立了另一种动态生成字节码的实现（下称动态实现），直接使用 invoke 指令来调用目标方法。之所以采用委派实现，便是为了能够在本地实现以及动态实现中切换。

// 动态实现的伪代码，这里只列举了关键的调用逻辑，其实它还包括调用者检测、参数检测的字节码。

```
package jdk.internal.reflect;
```

```
public class GeneratedMethodAccessor1 extends ... {
    @Override
    public Object invoke(Object obj, Object[] args) throws ... {
        Test.target((int) args[0]);
        return null;
    }
}
```

动态实现和本地实现相比，其运行效率要快上 20 倍 [2]。这是因为动态实现无需经过 Java 到 C++ 再到 Java 的切换，但由于生成字节码十分耗时，仅调用一次的话，反而是本地实现要快上 3 到 4 倍 [3]。

考虑到许多反射调用仅会执行一次，Java 虚拟机设置了一个阈值 15（可以通过 -Dsun.reflect.inflationThreshold= 来调整），当某个反射调用的调用次数在 15 之下时，采用本地实现；当达到 15 时，便开始动态生成字节码，并将委派实现的委派对象切换至动态实现，这个过程我们称之为 Inflation。

为了观察这个过程，我将刚才的例子更改为下面的 v1 版本。它会将反射调用循环 20 次。

```
// v1 版本

import java.lang.reflect.Method;

public class Test {

    public static void target(int i) {

        new Exception("#" + i).printStackTrace();

    }

    public static void main(String[] args) throws Exception {

        Class<?> klass = Class.forName("Test");

        Method method = klass.getMethod("target", int.class);

        for (int i = 0; i < 20; i++) {

            method.invoke(null, i);

        }

    }

}

# 使用 -verbose:class 打印加载的类
$ java -verbose:class Test
...
java.lang.Exception: #14
    at Test.target(Test.java:5)
    at java.base/jdk.internal.reflect.NativeMethodAccessorImpl.invoke0(Native Method)
    at java.base/jdk.internal.reflect.NativeMethodAccessorImpl
.invoke(NativeMethodAccessorImpl.java:62)
    at java.base/jdk.internal.reflect.DelegatingMethodAccessorImpl
.invoke(DelegatingMethodAccessorImpl.java:43)
    at java.base/java.lang.reflect.Method.invoke(Method.java:564)
    at Test.main(Test.java:12)
[0.158s][info][class,load] ...
...
[0.160s][info][class,load] jdk.internal.reflect.GeneratedMethodAccessor1 source: __JVM_DefineClass__
java.lang.Exception: #15
    at Test.target(Test.java:5)
    at java.base/jdk.internal.reflect.NativeMethodAccessorImpl.invoke0(Native Method)
```



```

        at java.base/jdk.internal.reflect.NativeMethodAccessorImpl
        .invoke(NativeMethodAccessorImpl.java:62)
        at java.base/jdk.internal.reflect.DelegatingMethodAccessorImpl
        .invoke(DelegatingMethodAccessorImpl.java:43)
        at java.base/java.lang.reflect.Method.invoke(Method.java:564)
        at Test.main(Test.java:12)
java.lang.Exception: #16
        at Test.target(Test.java:5)
        at jdk.internal.reflect.GeneratedMethodAccessor1 .invoke(Unknown Source)
        at java.base/jdk.internal.reflect.DelegatingMethodAccessorImpl
        .invoke(DelegatingMethodAccessorImpl.java:43)
        at java.base/java.lang.reflect.Method.invoke(Method.java:564)
        at Test.main(Test.java:12)

...

```

可以看到，在第 15 次（从 0 开始数）反射调用时，我们便触发了动态实现的生成。这时候，Java 虚拟机额外加载了不少类。其中，最重要的当属 `GeneratedMethodAccessor1`（第 30 行）。并且，从第 16 次反射调用开始，我们便切换至这个刚刚生成的动态实现（第 40 行）。

反射调用的 `Inflation` 机制是可以通过参数（`-Dsun.reflect.noInflation=true`）来关闭的。这样一来，在反射调用一开始便会直接生成动态实现，而不会使用委派实现或者本地实现。

反射调用的开销

下面，我们便来拆解反射调用的性能开销。

在刚才的例子中，我们先后进行了 `Class.forName`，`Class.getMethod` 以及 `Method.invoke` 三个操作。其中，`Class.forName` 会调用本地方法，`Class.getMethod` 则会遍历该类的公有方法。如果没有匹配到，它还将遍历父类的公有方法。可想而知，这两个操作都非常费时。

值得注意的是，以 `getMethod` 为代表的查找方法操作，会返回查找得到结果的一份拷贝。因此，我们应当避免在热点代码中使用返回 `Method` 数组的 `getMethods` 或者 `getDeclaredMethods` 方法，以减少不必要的堆空间消耗。

在实践中，我们往往会在应用程序中缓存 `Class.forName` 和 `Class.getMethod` 的结果。因此，下面我就只关注反射调用本身的性能开销。

为了比较直接调用和反射调用的性能差距，我将前面的例子改为下面的 v2 版本。它会将反射调用循环二十亿次。此外，它还将记录下每跑一亿次的时间。

我将取最后五个记录的平均值，作为预热后的峰值性能。（注：这种性能评估方式并不严谨，我会在专栏的第三部分介绍如何用 JMH 来测性能。）

在我这个老笔记本上，一亿次直接调用耗费的时间大约在 120ms。这和不调用的时间是一致的。其原因在于这段代码属于热循环，同样会触发即时编译。并且，即时编译会将 `Test.target` 的调用内联进来，从而消除了调用的开销。

```
// v2 版本
import java.lang.reflect.Method;

public class Test {
    public static void target(int i) {
        // 空方法
    }

    public static void main(String[] args) throws Exception {
        Class<?> klass = Class.forName("Test");
        Method method = klass.getMethod("target", int.class);

        long current = System.currentTimeMillis();
        for (int i = 1; i <= 2_000_000_000; i++) {
            if (i % 100_000_000 == 0) {
                long temp = System.currentTimeMillis();
                System.out.println(temp - current);
                current = temp;
            }

            method.invoke(null, 128);
        }
    }
}
```

下面我将以 120ms 作为基准，来比较反射调用的性能开销。

由于目标方法 `Test.target` 接收一个 `int` 类型的参数，因此我传入 128 作为反射调用的参数，测得的结果约为基准的 2.7 倍。我们暂且不管这个数字是高是低，先来看看在反射调用之前字节码都做了

什么。

```

59: aload_2                // 加载 Method 对象
60: aconst_null             // 反射调用的第一个参数 null
61: iconst_1
62: anewarray Object        // 生成一个长度为 1 的 Object 数组
65: dup
66: iconst_0
67: sipush 128
70: invokestatic Integer.valueOf // 将 128 自动装箱成 Integer
73: aastore                 // 存入 Object 数组中
74: invokevirtual Method.invoke // 反射调用

```

这里我截取了循环中反射调用编译而成的字节码。可以看到，这段字节码除了反射调用外，还额外做了两个操作。

第一，由于 `Method.invoke` 是一个变长参数方法，在字节码层面它的最后一个参数会是 `Object` 数组（感兴趣的同学私下可以用 `javap` 查看）。Java 编译器会在方法调用处生成一个长度为传入参数数量的 `Object` 数组，并将传入参数一一存储进该数组中。

第二，由于 `Object` 数组不能存储基本类型，Java 编译器会对传入的基本类型参数进行自动装箱。

这两个操作除了带来性能开销外，还可能占用堆内存，使得 GC 更加频繁。（如果你感兴趣的话，可以用虚拟机参数 `-XX:+PrintGC` 试试。）那么，如何消除这部分开销呢？

关于第二个自动装箱，Java 缓存了 `[-128, 127]` 中所有整数所对应的 `Integer` 对象。当需要自动装箱的整数在这个范围之内时，便返回缓存的 `Integer`，否则需要新建一个 `Integer` 对象。

因此，我们可以将这个缓存的范围扩大至覆盖 128（对应参数 `-Djava.lang.Integer.IntegerCache.high=128`），便可以避免需要新建 `Integer` 对象的场景。

或者，我们可以在循环外缓存 128 自动装箱得到的 `Integer` 对象，并且直接传入反射调用中。这两种方法测得的结果差不多，约为基准的 1.8 倍。

现在我们再回来看看第一个因变长参数而自动生成的 `Object` 数组。既然每个反射调用对应的参数个数是固定的，那么我们可以选择在循环外新建一个 `Object` 数组，设置好参数，并直接交给反射调用。改好的代码可以参照文稿中的 v3 版本。

```
// v3 版本
```

```
import java.lang.reflect.Method;

public class Test {

    public static void target(int i) {

        // 空方法

    }

    public static void main(String[] args) throws Exception {

        Class<?> klass = Class.forName("Test");

        Method method = klass.getMethod("target", int.class);

        Object[] arg = new Object[1]; // 在循环外构造参数数组
        arg[0] = 128;

        long current = System.currentTimeMillis();
        for (int i = 1; i <= 2_000_000_000; i++) {

            if (i % 100_000_000 == 0) {

                long temp = System.currentTimeMillis();

                System.out.println(temp - current);

                current = temp;

            }

            method.invoke(null, arg);

        }

    }

}
```

测得的结果反而更糟糕了，为基准的 2.9 倍。这是为什么呢？

如果你在上一步解决了自动装箱之后查看运行时的 GC 状况，你会发现这段程序并不会触发 GC。其原因在于，原本的反射调用被内联了，从而使得即时编译器中的逃逸分析将原本新建的 Object 数组判定为不逃逸的对象。

如果一个对象不逃逸，那么即时编译器可以选择栈分配甚至是虚拟分配，也就是不占用堆空间。具体我会在本专栏的第二部分详细解释。

如果在循环外新建数组，即时编译器无法确定这个数组会不会中途被更改，因此无法优化掉访问数组的操作，可谓是得不偿失。

到目前为止，我们的最好记录是 1.8 倍。那能不能再进一步提升呢？

刚才我曾提到，可以关闭反射调用的 Inflation 机制，从而取消委派实现，并且直接使用动态实现。此外，每次反射调用都会检查目标方法的权限，而这个检查同样可以在 Java 代码里关闭，在关闭了这两项机制之后，也就得到了我们的 v4 版本，它测得的结果约为基准的 1.3 倍。

```
// v4 版本
import java.lang.reflect.Method;

// 在运行指令中添加如下两个虚拟机参数：
// -Djava.lang.Integer.IntegerCache.high=128
// -Dsun.reflect.noInflation=true
public class Test {
    public static void target(int i) {
        // 空方法
    }

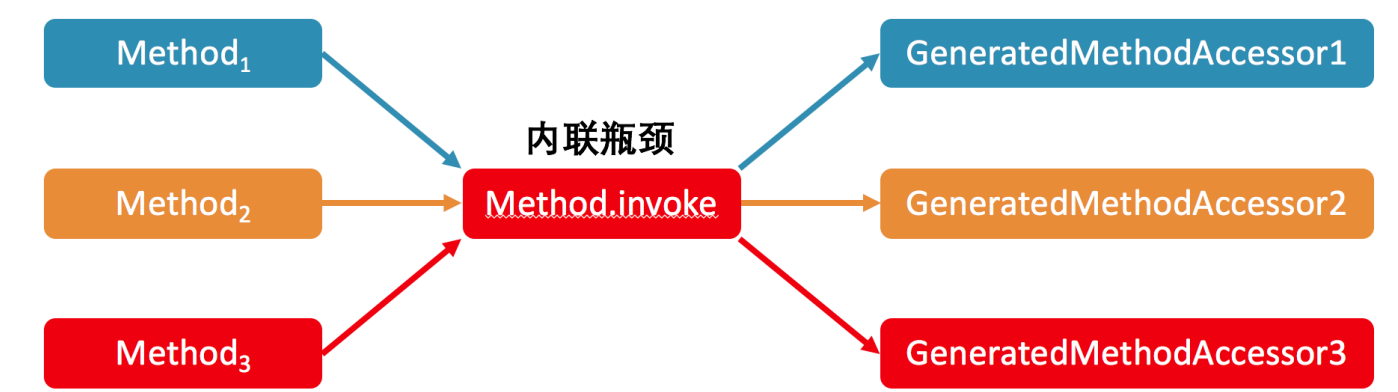
    public static void main(String[] args) throws Exception {
        Class<?> klass = Class.forName("Test");
        Method method = klass.getMethod("target", int.class);
        method.setAccessible(true); // 关闭权限检查

        long current = System.currentTimeMillis();
        for (int i = 1; i <= 2_000_000_000; i++) {
            if (i % 100_000_000 == 0) {
                long temp = System.currentTimeMillis();
                System.out.println(temp - current);
                current = temp;
            }

            method.invoke(null, 128);
        }
    }
}
```

到这里，我们基本上把反射调用的水分都榨干了。接下来，我来把反射调用的性能开销给提回去。

首先，在这个例子中，之所以反射调用能够变得这么快，主要是因为即时编译器中的方法内联。在关闭了 InBation 的情况下，内联的瓶颈在于 Method.invoke 方法中对 MethodAccessor.invoke 方法的调用。



我会在后面的文章中介绍方法内联的具体实现，这里先说个结论：在生产环境中，我们往往拥有多个不同的反射调用，对应多个 GeneratedMethodAccessor，也就是动态实现。

由于 Java 虚拟机的关于上述调用点的类型 profile（注：对于 invokevirtual 或者 invokeinterface，Java 虚拟机会记录下调用者的具体类型，我们称之为类型 profile）无法同时记录这么多个类，因此可能造成所测试的反射调用没有被内联的情况。

```
// v5 版本
import java.lang.reflect.Method;

public class Test {
    public static void target(int i) {
        // 空方法
    }

    public static void main(String[] args) throws Exception {
        Class<?> klass = Class.forName("Test");
        Method method = klass.getMethod("target", int.class);
        method.setAccessible(true); // 关闭权限检查
        polluteProfile();

        long current = System.currentTimeMillis();
        for (int i = 1; i <= 2_000_000_000; i++) {
            if (i % 100_000_000 == 0) {
```

```

        long temp = System.currentTimeMillis();

        System.out.println(temp - current);

        current = temp;
    }

    method.invoke(null, 128);
}

}

public static void polluteProfile() throws Exception {
    Method method1 = Test.class.getMethod("target1", int.class);
    Method method2 = Test.class.getMethod("target2", int.class);
    for (int i = 0; i < 2000; i++) {
        method1.invoke(null, 0);
        method2.invoke(null, 0);
    }
}

public static void target1(int i) { }
public static void target2(int i) { }
}

```

在上面的 v5 版本中，我在测试循环之前调用了 polluteProfile 的方法。该方法将反射调用另外两个方法，并且循环上 2000 遍。

而测试循环则保持不变。测得的结果约为基准的 6.7 倍。也就是说，只要误扰了 Method.invoke 方法的类型 profile，性能开销便会从 1.3 倍上升至 6.7 倍。

之所以这么慢，除了没有内联之外，另外一个原因是逃逸分析不再起效。这时候，我们便可以采用刚才 v3 版本中的解决方案，在循环外构造参数数组，并直接传递给反射调用。这样子测得的结果约为基准的 5.2 倍。

除此之外，我们还可以提高 Java 虚拟机关于每个调用能够记录的类型数目（对应虚拟机参数 -XX:TypeProfileWidth，默认值为 2，这里设置为 3）。最终测得的结果约为基准的 2.8 倍，尽管它和原本的 1.3 倍还有一定的差距，但总算是比 6.7 倍好多了。

总结与实践

今天我介绍了 Java 里的反射机制。

在默认情况下，方法的反射调用为委派实现，委派给本地实现来进行方法调用。在调用超过 15 次之后，委派实现便会将委派对象切换至动态实现。这个动态实现的字节码是自动生成的，它将直接使用 `invoke` 指令来调用目标方法。

方法的反射调用会带来不少性能开销，原因主要有三个：变长参数方法导致的 `Object` 数组，基本类型的自动装箱、拆箱，还有最重要的方法内联。

今天的实践环节，你可以将最后一段代码中 `polluteProfile` 方法的两个 `Method` 对象，都改成获取名字为“`target`”的方法。请问这两个获得的 `Method` 对象是同一个吗（`==`）？他们 `equal` 吗（`.equals(...)`）？对我们的运行结果有什么影响？

```
import java.lang.reflect.Method;

public class Test {

    public static void target(int i) {
        // 空方法
    }

    public static void main(String[] args) throws Exception {
        Class<?> klass = Class.forName("Test");
        Method method = klass.getMethod("target", int.class);
        method.setAccessible(true); // 关闭权限检查
        polluteProfile();

        long current = System.currentTimeMillis();
        for (int i = 1; i <= 2_000_000_000; i++) {
            if (i % 100_000_000 == 0) {
                long temp = System.currentTimeMillis();
                System.out.println(temp - current);
                current = temp;
            }

            method.invoke(null, 128);
        }
    }

    public static void polluteProfile() throws Exception {
```



```

Method method1 = Test.class.getMethod("target", int.class);
Method method2 = Test.class.getMethod("target", int.class);

for (int i = 0; i < 2000; i++) {
    method1.invoke(null, 0);
    method2.invoke(null, 0);
}

}

public static void target1(int i) { }
public static void target2(int i) { }

}

```

附录：反射 API 简介

通常来说，使用反射 API 的第一步便是获取 Class 对象。在 Java 中常见的有这么三种。

1. 使用静态方法 `Class.forName` 来获取。
2. 调用对象的 `getClass()` 方法。
3. 直接用类名 + ".class" 访问。对于基本类型来说，它们的包装类型（wrapper classes）拥有一个名为 "TYPE" 的 `final` 静态字段，指向该基本类型对应的 Class 对象。

例如，`Integer.TYPE` 指向 `int.class`。对于数组类型来说，可以使用类名 + "[].class" 来访问，如 `int[ ].class`。

除此之外，Class 类和 `java.lang.reflect` 包中还提供了许多返回 Class 对象的方法。例如，对于数组类的 Class 对象，调用 `Class.getComponentType()` 方法可以获得数组元素的类型。

一旦得到了 Class 对象，我们便可以正式地使用反射功能了。下面我列举了较为常用的几项。

1. 使用 `newInstance()` 来生成一个该类的实例。它要求该类中拥有一个无参数的构造器。
2. 使用 `isInstance(Object)` 来判断一个对象是否该类的实例，语法上等同于 `instanceof` 关键字（JIT 优化时会有差别，我会在本专栏的第二部分详细介绍）。
3. 使用 `Array.newInstance(Class,int)` 来构造该类型的数组。
4. 使用 `getFields()/getConstructors()/getMethods()` 来访问该类的成员。除了这三个之外，Class 类还提供了许多其他方法，详见 [4]。需要注意的是，方法名中带 `Declared` 的不会返回父类的成员，但是会返回私有成员；而不带 `Declared` 的则相反。

当获得了类成员之后，我们可以进一步做如下操作。

- 使用 `Constructor/Field/Method.setAccessible(true)` 来绕开 Java 语言的访问限制。

- 使用 `Constructor.newInstance(Object[])` 来生成该类的实例。
- 使用 `Field.get/set(Object)` 来访问字段的值。
- 使用 `Method.invoke(Object, Object[])` 来调用方法。

有关反射 API 的其他用法，可以参考 `reflect` 包的 javadoc [5]，这里就不详细展开了。

[1]: <https://docs.oracle.com/javase/tutorial/reflect/>

[2]:
<http://hg.openjdk.java.net/jdk10/jdk10/jdk/Ple/777356696811/src/java.base/share/classes/jdk/internal/reflect/ReflectionFactory.java#l80>

[3]:
<http://hg.openjdk.java.net/jdk10/jdk10/jdk/Ple/777356696811/src/java.base/share/classes/jdk/internal/reflect/ReflectionFactory.java#l78>

[4]: <https://docs.oracle.com/javase/tutorial/reflect/class/classMembers.html>

[5]: <https://docs.oracle.com/javase/10/docs/api/java/lang/reflect/package-summary.html>



版权归极客邦科技所有，未经许可不得转载

精选留言



李杰

当某个反射调用的调用次数在 15 之下时，采用本地实现；当达到 15 时，便开始动态生成字节码...
——可以认为第16次反射调用时的耗时是最长的吗？

2018-08-06

作者回复

动态生成发生在第15次(从0开始数的话)，所以第15次比较耗时。

2018-08-06



xiauguangme

7

开发人员日常接触到的 Java 集成开发环境（IDE）便运用了这一功能：每当我们敲入点号时，IDE 便会根据点号前的内容，动态展示可以访问的字段或者方法。//这个应该是不完全正确的，大部分应该是靠语法树来实现的。

2018-08-06

作者回复

谢谢指出！

2018-08-06



godtrue

5

小结

1:反射机制是Java语言的一个非常重要的特性，通过这个特性，我们能够动态的监控、调用、修改类的行为，许多的框架实现就用到了Java语言反射的机制

2:使用反射挺好的，但它也是不完美的，复杂的操作往往更耗时间和精力，使用反射也是一样，性能低下是她所被人诟病的一个地方，那为什么方法的反射如此耗费性能呐？它的性能耗在那里呢？方法的反射调用会带来不少性能开销，原因主要有三个：变长参数方法导致的 Object 数组，基本类型的自动装箱、拆箱，还有最重要的方法内联。

2018-08-07



ext4

2

雨迪您好，我有两个问题：

一是我自己的测试结果和文章中有些出入。在我自己的mac+jdk10环境中，v3版本的代码和v2版本性能是差不多的，多次测试看v3还略好一些。从v2的GC log来看for循环的每一亿次iteration中间都会有GC发生，似乎说明这里的escape analysis并没有做到allocation on stack。您能想到这是什么原因么？另有个小建议就是文章中提到测试结果时，注明一下您的环境。

另一个问题是在您v5版本的代码中，您故意用method1和method2两个对象霸占了2个ProPleType的位子，导致被测的反射操作性能很低。这是因为此处invoke方法的inline是压根儿就没有做呢？还是因为inline是依据target1或者target2来做的，而实际运行时发现类型不一致又触发了deoptimization呢？

望解答，谢谢~

2018-08-06

作者回复

多谢建议，我也是mac+jdk10。我这边裸跑v2是2.7x(因为每次要新建整数对象所以有GC)，加大整数缓存后跑v2是1.8x(无GC)。你是否忘了加大整数缓存？

第二个问题，研究得很深！Method.invoke一直会被内联，但是它里面的MethodAccesor.invoke则不一定。

实际上，在C2编译之前循环代码已经运行过非常多次，也就是说MethodAccesor.invoke 已经看到多次调用至target()的动态实现。在proPle里会显示为有target1，有target2，但是proPle不完整，即还有一大部分的调用者类型没有记录。

这时候C2会选择不inline这个MethodAccesor.invoke 调用，直接做虚调用。

2018-08-07



志远

2

老师您好，提个建议，您讲课过程中经常提到一些概念名词，您讲课总是预设了一个前提，就是假设我们已经知道那个概念，然而并不清楚。比如本文中被不断提到的内联，什么是内联呢？

2018-08-06

作者回复

谢谢建议！方法内联指的是编译器在编译一个方法时，将某个方法调用的目标方法也纳入编译范围内，并用其返回值替代原方法调用这么个过程。

2018-08-06



Kisho

2

郑老师，你好，

“动态实现无需经过Java到C++再到Java的切换”，这句话没太明白，能在解释下么？

2018-08-06

作者回复

在v0版本中我贴了一段stacktrace，你可以看到中间有个native method，这就是C++代码，也就是它先调用至这个C++代码，在C++代码里面再调用至Java代码。

2018-08-06



godtrue

2

老师请教个问题，如果手动修改某个Java字节码文件，如果JVM不重新加载此文件，有什么方式能让JVM识别并执行修改的内容呢？

如果一定需要JVM加载后才能识别并执行，有什么好的手动触发的方法呢？

2018-08-06

作者回复

你可以搜一下class redePnition的相关资料。我以前用cagent做过，Javaagent应该也可以。

2018-08-06



沉淀的梦想

1

在v5版本中，为什么我设置TypeProPleWidth为3，速度并没有变快呢？

2018-08-18



小明

1

可以看看这篇博客 有详细的生成动态调用的解释

<http://rednaxelafx.iteye.com/blog/548536>

2018-08-17



志远

1

文章中“一亿次直接调用耗费的时间大约在 120ms。这和不调用的时间是一致的。”这句话是不是病句啊？不调用指的是什么？指的是直接调用吗？

2018-08-06

作者回复

多谢指出！不调用就是字面意思，不做任何调用，也就是除了每一亿次调用的打印语句之外，循环就不包含其它东西了。

2018-08-06



Ennis LM

1

最后的问题，==是false，equals是true，对性能的影响我不知道怎么看。。。

2018-08-06



godtrue

1

JVM加载了一个Java字节码文件，在不停止JVM的情况下能再次的加载同一个Java字节码文件吗？如果能是覆盖了原来的那个Java字节码文件还是怎么着呢？

在IDE中是可以直接修改Java源代码的，然后可以手动触发Java源代码的编译和重新加载，请问老师知道IDE是怎么实现的吗？

2018-08-06



小明

0

在v2 版本测性能的过程中，加入判断 一亿次打印 和不加 时 执行时间 差距较大 导致测试的结果的偏差变大。测试性能应该将这部分的差距消除吧

2018-08-17



lee

0

反射在什么情况下，会出现无法内联呢

2018-08-16



Scott

0

有两个问题：

1.v3版本中，确定不逃逸的数组可以优化访问，这个是怎么做的？

2.v5版本中，为啥逃逸分析会失效，明明都封闭在循环里的？

2018-08-15

作者回复

1. 读数组被替换为之前写入数组的值。后面数组就只有写没有读了，因此可以优化掉。

2. 只要没有完全内联，就会将看似不逃逸的对象通过参数传递出去。即时编译器不知道所调用的方法对该对象有没有副作用，所以会将其判定为逃逸。

(如果你问的是不能分配到栈上，那我只能回答Java虚拟机从设计上不支持栈分配。它要不是堆分配，要不是虚拟分配+标量替换)

2018-08-15



姬野用菜刀

0

因此，我们应当避免在热点代码中使用返回 Method 数组的 `getMethods` 或者 `getDeclaredMethods` 方法，以减少不必要的堆空间消耗。

这个地方没有太明白，老师能帮忙在细讲一下嘛？

2018-08-14

作者回复

你可以进JRE代码看一下。相当于获取了一个Method数组，又将这个数组所有的内容都复制一份给你。

2018-08-15



阿康

0

请问，反射第16次生成字节码的用的什么方式阿？和asm有什么区别呢？

2018-08-09

作者回复

一般新的JDK代码都使用ASM来生成了。反射代码比较旧，我记得是自己构造byte数组的，非常难维护。

2018-08-10



一个坏人

0

老师好，请教一个问题。当某个反射调用的调用次数在 15 之下时，采用本地实现；当达到 15 时，便开始动态生成字节码...

——可以认为第16次反射调用时的耗时是最长的吗？如果刚好只调用16次是不是就不划算了？

2018-08-06

作者回复

首先这个数字是可调配的。

其次，是在第15次(从0开始数的话)动态生成的，也就是说第15次比较耗时。

最后，如果只调用16次，确实不划算。这是因为虚拟机猜测以后还会大量执行该反射调用，所以才动态生成字节码的。如果生成后不用，那生成字节码的时间就浪费了。

2018-08-06

讲堂 深入拆解 Java 虚拟机 文章详情

08 | JVM是怎么实现invokedynamic的？（上）

2018-08-08 郑雨迪



08 | JVM是怎么实现invokedynamic的？（上）

朗读人：郑雨迪 12'40" | 5.81M

前不久，“虚拟机”赛马俱乐部来了个年轻人，标榜自己是动态语言，是先进分子。

这一天，先进分子牵着一头鹿进来，说要参加赛马。咱部里的老学究 Java 就不同意了呀，鹿又不是马，哪能参加赛马。

当然了，这种墨守成规的调用方式，自然是先进分子所不齿的。现在年轻人里流行的是鸭子类型（duck typing）[1]，只要是跑起来像只马的，它就是一只马，也就能够参加赛马比赛。

```
class Horse {  
    public void race() {  
        System.out.println("Horse.race()");  
    }  
}
```



```

class Deer {
    public void race() {
        System.out.println("Deer.race()");
    }
}

class Cobra {
    public void race() {
        System.out.println("How do you turn this on?");
    }
}

```

(如何用同一种方式调用他们的赛跑方法?)

说到了这里，如果我们将赛跑定义为对赛跑方法（对应上述代码中的 `race()`）的调用的话，那么这个故事的关键，就在于能不能在马场中调用非马类型的赛跑方法。

为了解答这个问题，我们先来回顾一下 Java 里的方法调用。在 Java 中，方法调用会被编译为 `invokestatic`，`invokespecial`，`invokevirtual` 以及 `invokeinterface` 四种指令。这些指令与包含目标方法类名、方法名以及方法描述符的符号引用捆绑。在实际运行之前，Java 虚拟机将根据这个符号引用链接到具体的目标方法。

可以看到，在这四种调用指令中，Java 虚拟机明确要求方法调用需要提供目标方法的类名。在这种体系下，我们有两个解决方案。一是调用其中一种类型的赛跑方法，比如说马类的赛跑方法。对于非马的类型，则给它套一层马甲，当成马来赛跑。

另外一种解决方式，是通过反射机制，来查找并且调用各个类型中的赛跑方法，以此模拟真正的赛跑。

显然，比起直接调用，这两种方法都相当复杂，执行效率也可想而知。为了解决这个问题，Java 7 引入了一条新的指令 `invokedynamic`。该指令的调用机制抽象出调用点这一个概念，并允许应用程序将调用点链接至任意符合条件的方法上。

```

public static void startRace(java.lang.Object)
    0: aload_0           // 加载一个任意对象
    1: invokedynamic race // 调用赛跑方法

```


(理想的调用方式)

作为 invokedynamic 的准备工作，Java 7 引入了更加底层、更加灵活的方法抽象：方法句柄（MethodHandle）。

方法句柄的概念

方法句柄是一个强类型的，能够被直接执行的引用 [2]。该引用可以指向常规的静态方法或者实例方法，也可以指向构造器或者字段。当指向字段时，方法句柄实则指向包含字段访问字节码的虚构方法，语义上等价于目标字段的 getter 或者 setter 方法。

这里需要注意的是，它并不会直接指向目标字段所在类中的 getter/setter，毕竟你无法保证已有的 getter/setter 方法就是在访问目标字段。

方法句柄的类型（MethodType）是由所指向方法的参数类型以及返回类型组成的。它是用来确认方法句柄是否适配的唯一关键。当使用方法句柄时，我们其实并不关心方法句柄所指向方法的类名或者方法名。

打个比方，如果兔子的“赛跑”方法和“睡觉”方法的参数类型以及返回类型一致，那么对于兔子递过来的一个方法句柄，我们并不知道会是哪一个方法。

方法句柄的创建是通过 MethodHandles.Lookup 类来完成的。它提供了多个 API，既可以使用反射 API 中的 Method 来查找，也可以根据类、方法名以及方法句柄类型来查找。

当使用后者这种查找方式时，用户需要区分具体的调用类型，比如说对于用 invokestatic 调用的静态方法，我们需要使用 Lookup.PndStatic 方法；对于用 invokevirtual 调用的实例方法，以及用 invokeinterface 调用的接口方法，我们需要使用 PndVirtual 方法；对于用 invokespecial 调用的实例方法，我们则需要使用 PndSpecial 方法。

调用方法句柄，和原本对应的调用指令是一致的。也就是说，对于原本用 invokevirtual 调用的方法句柄，它也会采用动态绑定；而对于原本用 invokespecial 调用的方法句柄，它会采用静态绑定。

```
class Foo {  
    private static void bar(Object o) {  
        ..  
    }  
    public static Lookup lookup() {  
        return MethodHandles.lookup();  
    }  
}
```

```
// 获取方法句柄的不同方式

MethodHandles.Lookup l = Foo.lookup(); // 具备 Foo 类的访问权限

Method m = Foo.class.getDeclaredMethod("bar", Object.class);

MethodHandle mh0 = l.unreflect(m);

MethodType t = MethodType.methodType(void.class, Object.class);

MethodHandle mh1 = l.findStatic(Foo.class, "bar", t);
```

方法句柄同样也有权限问题。但它与反射 API 不同，其权限检查是在句柄的创建阶段完成的。在实际调用过程中，Java 虚拟机并不会检查方法句柄的权限。如果该句柄被多次调用的话，那么与反射调用相比，它将省下重复权限检查的开销。

需要注意的是，方法句柄的访问权限不取决于方法句柄的创建位置，而是取决于 Lookup 对象的创建位置。

举个例子，对于一个私有字段，如果 Lookup 对象是在私有字段所在类中获取的，那么这个 Lookup 对象便拥有对该私有字段的访问权限，即使是在所在类的外边，也能够通过该 Lookup 对象创建该私有字段的 getter 或者 setter。

由于方法句柄没有运行时权限检查，因此，应用程序需要负责方法句柄的管理。一旦它发布了某些指向私有方法的方法句柄，那么这些私有方法便被暴露出去了。

方法句柄的操作

方法句柄的调用可分为两种，一是需要严格匹配参数类型的 `invokeExact`。它有多严格呢？假设一个方法句柄将接收一个 `Object` 类型的参数，如果你直接传入 `String` 作为实际参数，那么方法句柄的调用会在运行时抛出方法类型不匹配的异常。正确的调用方式是将其 `String` 显式转化为 `Object` 类型。

在普通 Java 方法调用中，我们只有在选择重载方法时，才会用到这种显式转化。这是因为经过显式转化后，参数的声明类型发生了改变，因此有可能匹配到不同的方法描述符，从而选取不同的目标方法。调用方法句柄也是利用同样的原理，并且涉及了一个签名多态性（signature polymorphism）的概念。（在这里我们暂且认为签名等同于方法描述符。）

```
public final native @PolymorphicSignature Object invokeExact(Object... args) throws Throwable;
```

方法句柄 API 有一个特殊的注解类 `@PolymorphicSignature`。在碰到被它注解的方法调用时，Java 编译器会根据所传入参数的声明类型来生成方法描述符，而不是采用目标方法所声明的描述符。

在刚才的例子中，当传入的参数是 `String` 时，对应的方法描述符包含 `String` 类；而当我们转化为

Object 时，对应的方法描述符则包含 Object 类。

```
public void test(MethodHandle mh, String s) throws Throwable {
    mh.invokeExact(s);
    mh.invokeExact((Object) s);
}

// 对应的 Java 字节码
public void test(MethodHandle, String) throws java.lang.Throwable;
Code:
    0: aload_1
    1: aload_2
    2: invokevirtual MethodHandle.invokeExact:(Ljava/lang/String;)V
    5: aload_1
    6: aload_2
    7: invokevirtual MethodHandle.invokeExact:(Ljava/lang/Object;)V
   10: return
```

invokeExact 会确认该 invokevirtual 指令对应的方法描述符，和该方法句柄的类型是否严格匹配。在不匹配的情况下，便会在运行时抛出异常。

如果你需要自动适配参数类型，那么你可以选取方法句柄的第二种调用方式 invoke。它同样是一个签名多态性的方法。invoke 会调用 MethodHandle.asType 方法，生成一个适配器方法句柄，对传入的参数进行适配，再调用原方法句柄。调用原方法句柄的返回值同样也会先进行适配，然后再返回给调用者。

方法句柄还支持增删改参数的操作，这些操作都是通过生成另一个方法句柄来实现的。这其中，改操作就是刚刚介绍的 MethodHandle.asType 方法。删操作指的是将传入的部分参数就地抛弃，再调用另一个方法句柄。它对应的 API 是 MethodHandles.dropArguments 方法。

增操作则非常有意思。它会往传入的参数中插入额外的参数，再调用另一个方法句柄，它对应的 API 是 MethodHandle.bindTo 方法。Java 8 中捕获类型的 Lambda 表达式便是用这种操作来实现的，下一篇我会详细进行解释。

增操作还可以用来实现方法的柯里化 [3]。举个例子，有一个指向 $f(x, y)$ 的方法句柄，我们可以通过将 x 绑定为 4，生成另一个方法句柄 $g(y) = f(4, y)$ 。在执行过程中，每当调用 $g(y)$ 的方法句柄，它会在参数列表最前面插入一个 4，再调用指向 $f(x, y)$ 的方法句柄。

方法句柄的实现

下面我们来看看 HotSpot 虚拟机中方法句柄调用的具体实现。（由于篇幅原因，这里只讨论 `DirectMethodHandle`。）

前面提到，调用方法句柄所使用的 `invokeExact` 或者 `invoke` 方法具备签名多态性的特性。它们会根据具体的传入参数来生成方法描述符。那么，拥有这个描述符的方法实际存在吗？对 `invokeExact` 或者 `invoke` 的调用具体会进入哪个方法呢？

```
import java.lang.invoke.*;

public class Foo {

    public static void bar(Object o) {
        new Exception().printStackTrace();
    }

    public static void main(String[] args) throws Throwable {
        MethodHandles.Lookup l = MethodHandles.lookup();
        MethodType t = MethodType.methodType(void.class, Object.class);
        MethodHandle mh = l.findStatic(Foo.class, "bar", t);
        mh.invokeExact(new Object());
    }
}
```

和查阅反射调用的方式一样，我们可以通过新建异常实例来查看栈轨迹。打印出来的栈轨迹如下所示：

```
$ java Foo
java.lang.Exception
    at Foo.bar(Foo.java:5)
    at Foo.main(Foo.java:12)
```

也就是说，`invokeExact` 的目标方法竟然就是方法句柄指向的方法。

先别高兴太早。我刚刚提到过，`invokeExact` 会对参数的类型进行校验，并在不匹配的情况下抛出异常。如果它直接调用了方法句柄所指向的方法，那么这部分参数类型校验的逻辑将无处安放。因此，唯一的可能便是 Java 虚拟机隐藏了部分栈信息。

当我们启用了 `-XX:+ShowHiddenFrames` 这个参数来打印被 Java 虚拟机隐藏了的栈信息时，你会发现 `main` 方法和目标方法中间隔着两个貌似是生成的方法。

```
$ java -XX:+UnlockDiagnosticVMOptions -XX:+ShowHiddenFrames Foo
java.lang.Exception
    at Foo.bar(Foo.java:5)
    at java.base/java.lang.invoke.DirectMethodHandle$Holder.
invokeStatic(DirectMethodHandle$Holder:1000010)
    at java.base/java.lang.invoke.LambdaForm$MH000/766572210.
invokeExact_MT000_LLL_V(LambdaForm$MH000:1000019)
    at Foo.main(Foo.java:12)
```

实际上，Java 虚拟机会对 `invokeExact` 调用做特殊处理，调用至一个共享的、与方法句柄类型相关的特殊适配器中。这个适配器是一个 `LambdaForm`，我们可以通过添加虚拟机参数将之导出成 class 文件（`-Djava.lang.invoke.MethodHandle.DUMP_CLASS_FILES=true`）。

```
final class java.lang.invoke.LambdaForm$MH000 { static void
invokeExact_MT000_LLLL_V(jeava.lang.bjject, jjava.lang.bjject, jjava.lang.bjject);

Code:
    : aload_0
    1 : checkcast    #14                //Mclass java/lang/invoke/ethodHandle
    : dup
    5 : astore_0
    : aload_32      : checkcast    #16                //Mclass java/lang/invoke/ethodType
    10: invokestatic  I#22                // Method java/lang/invoke/nvokers.checkExactType:
(MLjava/lang/invoke/ethodHandle;;Ljava/lang/invoke/ethodType);V
    13: aload_0
    14: invokestatic  #26      I          // Method java/lang/invoke/nvokers.checkCustomized:
(MLjava/lang/invoke/ethodHandle);V
    17: aload_0
    18: aload_1
    19: ainvokevirtudl #30                2    // Methodijava/lang/nvokev/ethodHandle.invokeBasic:
(LLjava/lang/bjject;;V
    23 return
```

可以看到，在这个适配器中，它会调用 `Invokers.checkExactType` 方法来检查参数类型，然后调用 `Invokers.checkCustomized` 方法。后者会在方法句柄的执行次数超过一个阈值时进行优化（对应参数 `-Djava.lang.invoke.MethodHandle.CUSTOMIZE_THRESHOLD`，默认值为 127）。最后，它会调用方法句柄的 `invokeBasic` 方法。

Java 虚拟机同样会对 `invokeBasic` 调用做特殊处理，这会将调用至方法句柄本身所持有的适配器中。这个适配器同样是一个 `LambdaForm`，你可以通过反射机制将其打印出来。

```
// 该方法句柄持有的 LambdaForm 实例的 toString() 结果
DMH.invokeStatic_LL_V=Lambda(a0:L,a1:L)=>{
    t2:L=DirectMethodHandle.internalMemberName(a0:L);
    t3:V=MethodHandle.linkToStatic(a1:L,t2:L);void}
```

这个适配器将获取方法句柄中的 `MemberName` 类型的字段，并且以它为参数调用 `linkToStatic` 方法。估计你已经猜到了，Java 虚拟机也会对 `linkToStatic` 调用做特殊处理，它将根据传入的 `MemberName` 参数所存储的方法地址或者方法表索引，直接跳转至目标方法。

```
final class MemberName implements Member, Cloneable {
    ...
    //@Injected JVM_Method* vmtarget;
    //@Injected int      vmindex;
    ...
}
```

那么前面那个适配器中的优化又是怎么回事？实际上，方法句柄一开始持有的适配器是共享的。当它被多次调用之后，`Invokers.checkCustomized` 方法会为该方法句柄生成一个特有的适配器。这个特有的适配器会将方法句柄作为常量，直接获取其 `MemberName` 类型的字段，并继续后面的 `linkToStatic` 调用。

```
final class java.lang.invoke.LambdaForm$DMH000 {
    static void invokeStatic000_LL_V(java.lang.Object, java.lang.Object);
    Code:
        0: ldc          #14          // String CONSTANT_PLACEHOLDER_1
    <<Foo.bar(Object)void/invokeStatic>>
        2: checkcast     #16          // class java/lang/invoke/MethodHandle
        5: astore_0      // 上面的优化代码覆盖了传入的方法句柄
        6: aload_0       // 从这里开始跟初始版本一致
        7: invokestatic  #22          // Method
```

```

java/lang/invoke/DirectMethodHandle.internalMemberName:(Ljava/lang/Object;)Ljava/lang/Object;

    10: astore_2
    11: aload_1
    12: aload_2
    13: checkcast    #24                // class java/lang/invoke/MemberName
    16: invokestatic #28                // Method java/lang/invoke/MethodHandle.linkToStatic:
(Ljava/lang/Object;Ljava/lang/invoke/MemberName;)V
    19: return

```

可以看到，方法句柄的调用和反射调用一样，都是间接调用。因此，它也会面临无法内联的问题。不过，与反射调用不同的是，方法句柄的内联瓶颈在于即时编译器能否将该方法句柄识别为常量。具体内容我会在下一篇中进行详细的解释。

总结与实践

今天我介绍了 invokedynamic 底层机制的基石：方法句柄。

方法句柄是一个强类型的、能够被直接执行的引用。它仅关心所指向方法的参数类型以及返回类型，而不关心方法所在的类以及方法名。方法句柄的权限检查发生在创建过程中，相较于反射调用节省了调用时反复权限检查的开销。

方法句柄可以通过 invokeExact 以及 invoke 来调用。其中，invokeExact 要求传入的参数和所指向方法的描述符严格匹配。方法句柄还支持增删改参数的操作，这些操作是通过生成另一个充当适配器的方法句柄来实现的。

方法句柄的调用和反射调用一样，都是间接调用，同样会面临无法内联的问题。

今天的实践环节，我们来测量一下方法句柄的性能。你可以尝试通过重构代码，将方法句柄变成常量，来提升方法句柄调用的性能。

```

public class Foo {
    public void bar(Object o) {
    }

    public static void main(String[] args) throws Throwable {
        MethodHandles.Lookup l = MethodHandles.lookup();
        MethodType t = MethodType.methodType(void.class, Object.class);
        MethodHandle mh = l.findVirtual(Foo.class, "bar", t);
    }
}

```

```
long current = System.currentTimeMillis();
for (int i = 1; i <= 2_000_000_000; i++) {
    if (i % 100_000_000 == 0) {
        long temp = System.currentTimeMillis();
        System.out.println(temp - current);
        current = temp;
    }
    mh.invokeExact(new Foo(), new Object());
}
}
```

[1] https://en.wikipedia.org/wiki/Duck_typing

[2]

<https://docs.oracle.com/javase/10/docs/api/java/lang/invoke/MethodHandle.html>

[3]

<https://en.wikipedia.org/wiki/Currying>



版权归极客邦科技所有，未经许可不得转载

精选留言



三木子

那么方法句柄是否可以取代反射了呢？

2018-08-08



秋天

1

这个东西的应用场景是什么？讲的挺深，联系不起来知识

2018-08-13



雨亦奇

1

个人觉得还是按老师的课程安排来走吧。跳来跳去的讲可能会零散不系统。上面两位的说法我不赞同。

2018-08-09



ext4

0

雨迪，我看了一下MethodHandle的增操作，即你所提到的bindTo 这个API，它貌似只能用于为virtual method绑定第一个参数（即caller的this*指针），并不能普适地为方法绑定一个任意参数（例如把参数列表(int, int)里的第一个参数绑定为常数4）。那么你例子中所提到的更为一般性的柯里化又是怎么实现的呢？

2018-08-09

作者回复

多谢指出：)

bindTo 确实限制了只能使用引用类型，而且正如你所说普遍是用来绑定this的。但是由于方法句柄不区分调用者和参数，所以还是可以滥用的。

你可以试试用Integer，然后使用静态方法，或者在使用virtual方法时将bindTo 返回的方法句柄再bindTo 一个Integer.valueOf(4)

BoundMethodHandle里有各种非公有的bindArgument*方法，感兴趣可以了解一下

2018-08-10



齐帆

0

老师能讲下JVM对线程的实现方式吗？

2018-08-08

讲堂 深入拆解 Java 虚拟机 文章详情

09 | JVM是怎么实现invokedynamic的？（下）

2018-08-10 郑雨迪



09 | JVM是怎么实现invokedynamic的？（下）

朗读人：郑雨迪 11'38" | 5.34M

上回讲到，为了让所有的动物都能参加赛马，Java 7 引入了 `invokedynamic` 机制，允许调用任意类的“赛跑”方法。不过，我们并没有讲解 `invokedynamic`，而是深入地探讨了它所依赖的方法句柄。

今天，我便来正式地介绍 `invokedynamic` 指令，讲讲它是如何生成调用点，并且允许应用程序自己决定链接至哪一个方法中的。

`invokedynamic` 指令

`invokedynamic` 是 Java 7 引入的一条新指令，用以支持动态语言的方法调用。具体来说，它将调用点（`CallSite`）抽象成一个 Java 类，并且将原本由 Java 虚拟机控制的方法调用以及方法链接暴露给了应用程序。在运行过程中，每一条 `invokedynamic` 指令将捆绑一个调用点，并且会调用该调用点所链接的方法句柄。

在第一次执行 `invokedynamic` 指令时，Java 虚拟机会调用该指令所对应的启动方法（`Bootstrap Method`），来生成前面提到的调用点，并且将之绑定至该 `invokedynamic` 指令中。在之后的运行

过程中，Java 虚拟机则会直接调用绑定的调用点所链接的方法句柄。

在字节码中，启动方法是用方法句柄来指定的。这个方法句柄指向一个返回类型为调用点的静态方法。该方法必须接收三个固定的参数，分别为一个 Lookup 类实例，一个用来指代目标方法名字的字符串，以及该调用点能够链接的方法句柄的类型。

除了这三个必需参数之外，启动方法还可以接收若干个其他的参数，用来辅助生成调用点，或者定位所要链接的目标方法。

```
import java.lang.invoke.*;

class Horse {
    public void race() {
        System.out.println("Horse.race()");
    }
}

class Deer {
    public void race() {
        System.out.println("Deer.race()");
    }
}

// javac Circuit.java
// java Circuit
public class Circuit {

    public static void startRace(Object obj) {
        // aload obj
        // invokedynamic race()
    }

    public static void main(String[] args) {
        startRace(new Horse());
        // startRace(new Deer());
    }

    public static CallSite bootstrap(MethodHandles.Lookup l, String name, MethodType callSiteType)
```

```
throws Throwable {
    MethodHandle mh = l.findVirtual(Horse.class, name, MethodType.methodType(void.class));
    return new ConstantCallSite(mh.asType(callSiteType));
}
}
```

我在文稿中贴了一段代码，其中便包含一个启动方法。它将接收前面提到的三个固定参数，并且返回一个链接至 `Horse.race` 方法的 `ConstantCallSite`。

这里的 `ConstantCallSite` 是一种不可以更改链接对象的调用点。除此之外，Java 核心类库还提供多种可以更改链接对象的调用点，比如 `MutableCallSite` 和 `VolatileCallSite`。

这两者的区别就好比正常字段和 `volatile` 字段之间的区别。此外，应用程序还可以自定义调用点类，来满足特定的重链接需求。

由于 Java 暂不支持直接生成 `invokedynamic` 指令 [1]，所以接下来我会借助之前介绍过的字节码工具 ASM 来实现这一目的。

```
import java.io.IOException;
import java.lang.invoke.*;
import java.nio.file.*;

import org.objectweb.asm.*;

// javac -cp /path/to/asm-all-6.0_BETA.jar:. ASMHelper.java
// java -cp /path/to/asm-all-6.0_BETA.jar:. ASMHelper
// java Circuit

public class ASMHelper implements Opcodes {

    private static class MyMethodVisitor extends MethodVisitor {

        private static final String BOOTSTRAP_CLASS_NAME = Circuit.class.getName().replace('.', '/');
        private static final String BOOTSTRAP_METHOD_NAME = "bootstrap";
        private static final String BOOTSTRAP_METHOD_DESC = MethodType
            .methodType(CallSite.class, MethodHandles.Lookup.class, String.class, MethodType.class)
            .toMethodDescriptorString();

        private static final String TARGET_METHOD_NAME = "race";
```

```

private static final String TARGET_METHOD_DESC = "(Ljava/lang/Object;)V";

public final MethodVisitor mv;

public MyMethodVisitor(int api, MethodVisitor mv) {
    super(api);
    this.mv = mv;
}

@Override
public void visitCode() {
    mv.visitCode();
    mv.visitVarInsn(ALOAD, 0);
    Handle h = new Handle(H_INVOKESTATIC, BOOTSTRAP_CLASS_NAME, BOOTSTRAP_METHOD_NAME,
BOOTSTRAP_METHOD_DESC, false);
    mv.visitInvokeDynamicInsn(TARGET_METHOD_NAME, TARGET_METHOD_DESC, h);
    mv.visitInsn(RETURN);
    mv.visitMaxs(1, 1);
    mv.visitEnd();
}
}

public static void main(String[] args) throws IOException {
    ClassReader cr = new ClassReader("Circuit");
    ClassWriter cw = new ClassWriter(cr, ClassWriter.COMPUTE_FRAMES);
    ClassVisitor cv = new ClassVisitor(ASM6, cw) {
        @Override
        public MethodVisitor visitMethod(int access, String name, String descriptor, String signature,
            String[] exceptions) {
            MethodVisitor visitor = super.visitMethod(access, name, descriptor, signature, exceptions);
            if ("startRace".equals(name)) {
                return new MyMethodVisitor(ASM6, visitor);
            }
            return visitor;
        }
    };
    cr.accept(cv, ClassReader.SKIP_FRAMES);
}

```

```

        Files.write(Paths.get("Circuit.class"), cw.toByteArray());
    }
}

```

你无需理解上面这段代码的具体含义，只须了解它会更改同一目录下 `Circuit` 类的 `startRace(Object)` 方法，使之包含 `invokedynamic` 指令，执行所谓的赛跑方法。

```

public static void startRace(java.lang.Object);
    0: aload_0
    1: invokedynamic #80, 0 // race:(Ljava/lang/Object;)V
    6: return

```

如果你足够细心的话，你会发现该指令所调用的赛跑方法的描述符，和 `Horse.race` 方法或者 `Deer.race` 方法的描述符并不一致。这是因为 `invokedynamic` 指令最终调用的是方法句柄，而方法句柄会将调用者当成第一个参数。因此，刚刚提到的那两个方法恰恰符合这个描述符所对应的方法句柄类型。

到目前为止，我们已经可以通过 `invokedynamic` 调用 `Horse.race` 方法了。为了支持调用任意类的 `race` 方法，我实现了一个简单的单态内联缓存。如果调用者的类型命中缓存中的类型，便直接调用缓存中的方法句柄，否则便更新缓存。

```

// 需要更改 ASMHelper.MyMethodVisitor 中的 BOOTSTRAP_CLASS_NAME
import java.lang.invoke.*;

public class MonomorphicInlineCache {

    private final MethodHandles.Lookup lookup;
    private final String name;

    public MonomorphicInlineCache(MethodHandles.Lookup lookup, String name) {
        this.lookup = lookup;
        this.name = name;
    }

    private Class<?> cachedClass = null;

```

```

private MethodHandle mh = null;

public void invoke(Object receiver) throws Throwable {
    if (cachedClass != receiver.getClass()) {
        cachedClass = receiver.getClass();
        mh = lookup.findVirtual(cachedClass, name, MethodType.methodType(void.class));
    }
    mh.invoke(receiver);
}

public static CallSite bootstrap(MethodHandles.Lookup l, String name, MethodType callSiteType)
throws Throwable {
    MonomorphicInlineCache ic = new MonomorphicInlineCache(l, name);
    MethodHandle mh = l.findVirtual(MonomorphicInlineCache.class, "invoke",
MethodType.methodType(void.class, Object.class));
    return new ConstantCallSite(mh.bindTo(ic));
}
}

```

可以看到，尽管 `invokedynamic` 指令调用的是所谓的 `race` 方法，但是实际上我返回了一个链接至名为“`invoke`”的方法的调用点。由于调用点仅要求方法句柄的类型能够匹配，因此这个链接是合法的。

不过，这正是 `invokedynamic` 的目的，也就是将调用点与目标方法的链接交由应用程序来做，并且依赖于应用程序对目标方法进行验证。所以，如果应用程序将赛跑方法链接至兔子的睡觉方法，那也只能怪应用程序自己了。

Java 8 的 Lambda 表达式

在 Java 8 中，Lambda 表达式也是借助 `invokedynamic` 来实现的。

具体来说，Java 编译器利用 `invokedynamic` 指令来生成实现了函数式接口的适配器。这里的函数式接口指的是仅包括一个非 default 接口方法的接口，一般通过 `@FunctionalInterface` 注解。不过就算是没有使用该注解，Java 编译器也会将符合条件的接口辨认为函数式接口。

```

int x = ..

IntStream.of(1, 2, 3).map(i -> i * 2).map(i -> i * x);

```

举个例子，上面这段代码会对 `IntStream` 中的元素进行两次映射。我们知道，映射方法 `map` 所接收的参数是 `IntUnaryOperator`（这是一个函数式接口）。也就是说，在运行过程中我们需要将 `i -> i*2` 和 `i -> i*x` 这两个 Lambda 表达式转化成 `IntUnaryOperator` 的实例。这个转化过程便是由 `invokedynamic` 来实现的。

在编译过程中，Java 编译器会对 Lambda 表达式进行解语法糖（desugar），生成一个方法来保存 Lambda 表达式的内容。该方法的参数列表不仅包含原本 Lambda 表达式的参数，还包含它所捕获的变量。（注：方法引用，如 `Horse::race`，则不会生成额外的方法。）

在上面那个例子中，第一个 Lambda 表达式没有捕获其他变量，而第二个 Lambda 表达式（也就是 `i -> i*x`）则会捕获局部变量 `x`。这两个 Lambda 表达式对应的方法如下所示。可以看到，所捕获的变量同样也会作为参数传入生成的方法之中。

```
// i -> i * 2
private static int lambda$0(int);

Code:
    0: iload_0
    1: iconst_2
    2: imul
    3: ireturn

// i -> i * x
private static int lambda$1(int, int);

Code:
    0: iload_1
    1: iload_0
    2: imul
    3: ireturn
```

第一次执行 `invokedynamic` 指令时，它所对应的启动方法会通过 ASM 来生成一个适配器类。这个适配器类实现了对应的函数式接口，在我们的例子中，也就是 `IntUnaryOperator`。启动方法的返回值是一个 `ConstantCallSite`，其链接对象为一个返回适配器类实例的方法句柄。

根据 Lambda 表达式是否捕获其他变量，启动方法生成的适配器类以及所链接的方法句柄皆不同。

如果该 Lambda 表达式没有捕获其他变量，那么可以认为它是上下文无关的。因此，启动方法将新建一个适配器类的实例，并且生成一个特殊的方法句柄，始终返回该实例。

如果该 Lambda 表达式捕获了其他变量，那么每次执行该 `invokedynamic` 指令，我们都要更新这

些捕获了的变量，以防止它们发生了变化。

另外，为了保证 Lambda 表达式的线程安全，我们无法共享同一个适配器类的实例。因此，在每次执行 invokedynamic 指令时，所调用的方法句柄都需要新建一个适配器类实例。

在这种情况下，启动方法生成的适配器类将包含一个额外的静态方法，来构造适配器类的实例。该方法将接收这些捕获的参数，并且将它们保存为适配器类实例的实例字段。

你可以通过虚拟机参数 -Djdk.internal.lambda.dumpProxyClasses=/DUMP/PATH 导出这些具体的适配器类。这里我导出了上面这个例子中两个 Lambda 表达式对应的适配器类。

```
// i->i*2 对应的适配器类
final class LambdaTest$$Lambda$1 implements IntUnaryOperator {
    private LambdaTest$$Lambda$1();

    Code:
        0: aload_0
        1: invokespecial java/lang/Object."<init>":()V
        4: return

    public int applyAsInt(int);
    Code:
        0: iload_1
        1: invokestatic LambdaTest.lambda$0:(I)I
        4: ireturn
}

// i->i*x 对应的适配器类
final class LambdaTest$$Lambda$2 implements IntUnaryOperator {
    private final int arg$1;

    private LambdaTest$$Lambda$2(int);

    Code:
        0: aload_0
        1: invokespecial java/lang/Object."<init>":()V
        4: aload_0
        5: iload_1
        6: putfield arg$1:I
        9: return
}
```

```
private static java.util.function.IntUnaryOperator get$Lambda(int);
```

Code:

```
0: new LambdaTest$$Lambda$2
3: dup
4: iload_0
5: invokespecial "<init>":(I)V
8: areturn
```

```
public int applyAsInt(int);
```

Code:

```
0: aload_0
1: getfield arg$1:I
4: iload_1
5: invokestatic LambdaTest.lambda$1:(II)I
8: ireturn
```

```
}
```

可以看到，捕获了局部变量的 Lambda 表达式多出了一个 get\$Lambda 的方法。启动方法便会所返回的调用点链接至指向该方法的方法句柄。也就是说，每次执行 invokedynamic 指令时，都会调用至这个方法中，并构造一个新的适配器类实例。

这个多出来的新建实例会对程序性能造成影响吗？

Lambda 以及方法句柄的性能分析

我再次请出测试反射调用性能开销的那段代码，并将其改造成使用 Lambda 表达式的 v6 版本。

```
// v6 版本
```

```
import java.util.function.IntConsumer;
```

```
public class Test {
```

```
    public static void target(int i) { }
```

```
    public static void main(String[] args) throws Exception {
```

```
        long current = System.currentTimeMillis();
```

```
        for (int i = 1; i <= 2_000_000_000; i++) {
```

```
            if (i % 100_000_000 == 0) {
```

```

        long temp = System.currentTimeMillis();

        System.out.println(temp - current);

        current = temp;
    }

    ((IntConsumer) j -> Test.target(j)).accept(128);
    // ((IntConsumer) Test::target).accept(128);
}
}
}

```

测量结果显示，它与直接调用的性能并无太大的区别。也就是说，即时编译器能够将转换 Lambda 表达式所使用的 invokedynamic，以及对 IntConsumer.accept 方法的调用统统内联进来，最终优化为空操作。

这个其实不难理解：Lambda 表达式所使用的 invokedynamic 将绑定一个 ConstantCallSite，其链接的目标方法无法改变。因此，即时编译器会将该目标方法直接内联进来。对于这类没有捕获变量的 Lambda 表达式而言，目标方法只完成了一个动作，便是加载缓存的适配器类常量。

另一方面，对 IntConsumer.accept 方法的调用实则是对适配器类的 accept 方法的调用。

如果你查看了 accept 方法对应的字节码的话，你会发现它仅包含一个方法调用，调用至 Java 编译器在解 Lambda 语法糖时生成的方法。

该方法的内容便是 Lambda 表达式的内容，也就是直接调用目标方法 Test.target。将这几个方法调用内联进来之后，原本对 accept 方法的调用则会被优化为空操作。

下面我将之前的代码更改为带捕获变量的 v7 版本。理论上，每次调用 invokedynamic 指令，Java 虚拟机都会新建一个适配器类的实例。然而，实际运行结果还是与直接调用的性能一致。

```

// v7 版本

import java.util.function.IntConsumer;

public class Test {

    public static void target(int i) { }

    public static void main(String[] args) throws Exception {

        int x = 2;
    }
}

```

```

    long current = System.currentTimeMillis();
    for (int i = 1; i <= 2_000_000_000; i++) {
        if (i % 100_000_000 == 0) {
            long temp = System.currentTimeMillis();
            System.out.println(temp - current);
            current = temp;
        }

        ((IntConsumer) j -> Test.target(x + j)).accept(128);
    }
}
}

```

显然，即时编译器的逃逸分析又将该新建实例给优化掉了。我们可以通过虚拟机参数 `-XX:-DoEscapeAnalysis` 来关闭逃逸分析。果然，这时候测得的值约为直接调用的 2.5 倍。

尽管逃逸分析能够去除这些额外新建实例开销，但是它也不是时时奏效。它需要同时满足两件事：`invokedynamic` 指令所执行的方法句柄能够内联，和接下来的对 `accept` 方法的调用也能内联。

只有这样，逃逸分析才能判定该适配器实例不逃逸。否则，我们会在运行过程中不停地生成适配器类实例。所以，我们应当尽量使用非捕获的 Lambda 表达式。

总结与实践

今天我介绍了 `invokedynamic` 指令以及 Lambda 表达式的实现。

`invokedynamic` 指令抽象出调用点的概念，并且将调用该调用点所链接的方法句柄。在第一次执行 `invokedynamic` 指令时，Java 虚拟机将执行它所对应的启动方法，生成并且绑定一个调用点。之后如果再次执行该指令，Java 虚拟机则直接调用已经绑定了的调用点所链接的方法。

Lambda 表达式到函数式接口的转换是通过 `invokedynamic` 指令来实现的。该 `invokedynamic` 指令对应的启动方法将通过 ASM 生成一个适配器类。

对于没有捕获其他变量的 Lambda 表达式，该 `invokedynamic` 指令始终返回同一个适配器类的实例。对于捕获了其他变量的 Lambda 表达式，每次执行 `invokedynamic` 指令将新建一个适配器类实例。

不管是捕获型的还是未捕获型的 Lambda 表达式，它们的性能上限皆可以达到直接调用的性能。其中，捕获型 Lambda 表达式借助了即时编译器中的逃逸分析，来避免实际的新建适配器类实例的操

作。

在上一篇的课后实践中，你应该测过这一段代码的性能开销了。我这边测得的结果约为直接调用的 3.5 倍。

```
// v8 版本

import java.lang.invoke.MethodHandle;
import java.lang.invoke.MethodHandles;
import java.lang.invoke.MethodType;

public class Test {

    public static void target(int i) { }

    public static void main(String[] args) throws Exception {

        MethodHandles.Lookup l = MethodHandles.lookup();

        MethodType t = MethodType.methodType(void.class, int.class);

        MethodHandle mh = l.findStatic(Test.class, "target", t);

        long current = System.currentTimeMillis();
        for (int i = 1; i <= 2_000_000_000; i++) {

            if (i % 100_000_000 == 0) {

                long temp = System.currentTimeMillis();

                System.out.println(temp - current);

                current = temp;

            }

            mh.invokeExact(128);

        }

    }

}
```

实际上，它与使用 Lambda 表达式或者方法引用的差别在于，即时编译器无法将该方法句柄识别为常量，从而无法进行内联。那么如果将它变成常量行不行呢？

一种方法便是将其赋值给 `final` 的静态变量，如下面的 v9 版本所示：

```
// v9 版本
```

```
import java.lang.invoke.MethodHandle;
import java.lang.invoke.MethodHandles;
import java.lang.invoke.MethodType;

public class Test {

    public static void target(int i) { }

    static final MethodHandle mh;

    static {
        try {
            MethodHandles.Lookup l = MethodHandles.lookup();
            MethodType t = MethodType.methodType(void.class, int.class);
            mh = l.findStatic(Test.class, "target", t);
        } catch (Throwable e) {
            throw new RuntimeException(e);
        }
    }

    public static void main(String[] args) throws Throwable {
        long current = System.currentTimeMillis();
        for (int i = 1; i <= 2_000_000_000; i++) {
            if (i % 100_000_000 == 0) {
                long temp = System.currentTimeMillis();
                System.out.println(temp - current);
                current = temp;
            }

            mh.invokeExact(128);
        }
    }
}
```

这个版本测得的数据和直接调用的性能数据一致。也就是说，即时编译器能够将该方法句柄完全内联进来，成为空操作。

今天的实践环节，我们来继续探索方法句柄的性能。运行下面的 v10 版本以及 v11 版本，比较它们的性能并思考为什么。

```
// v10 版本

import java.lang.invoke.*;

public class Test {

    public static void target(int i) {

    }

    public static class MyCallSite {

        public final MethodHandle mh;

        public MyCallSite() {
            mh = findTarget();
        }

        private static MethodHandle findTarget() {
            try {
                MethodHandles.Lookup l = MethodHandles.lookup();
                MethodType t = MethodType.methodType(void.class, int.class);
                return l.findStatic(Test.class, "target", t);
            } catch (Throwable e) {
                throw new RuntimeException(e);
            }
        }
    }

    private static final MyCallSite myCallSite = new MyCallSite();

    public static void main(String[] args) throws Throwable {
        long current = System.currentTimeMillis();
        for (int i = 1; i <= 2_000_000_000; i++) {
            if (i % 100_000_000 == 0) {
                long temp = System.currentTimeMillis();
                System.out.println(temp - current);
                current = temp;
            }
        }
    }
}
```

```

        myCallSite.mh.invokeExact(128);
    }
}

// v11 版本
import java.lang.invoke.*;

public class Test {
    public static void target(int i) {

    }

    public static class MyCallSite extends ConstantCallSite {

        public MyCallSite() {
            super(findTarget());
        }

        private static MethodHandle findTarget() {
            try {
                MethodHandles.Lookup l = MethodHandles.lookup();
                MethodType t = MethodType.methodType(void.class, int.class);
                return l.findStatic(Test.class, "target", t);
            } catch (Throwable e) {
                throw new RuntimeException(e);
            }
        }
    }

    public static final MyCallSite myCallSite = new MyCallSite();

    public static void main(String[] args) throws Throwable {
        long current = System.currentTimeMillis();
        for (int i = 1; i <= 2_000_000_000; i++) {
            if (i % 100_000_000 == 0) {
                long temp = System.currentTimeMillis();
            }
        }
    }
}

```



```
        System.out.println(temp - current);

        current = temp;
    }

    myCallSite.getTarget().invokeExact(128);
}
}
```

感谢你的收听，我们下次再见。

[1] <http://openjdk.java.net/jeps/303>



版权归极客邦科技所有，未经许可不得转载

精选留言



ext4

2

我知道Java对Lambda有个规定：“The variable used in Lambda should be final or effectively final”，也就是说Lambda表达式捕获的变量必须是final或等同于final的。而文中您又讲到：“对于捕获了变量的Lambda，每次invokedynamic都需要新建适配器类实例，以防止他们发生变化”。JVM之所以这么做，是因为这种final的要求仅限于Java source层面，在bytecode层面是无法保证的。我理解的对吗？

2018-08-11

作者回复

语言里的final，是对于当前方法调用而言的。这是因为它实际上就传了个值进去。比如说你定义了

int a, 然后传入 i -> i + a里, 那么之后你对a的修改lambda是看不到的。

适配器针对的是多次不同调用, 比如说每次调用你定义的Pnal int a都不一样。

2018-08-15



code-artist

1

一直没理解“逃逸分析”啥意思?

2018-08-14

作者回复

逃逸分析是指通过数据流分析, 判断一个对象会不会被传递到当前编译的方法之外。比如说你调用了一个方法, 将一个新建的对象作为参数传递出去, 如果这个方法没有被内联, 则说明该新建对象会逃逸。

逃逸分析是一项比较重要的优化, 我后面会详细讲。

2018-08-15



karl

1

看了两遍 勉强有个概念了
还是基础不够 看不懂啊

2018-08-14

作者回复

invokedynamic涉及到的东西很多, 底层实现也在不断改进。看懂个大概就好啦

2018-08-15



小橙橙

0

其实有个地方一直没有想透, 为什么要学习字节码, 学习字节码对我们日常开发有什么作用吗, 老师能否给指点迷津一下?

2018-08-19



Scott

0

老师你好, 我有两个问题, 1是我看了几个有invokedynmaic指令的文件, 都是invokedynamic #31, 0这种形式, 似乎后面这个0没有什么作用, 网上invokedynamic的解说也大多过时, 我使用的是1.8 .0_181版本。2. v10版本和v11版本性能的差距我猜想是v10版本不能正确的内联方法吧? 虽然mh是Pnal的, 但是字节码层面已经丢失这个信息了。

2018-08-19



王贺

0

单态内联缓存的实现代码段, bootstrap方法的实现有问题, 没有return一个CallSite类型返回值。另外, 这篇有点难度了, 看了三遍, 勉强理解。

2018-08-11

作者回复

多谢指出!

2018-08-13



小江

老师邮箱可以提供一下吗，咨询一个问题，经过btrace增强后class文件错误问题

2018-08-10

0

讲堂 深入拆解 Java 虚拟机 文章详情

10 | Java对象的内存布局

2018-08-13 郑雨迪



10 | Java对象的内存布局

朗读人：郑雨迪 11'19" | 5.19M

在 Java 程序中，我们拥有多种新建对象的方式。除了最为常见的 new 语句之外，我们还可以通过反射机制、Object.clone 方法、反序列化以及 Unsafe.allocateInstance 方法来新建对象。

其中，Object.clone 方法和反序列化通过直接复制已有的数据，来初始化新建对象的实例字段。Unsafe.allocateInstance 方法则没有初始化实例字段，而 new 语句和反射机制，则是通过调用构造器来初始化实例字段。

以 new 语句为例，它编译而成的字节码将包含用来请求内存的 new 指令，以及用来调用构造器的 invokespecial 指令。

```
// Foo foo = new Foo(); 编译而成的字节码
0 new Foo
3 dup
4 invokespecial Foo()
```

提到构造器，就不得不提到 Java 对构造器的诸多约束。首先，如果一个类没有定义任何构造器的话，Java 编译器会自动添加一个无参数的构造器。

```
// Foo 类构造器会调用其父类 Object 的构造器
public Foo();
    0 aload_0 [this]
    1 invokespecial java.lang.Object() [8]
    4 return
```

然后，子类的构造器需要调用父类的构造器。如果父类存在无参数构造器的话，该调用可以是隐式的，也就是说 Java 编译器会自动添加对父类构造器的调用。但是，如果父类没有无参数构造器，那么子类的构造器则需要显式地调用父类带参数的构造器。

显式调用又可分为两种，一是直接使用“super”关键字调用父类构造器，二是使用“this”关键字调用同一个类中的其他构造器。无论是直接的显式调用，还是间接的显式调用，都需要作为构造器的第一条语句，以便优先初始化继承而来的父类字段。（不过这可以通过调用其他生成参数的方法，或者字节码注入来绕开。）

总而言之，当我们调用一个构造器时，它将优先调用父类的构造器，直至 Object 类。这些构造器的调用者皆为同一对象，也就是通过 new 指令新建而来的对象。

你应该已经发现了其中的玄机：通过 new 指令新建出来的对象，它的内存其实涵盖了所有父类中的实例字段。也就是说，虽然子类无法访问父类的私有实例字段，或者子类的实例字段隐藏了父类的同名实例字段，但是子类的实例还是会为这些父类实例字段分配内存的。

这些字段在内存中的具体分布是怎么样的呢？今天我们就来看看对象的内存布局。

压缩指针

在 Java 虚拟机中，每个 Java 对象都有一个对象头（object header），这个由标记字段和类型指针所构成。其中，标记字段用以存储 Java 虚拟机有关该对象的运行数据，如哈希码、GC 信息以及锁信息，而类型指针则指向该对象的类。

在 64 位的 Java 虚拟机中，对象头的标记字段占 64 位，而类型指针又占了 64 位。也就是说，每一个 Java 对象在内存中的额外开销就是 16 个字节。以 Integer 类为例，它仅有一个 int 类型的私有字段，占 4 个字节。因此，每一个 Integer 对象的额外内存开销至少是 400%。这也是为什么 Java 要引入基本类型的原因之一。

为了尽量较少对象的内存使用量，64 位 Java 虚拟机引入了压缩指针 [1] 的概念（对应虚拟机选项 `-XX:+UseCompressedOops`，默认开启），将堆中原本 64 位的 Java 对象指针压缩成 32 位的。

这样一来，对象头中的类型指针也会被压缩成 32 位，使得对象头的大小从 16 字节降至 12 字节。当然，压缩指针不仅可以作用于对象头的类型指针，还可以作用于引用类型的字段，以及引用类型数组。

那么压缩指针是什么原理呢？

打个比方，路上停着的全是房车，而且每辆房车恰好占据两个停车位。现在，我们按照顺序给它们编号。也就是说，停在 0 号和 1 号停车位上的叫 0 号车，停在 2 号和 3 号停车位上的叫 1 号车，依次类推。

原本的内存寻址用的是车位号。比如说我有一个值为 6 的指针，代表第 6 个车位，那么沿着这个指针可以找到 3 号车。现在我们规定指针里存的值是车号，比如 3 指代 3 号车。当需要查找 3 号车时，我便可以将该指针的值乘以 2，再沿着 6 号车位找到 3 号车。

这样一来，32 位压缩指针最多可以标记 2 的 32 次方辆车，对应着 2 的 33 次方个车位。当然，房车也有大小之分。大房车占据的车位可能是三个甚至是更多。不过这并不会影响我们的寻址算法：我们只需跳过部分车号，便可以保持原本车号 *2 的寻址系统。

上述模型有一个前提，你应该已经想到了，就是每辆车都从偶数号车位停起。这个概念我们称之为内存对齐（对应虚拟机选项 `-XX:ObjectAlignmentInBytes`，默认值为 8）。

默认情况下，Java 虚拟机堆中对象的起始地址需要对齐至 8 的倍数。如果一个对象用不到 8N 个字节，那么空白的那部分空间就浪费掉了。这些浪费掉的空间我们称之为对象间的填充（padding）。

在默认情况下，Java 虚拟机中的 32 位压缩指针可以寻址到 2 的 35 次方个字节，也就是 32GB 的地址空间（超过 32GB 则会关闭压缩指针）。

在对压缩指针解引用时，我们需要将其左移 3 位，再加上一个固定偏移量，便可以得到能够寻址 32GB 地址空间的伪 64 位指针了。

此外，我们可以通过配置刚刚提到的内存对齐选项（`-XX:ObjectAlignmentInBytes`）来进一步提升寻址范围。但是，这同时也可能增加对象间填充，导致压缩指针没有达到原本节省空间的效果。

举例来说，如果规定每辆车都需要从偶数车位号停起，那么对于占据两个车位的小房车来说刚刚好，而对于需要三个车位的大房车来说，也仅是浪费一个车位。

但是如果规定需要从 4 的倍数号车位停起，那么小房车则会浪费两个车位，而大房车至多可能浪费三个车位。

当然，就算是关闭了压缩指针，Java 虚拟机还是会进行内存对齐。此外，内存对齐不仅存在于对象与对象之间，也存在于对象中的字段之间。比如说，Java 虚拟机要求 long 字段、double 字段，以及非压缩指针状态下的引用字段地址为 8 的倍数。

字段内存对齐的其中一个原因，是让字段只出现在同一 CPU 的缓存行中。如果字段不是对齐的，那么就有可能出现跨缓存行的字段。也就是说，该字段的读取可能需要替换两个缓存行，而该字段的存储也会同时污染两个缓存行。这两种情况对程序的执行效率而言都是不利的。

下面我来介绍一下对象内存布局另一个有趣的特性：字段重排列。

字段重排列

字段重排列，顾名思义，就是 Java 虚拟机重新分配字段的先后顺序，以达到内存对齐的目的。Java 虚拟机中有三种排列方法（对应 Java 虚拟机选项 -XX:FieldsAllocationStyle，默认值为 1），但都会遵循如下两个规则。

其一，如果一个字段占据 C 个字节，那么该字段的偏移量需要对齐至 NC。这里偏移量指的是字段地址与对象的起始地址差值。

以 long 类为例，它仅有一个 long 类型的实例字段。在使用了压缩指针的 64 位虚拟机中，尽管对象头的大小为 12 个字节，该 long 类型字段的偏移量也只能是 16，而中间空着的 4 个字节便会被浪费掉。

其二，子类所继承字段的偏移量，需要与父类对应字段的偏移量保持一致。

在具体实现中，Java 虚拟机还会对齐子类字段的起始位置。对于使用了压缩指针的 64 位虚拟机，子类第一个字段需要对齐至 4N；而对于关闭了压缩指针的 64 位虚拟机，子类第一个字段则需要对齐至 8N。

```
class A {
    long l;
    int i;
}

class B extends A {
    long l;
    int i;
}
```

我在文中贴了一段代码，里边定义了两个类 A 和 B，其中 B 继承 A。A 和 B 各自定义了一个 long

类型的实例字段和一个 int 类型的实例字段。下面我分别打印了 B 类在启用压缩指针和未启用压缩指针时，各个字段的偏移量。

```
# 启用压缩指针时，B 类的字段分布

B object internals:
  OFFSET  SIZE   TYPE DESCRIPTION     VALUE
    0     4       (object header)
    4     4       (object header)
    8     4       (object header)
   12     4    int A.i             0
   16     8   long A.l             0
   24     8   long B.l             0
   32     4    int B.i             0
   36     4       (loss due to the next object alignment)
```

当启用压缩指针时，可以看到 Java 虚拟机将 A 类的 int 字段放置于 long 字段之前，以填充因为 long 字段对齐造成的 4 字节缺口。由于对象整体大小需要对齐至 8N，因此对象的最后会有 4 字节的空白填充。

```
# 关闭压缩指针时，B 类的字段分布

B object internals:
  OFFSET  SIZE   TYPE DESCRIPTION     VALUE
    0     4       (object header)
    4     4       (object header)
    8     4       (object header)
   12     4       (object header)
   16     8   long A.l
   24     4    int A.i
   28     4       (alignment/padding gap)
   32     8   long B.l
   40     4    int B.i
   44     4       (loss due to the next object alignment)
```

当关闭压缩指针时，B 类字段的起始位置需对齐至 8N。这么一来，B 类字段的前后各有 4 字节的空白。那么我们可不可以将 B 类的 int 字段移至前面的空白中，从而节省这 8 字节呢？

我认为是可以的，并且我修改过后的 Java 虚拟机也没有跑崩。由于 HotSpot 中的这块代码年久失

修，公司的同事也已经记不得是什么原因了，那么姑且先认为是一些历史遗留问题吧。

Java 8 还引入了一个新的注释 `@Contended`，用来解决对象字段之间的虚共享（false sharing）问题 [2]。这个注释也会影响到字段的排列。

虚共享是怎么回事呢？假设两个线程分别访问同一对象中不同的 `volatile` 字段，逻辑上它们并没有共享内容，因此不需要同步。

然而，如果这两个字段恰好在同一个缓存行中，那么对这些字段的写操作会导致缓存行的写回，也就造成了实质上的共享。（`volatile` 字段和缓存行的故事我会在之后的篇章中详细介绍。）

Java 虚拟机会让不同的 `@Contended` 字段处于独立的缓存行中，因此你会看到大量的空间被浪费掉。具体的分布算法属于实现细节，随着 Java 版本的变动也比较大，因此这里就不做阐述了。

如果你感兴趣，可以利用实践环节的工具，来查阅 `Contended` 字段的内存布局。注意使用虚拟机选项 `-XX:-RestrictContended`。如果你在 Java 9 以上版本试验的话，在使用 `javac` 编译时需要添加 `--add-exports java.base/jdk.internal.vm.annotation=ALL-UNNAMED`

总结和实践

今天我介绍了 Java 虚拟机构造对象的方式，所构造对象的大小，以及对象的内存布局。

常见的 `new` 语句会被编译为 `new` 指令，以及对构造器的调用。每个类的构造器皆会直接或者间接调用父类的构造器，并且在同一个实例中初始化相应的字段。

Java 虚拟机引入了压缩指针的概念，将原本的 64 位指针压缩成 32 位。压缩指针要求 Java 虚拟机堆中对象的起始地址要对齐至 8 的倍数。Java 虚拟机还会对每个类的字段进行重排列，使得字段也能够内存对齐。

今天的实践环节比较简单，你可以使用我在工具篇中介绍过的 JOL 工具，来打印你工程中的类的字段分布情况。

```
curl -L -O http://central.maven.org/maven2/org/openjdk/jol/jol-cli/0.9/jol-cli-0.9-full.jar
java -cp jol-cli-0.9-full.jar org.openjdk.jol.Main internals java.lang.String
```

[1] <https://wiki.openjdk.java.net/display/HotSpot/CompressedOops>

[2] <http://openjdk.java.net/jeps/142>



版权归极客邦科技所有，未经许可不得转载

精选留言



godtrue

小结

1:Java中创建对象的方式

1-1:new -通过调用构造器来初始化实例字段

1-2:反射-通过调用构造器来初始化实例字段

1-3:Object.clone-通过直接复制已有的数据，来初始化新建对象的实例字段

1-4:反序列化-通过直接复制已有的数据，来初始化新建对象的实例字段

1-5:Unsafe.allocateInstance-没有初始化对象的实例字段

2:Java对象的空间占用

2-1:通过new创建的对象，涵盖了它所有父类中的对象实例的字段

2-2:对象头，由标记字段和类型指针构成

2-3:标记字段，用于存储Java虚拟机有关该对象的运行数据，比如：哈希码、GC信息、锁信息等

2-4:类型指针，用于指向该对象的类

2

2-5:此对象的实例字段对应的内存空间

3:压缩指针

JVM的内存空间有限且昂贵，所以，能缩减的就缩减，通过一定的算法改进压缩类型指针的空间后仍可以寻址到对象的实例对应的类，所以，就采用了

4:字段重排

意思是JVM会重新分配字段的位置，和我们Java源码中属性声明的位置存在差异，猜想Java编译器编译后的字节码是没有改变源码中字段声明的位置的，这样做是为了更好的实现内存对齐，内存对齐本质上会浪费一定的内存空间，不过可以减少内存行的读取次数，通过一消一涨的比对发现这样对于JVM的性能有一定的提高，所以，也就使用了这种方式，浪费点空间能提高性能也是值得的

疑问

1:为什么一个子类即使无法访问父类的私有实例字段，或者子类实例字段隐藏了父类的同名实例字段，子类的实例还是会为这些父类实例字段分配内存呢？

另外，如果采用指针指向的方式定位父类实例的内容是否能更节省内存空间？

2:五种创建对象的方式，通过new指令新建出来的对象，他的内存其实涵盖了所有父类中的实例字段，其他的方式是怎样的哪？

2018-08-13



三木子

2

有一个小白问题，new一个对象(继承一个类)会调用父类构造器，这个可以理解，因为对象可能调用父类方法。那么为什么new对象会调用到object呢？这有什么用意吗？

2018-08-13



倔强

1

也就是说默认情况下，小于32G的堆内存中的对象引用为4个字节，一旦堆内存大于32G，对象引用为8个字节

2018-08-14



amourling

1

作者大大辛苦了，货很干，搭配《深入理解java虚拟机》会很香

2018-08-13



小白猪

0

引用类型是4个字节吗，还是压缩开启后是4个字节

2018-08-15



贾智文

0

默认情况下32位可以寻址2的35次，应该是因为地址是32位乘以2的三次（默认对其为8），那么如果不采用压缩指针，能够寻址的范围应该是2的64次对吧。然后之前模糊的地方就是觉得两个寻址

范围并不一致，是不是可以这么理解并没有通过压缩指针让两个寻址范围一致，而是通过压缩指针放大了32位的寻址空间使它够用了

2018-08-15

作者回复

对的

2018-08-15



贾智文

0

有一点想不明白，既然内存对齐是八位而不是举例的两位为什么空间只是从64位变成32而不是从64变成8

2018-08-15

作者回复

不是很理解你的问题。

对象间需要内存对齐至8字节。64位和32位对应8字节和4字节。

2018-08-15



大能猫

0

最近研究String时遇到一个跟Java内存相关的问题：常量池里到底有没有存放对象？常量池主要存放两大类常量：字面量（Literal）和符号引用（Symbolic Reference）；如果常量池里有一个“hello”的字面量，这个字面量算是一个对象吗？如果不算对象，那么它所指向的对象又存放在哪里呢

2018-08-14

作者回复

String literal指向的对象存放在JVM的String pool里。

2018-08-15



熊猫酒仙

0

接触过C/C++的内存字节对齐，就比较好理解本章内容了。希望老师后面讲讲java内存在并发上的相关机制，譬如搞不懂的内存障是怎么实现的！

2018-08-14

作者回复

底层的内存屏障，比如说mfence，lock指令，是用来防止指令重排布的。Java里的内存屏障除了生成上述底层指令外，还会限制即时编译器对内存访问的重排序。之后我会讲Java内存模型。

2018-08-15



xianhai

0

压缩指针指向的类对象是存在heap中，还是heap外？

2018-08-14

讲堂 深入拆解 Java 虚拟机 文章详情

11 | 垃圾回收（上）

2018-08-15 郑雨迪



11 | 垃圾回收（上）

朗读人：郑雨迪 12'11" | 5.59M

你应该听说过这么一句话：免费的其实是最贵的。

Java 虚拟机的自动内存管理，将原本需要由开发人员手动回收的内存，交给垃圾回收器来自动回收。不过既然是自动机制，肯定没法做到像手动回收那般精准高效 [1]，而且还会带来不少与垃圾回收实现相关的问题。

接下来的两篇，我们会深入探索 Java 虚拟机中的垃圾回收器。今天这一篇，我们来回顾一下垃圾回收的基础知识。

引用计数法与可达性分析

垃圾回收，顾名思义，便是将已经分配出去的，但却不再使用的内存回收回来，以便能够再次分配。在 Java 虚拟机的语境下，垃圾指的是死亡的对象所占据的堆空间。这里便涉及了一个关键的问题：如何辨别一个对象是存是亡？

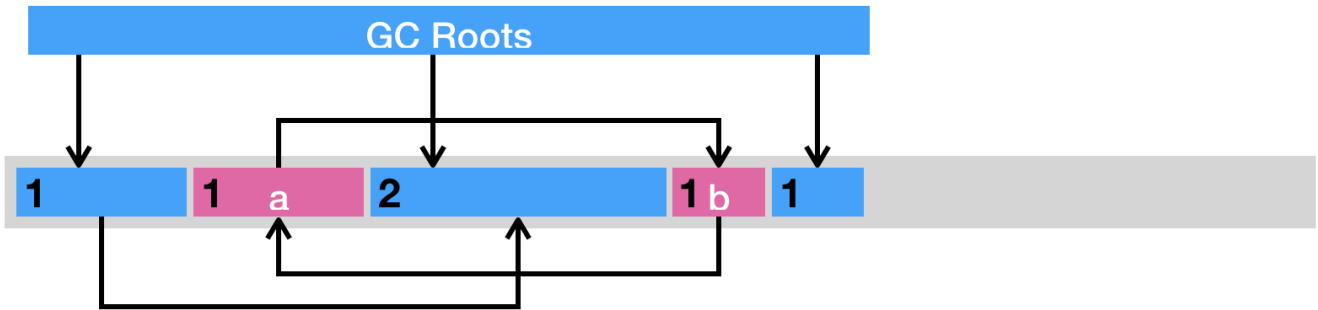
我们先来讲一种古老的辨别方法：引用计数法（reference counting）。它的做法是为每个对象添

加一个引用计数器，用来统计指向该对象的引用个数。一旦某个对象的引用计数器为 0，则说明该对象已经死亡，便可以被回收了。

它的具体实现是这样子的：如果有一个引用，被赋值为某一对象，那么将该对象的引用计数器 +1。如果一个指向某一对象的引用，被赋值为其他值，那么将该对象的引用计数器 -1。也就是说，我们需要截获所有的引用更新操作，并且相应地增减目标对象的引用计数器。

除了需要额外的空间来存储计数器，以及繁琐的更新操作，引用计数法还有一个重大的漏洞，那便是无法处理循环引用对象。

举个例子，假设对象 a 与 b 相互引用，除此之外没有其他引用指向 a 或者 b。在这种情况下，a 和 b 实际上已经死了，但由于它们的引用计数器皆不为 0，在引用计数法的心中，这两个对象还活着。因此，这些循环引用对象所占据的空间将不可回收，从而造成了内存泄露。



目前 Java 虚拟机的主流垃圾回收器采取的是可达性分析算法。这个算法的实质在于将一系列 GC Roots 作为初始的存活对象合集（live set），然后从该合集出发，探索所有能够被该集合引用到的对象，并将其加入到该集合中，这个过程我们也称之为标记（mark）。最终，未被探索到的对象便是死亡的，是可以回收的。

那么什么是 GC Roots 呢？我们可以暂时理解为由堆外指向堆内的引用，一般而言，GC Roots 包括（但不限于）如下几种：

- 1. Java 方法栈帧中的局部变量；
- 2. 已加载类的静态变量；
- 3. JNI handles；
- 4. 已启动且未停止的 Java 线程。

可达性分析可以解决引用计数法所不能解决的循环引用问题。举例来说，即便对象 a 和 b 相互引用，只要从 GC Roots 出发无法到达 a 或者 b，那么可达性分析便不会将它们加入存活对象合集之中。

虽然可达性分析的算法本身很简明，但是在实践中还是有不少其他问题需要解决的。

比如说，在多线程环境下，其他线程可能会更新已经访问过的对象中的引用，从而造成误报（将引

用设置为 null) 或者漏报 (将引用设置为未被访问过的对象)。

误报并没有什么伤害, Java 虚拟机至多损失了部分垃圾回收的机会。漏报则比较麻烦, 因为垃圾回收器可能回收事实上仍被引用的对象内存。一旦从原引用访问已经被回收了的对象, 则很有可能会直接导致 Java 虚拟机崩溃。

Stop-the-world 以及安全点

怎么解决这个问题呢? 在 Java 虚拟机里, 传统的垃圾回收算法采用的是一种简单粗暴的方式, 那便是 Stop-the-world, 停止其他非垃圾回收线程的工作, 直到完成垃圾回收。这也就造成了垃圾回收所谓的暂停时间 (GC pause)。

Java 虚拟机中的 Stop-the-world 是通过安全点 (safepoint) 机制来实现的。当 Java 虚拟机收到 Stop-the-world 请求, 它便会等待所有的线程都到达安全点, 才允许请求 Stop-the-world 的线程进行独占的工作。

这篇博客 [2] 还提到了一种比较另类的解释: 安全词。一旦垃圾回收线程喊出了安全词, 其他非垃圾回收线程便会一一停下。

当然, 安全点的初始目的并不是让其他线程停下, 而是找到一个稳定的执行状态。在这个执行状态下, Java 虚拟机的堆栈不会发生变化。这么一来, 垃圾回收器便能够“安全”地执行可达性分析。

举个例子, 当 Java 程序通过 JNI 执行本地代码时, 如果这段代码不访问 Java 对象、调用 Java 方法或者返回至原 Java 方法, 那么 Java 虚拟机的堆栈不会发生改变, 也就代表着这段本地代码可以作为同一个安全点。

只要不离开这个安全点, Java 虚拟机便能够在垃圾回收的同时, 继续运行这段本地代码。

由于本地代码需要通过 JNI 的 API 来完成上述三个操作, 因此 Java 虚拟机仅需在 API 的入口处进行安全点检测 (safepoint poll), 测试是否有其他线程请求停留在安全点里, 便可以在必要的时候挂起当前线程。

除了执行 JNI 本地代码外, Java 线程还有其他几种状态: 解释执行字节码、执行即时编译器生成的机器码和线程阻塞。阻塞的线程由于处于 Java 虚拟机线程调度器的掌控之下, 因此属于安全点。

其他几种状态则是运行状态, 需要虚拟机保证在可预见的时间内进入安全点。否则, 垃圾回收线程可能长期处于等待所有线程进入安全点的状态, 从而变相地提高了垃圾回收的暂停时间。

对于解释执行来说, 字节码与字节码之间皆可作为安全点。Java 虚拟机采取的做法是, 当有安全点请求时, 执行一条字节码便进行一次安全点检测。

执行即时编译器生成的机器码则比较复杂。由于这些代码直接运行在底层硬件之上, 不受 Java 虚拟机掌控, 因此在生成机器码时, 即时编译器需要插入安全点检测, 以避免机器码长时间没有安全点

检测的情况。HotSpot 虚拟机的做法便是在生成代码的方法出口以及非计数循环的循环回边（back-edge）处插入安全点检测。

那么为什么不在每一条机器码或者每一个机器码基本块处插入安全点检测呢？原因主要有两个。

第一，安全点检测本身也有一定的开销。不过 HotSpot 虚拟机已经将机器码中安全点检测简化为一个内存访问操作。在有安全点请求的情况下，Java 虚拟机会将安全点检测访问的内存所在的页设置为不可读，并且定义一个 segfault 处理器，来截获因访问该不可读内存而触发 segfault 的线程，并将它们挂起。

第二，即时编译器生成的机器码打乱了原本栈帧上的对象分布状况。在进入安全点时，机器码还需提供一些额外的信息，来表明哪些寄存器，或者当前栈帧上的哪些内存空间存放着指向对象的引用，以便垃圾回收器能够枚举 GC Roots。

由于这些信息需要不少空间来存储，因此即时编译器会尽量避免过多的安全点检测。

不过，不同的即时编译器插入安全点检测的位置也可能不同。以 Graal 为例，除了上述位置外，它还会在计数循环的循环回边处插入安全点检测。其他的虚拟机也可能选取方法入口而非方法出口来插入安全点检测。

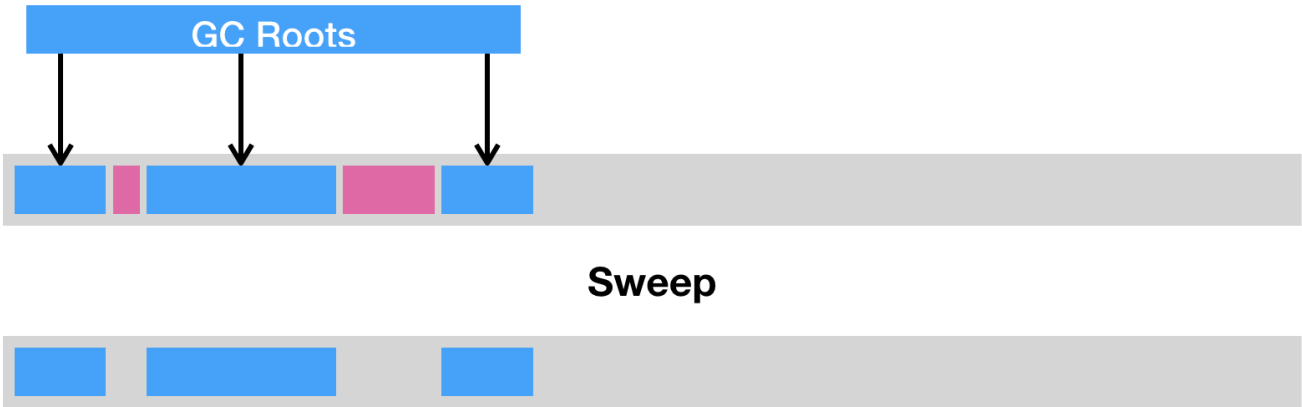
不管如何，其目的都是在可接受的性能开销以及内存开销之内，避免机器码长时间不进入安全点的情况，间接地减少垃圾回收的暂停时间。

除了垃圾回收之外，Java 虚拟机其他一些对堆栈内容的一致性有要求的操作也会用到安全点这一机制。我会在涉及的时候再进行具体的讲解。

垃圾回收的三种方式

当标记完所有的存活对象时，我们便可以进行死亡对象的回收工作了。主流的基础回收方式可分为三种。

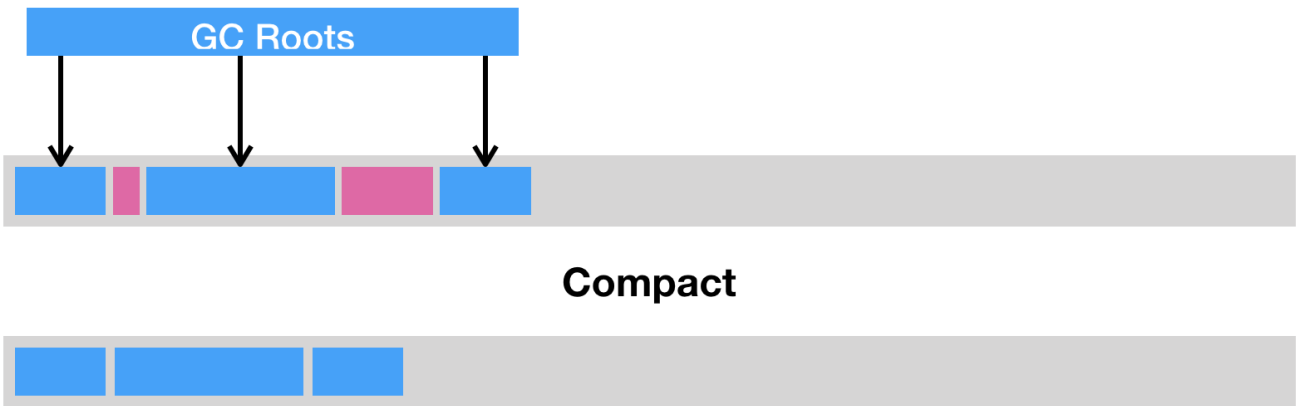
第一种是清除（sweep），即把死亡对象所占据的内存标记为空闲内存，并记录在一个空闲列表（free list）之中。当需要新建对象时，内存管理模块便会从该空闲列表中寻找空闲内存，并划分给新建的对象。



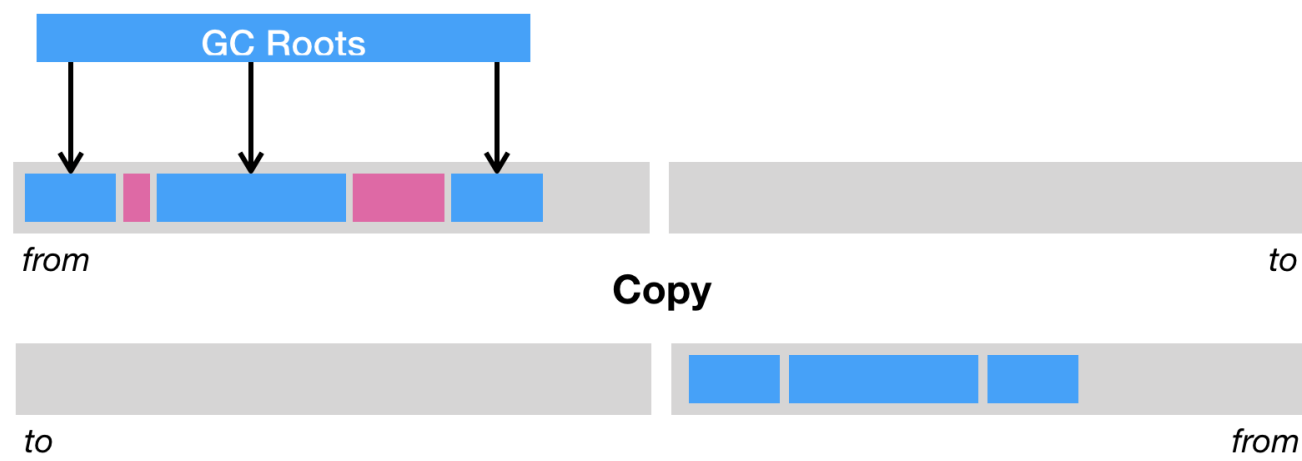
清除这种回收方式的原理及其简单，但是有两个缺点。一是会造成内存碎片。由于 Java 虚拟机的堆中对象必须是连续分布的，因此可能出现总空闲内存足够，但是无法分配的极端情况。

另一个则是分配效率较低。如果是一块连续的内存空间，那么我们可以通过指针加法（pointer bumping）来做分配。而对于空闲列表，Java 虚拟机则需要逐个访问列表中的项，来查找能够放入新建对象的空闲内存。

第二种是压缩（compact），即把存活的对象聚集到内存区域的起始位置，从而留下一段连续的内存空间。这种做法能够解决内存碎片化的问题，但代价是压缩算法的性能开销。



第三种则是复制（copy），即把内存区域分为两等分，分别用两个指针 from 和 to 来维护，并且只是用 from 指针指向的内存区域来分配内存。当发生垃圾回收时，便把存活的对象复制到 to 指针指向的内存区域中，并且交换 from 指针和 to 指针的内容。复制这种回收方式同样能够解决内存碎片化的问题，但是它的缺点也极其明显，即堆空间的使用效率极其低下。



当然，现代的垃圾回收器往往会综合上述几种回收方式，综合它们优点的同时规避它们的缺点。在下一篇中我们会详细介绍 Java 虚拟机中垃圾回收算法的具体实现。

总结与实践

今天我介绍了垃圾回收的一些基础知识。

Java 虚拟机中的垃圾回收器采用可达性分析来探索所有存活的对象。它从一系列 GC Roots 出发，边标记边探索所有被引用的对象。

为了防止在标记过程中堆栈的状态发生改变，Java 虚拟机采取安全点机制来实现 Stop-the-world 操作，暂停其他非垃圾回收线程。

回收死亡对象的内存共有三种方式，分别为：会造成内存碎片的清除、性能开销较大的压缩、以及堆使用效率较低的复制。

今天的实践环节，你可以体验一下无安全点检测的计数循环带来的长暂停。你可以分别测单独跑 foo 方法或者 bar 方法的时间，然后与合起来跑的时间比较一下。

```
// time java SafepointTest
/ 你还可以使用如下几个选项
// -XX:+PrintGC
// -XX:+PrintGCApplicationStoppedTime
// -XX:+PrintSafepointStatistics
// -XX:+UseCountedLoopSafepoints

public class SafepointTest {
    static double sum = 0;

    public static void foo() {
```

```
for (int i = 0; i < 0x77777777; i++) {  
    sum += Math.sqrt(i);  
}  
}  
  
public static void bar() {  
    for (int i = 0; i < 50_000_000; i++) {  
        new Object().hashCode();  
    }  
}  
  
public static void main(String[] args) {  
    new Thread(SafepointTest::foo).start();  
    new Thread(SafepointTest::bar).start();  
}  
}
```

[1] <https://media.giphy.com/media/EZ8QO0myvsSk/giphy.gif>

[2] <http://psy-lob-saw.blogspot.com/2015/12/safepoints.html>



版权归极客邦科技所有，未经许可不得转载

精选留言



非常感谢，此篇可用通俗易懂来形容，其他同学问的问题也很棒！

小结：

1:垃圾回收-核心工作就是回收垃圾，哪关键点回来了。什么是垃圾？这个垃圾需要分类嘛？怎么定位垃圾？怎么回收垃圾？回收垃圾的方法都有哪些？他们都有什么优缺点？另外，就是我们为什么要学习垃圾回收？

2:站在JVM的视角来看

垃圾-就是无用对象所占用的堆内存空间

貌似不需要垃圾分类，识别垃圾并回收就行

定位垃圾，是垃圾回收的关键点

晚安💎💎，明天继续写

2018-08-16



茶底

14

老师下一期能讲一下g1算法吗。讲深一点💎💎

2018-08-15



黑崽

1

第二，即时编译器生成的机器码打乱了原本栈帧上的对象分布状况。没明白这个原因。第一个原因中解释，只要去访问一个内存地址就可以知道是不是要暂停了，那我只有判断完暂停以后再去恢复寄存器中状态不就可以了？反正只有一次，这个打乱不打乱有什么区别呢？

2018-08-19



godtrue

1

疑问

1:JVM的stop-the-world机制非常不友好，有哪些解决之道？原理是什么？

2:压测时出现频繁的gc容易理解，但是有时出现毛刺是因为什么呢？

3:fullgc有卡顿，对性能很不利，怎么避免呢？

2018-08-17



godtrue

1

非常感谢，此篇可用通俗易懂来形容，其他同学问的问题也很棒！

小结：

1:垃圾回收-核心工作就是回收垃圾，哪关键点回来了。什么是垃圾？这个垃圾需要分类嘛？怎么定位垃圾？怎么回收垃圾？回收垃圾的方法都有哪些？他们都有什么优缺点？另外，就是我们为什么要学习垃圾回收？

2:站在JVM的视角来看

垃圾-就是无用对象所占用的堆内存空间

垃圾分类-貌似不需要垃圾分类，识别垃圾并回收就行

定位垃圾-是垃圾回收的关键点，无用的对象占用的堆空间即是垃圾，那就需要先定位无用的对象，这里的无用是不再使用的意思，咋判断呢？文中介绍了两种方法，计数法和标记法（祥看原文）核心在于能定位出无用的对象，后出现的方法往往比早出现的更好一点，这里也一样，标记法能解决计数法，解决不了的循环引用不能回收的问题，但是也存在其他的问题，误报和漏报的问题，误报浪费点垃圾回收的机会浪费点空间，漏报在多线程并发工作时可能会死JVM的，所以，比较严重，所以，JVM采用了简单粗暴的stop-the-world的方式来对待，所以，老年代的回收有卡顿的现象

怎么回收垃圾-定位出垃圾，回收就是一个简单的事情了，当然也非常关键，把要回收的堆内存空间标记为可继续使用就行，下次有新对象能在此空间创建就行

回收垃圾的方法-文中介绍了三种，清除、压缩、复制

清除法-简单，但易产生碎片，可能总空间够但分配不了的问题

压缩法-能解决清除法的问题，但是复杂且耗性能

复制法-折衷一些，但是空间利用率低，总之，各有千秋

为什么要学-这个最容易，因为面试需要、装逼需要、升职加薪需要、人类天生好奇、还有免于被鄙视及可以鄙视其他人

2018-08-17



风动静泉

1

有个内存泄露相关的问题想请教一下老师。

基本描述:

一个单线程的程序中，在其run方法中有局部变量（map list等类型），也使用了该线程类ThreadDemo的全局变量（map），线程中会去执行ftp任务（同步）。

问题:

如果ftp任务由于某种原因阻塞了，调用ftp任务的线程ThreadDemo中的变量（局部变量和全局变量）会变成不可达状态吗？还是说此时线程也阻塞，会导致内存泄露？

2018-08-16



彩色的沙漠

1

@正是那朵玫瑰老师有几个不明白的地方，误报和漏报不太明白：

- 1、假设A引用开始指向A1对象：A----->A1,按老师说的误报就是将引用A指向null：A----->null，那么此时A1对象不是没有引用了，不就可以垃圾回收了么，为什么会错过垃圾回收的机会呢？
- 2、漏报，是将A引用指向一个未被访问的对象假设对象为B：A----->B,此时A引用原来指向的对象应该没有引用了吧，为什么会垃圾回收器可能会回收事实上仍被引用的对象呢？

2018-08-15

作者回复

这里指的是，GC已经标记完成，然后其他线程进行修改的情况(也是并发GC所要解决的问题)。

当GC标记完成，还未开始回收时，你更新了其中一个引用，使之指向null，那么原来指向的对象本可以被回收的。

如果指向一个新的对象，这个对象可没有被标记为不能回收，垃圾回收器就直接给回收掉了

老师我也有和@正是那朵玫瑰一样的问题，看了老师的讲解，还是不太明白。GC标记完成，那GC标记的是引用还是具体的堆空间对象。如果标记的具体的堆空间对象，并不会造成GC并发问题，误报和漏报，改变的是引用关系。请老师解答，谢谢！

2018-08-15



王贺

猜一下，作业里面的应该是Math.log(10)

2018-08-15

作者回复

厉害！这都看得出来！是Math.log10(i)，不过这个时间太长了，我新版本的代码是sqrt。然后不知怎么给回滚到这个log10，而且还是乱的

2018-08-15



no13bus

昨天看书正好看到这章节，真的不错

2018-08-15



明天更美好

总算可以听的懂了

2018-08-15



茶底

老师下一期能讲一下g1算法吗。讲深一点💎💎

2018-08-15



code-artist

“最终，未被探索到的对象便是死亡的，是可以回收的。”，已经通过GC_Roots探索到所有的存活对象集。是不是整个堆空间减去存活对象集占用的空间就是空闲堆空间了？这样就不用具体去追究死亡对象的地址空间了？

2018-08-17

1

1

1

1

0

0



WolvesLeader

很是不明白，我的理解有没有stop the word 是和垃圾回收器有关的，看完之后怎么觉得您的意思是，不管什么垃圾回收器都会出现stop the word

2018-08-17



仰望

“举例来说，即便对象 a 和 b 相互引用，只要从 GC Roots 出发无法到达 a 或者 b，”
这个如何判断的？

2018-08-17



魔都浪子

能做一些源码分析吗，比如G1

2018-08-16



维维

请教老师，可达性分析a与b互为引用，这不是死循环了吗，为什么能被回收的，这里不太明白。

2018-08-16



涛哥

老师，特别想了解下G1的原理和机制

2018-08-15



正是那朵玫瑰

感谢老师解答，菜鸟实在还是不太明白[捂脸]，一个对象不可达了，gc才会标记为可以回收，老师所说的：“你更新了其中一个引用”，“你”指的是gc线程还是工作线程，引用指的是哪里的引用，GC都已经标记完成，怎么会有引用被更新？

2018-08-15

作者回复

工作线程。

你已经看出来这里的问题啦。如果工作线程还在继续工作，更新引用，那么就会引发你能想到的这些问题。传统的做法就是接下来讲的Safepoint，禁止工作线程继续更新引用。新的做法是通过一系列算法保证不会出现漏报的情况，从而支持并发回收。

2018-08-15



有铭

代码跑不起来啊，wrMpth.log10是什么

2018-08-15

作者回复

多谢指出，我更新一下

2018-08-15

0

0

0

0

0

0

0



正是那朵玫瑰

老师有几个不明白的地方，误报和漏报不太明白：

- 1、假设A引用开始指向A1对象：A----->A1,按老师说的误报就是将引用A指向null：A----->null，那么此时A1对象不是没有引用了，不就可以垃圾回收了么，为什么会错过垃圾回收的机会呢？
- 2、漏报，是将A引用指向一个未被访问的对象假设对象为B：A----->B,此时A引用原来指向的对象应该没有引用了吧，为什么会垃圾回收器可能会回收事实上仍被引用的对象呢？

2018-08-15

作者回复

这里指的是，GC已经标记完成，然后其他线程进行修改的情况(也是并发GC所要解决的问题)。

当GC标记完成，还未开始回收时，你更新了其中一个引用，使之指向null，那么原来指向的对象本可以被回收的。

如果指向一个新的对象，这个对象可没有被标记为不能回收，垃圾回收器就直接给回收掉了

2018-08-15

讲堂 深入拆解 Java 虚拟机 文章详情

12 | 垃圾回收（下）

2018-08-17 郑雨迪

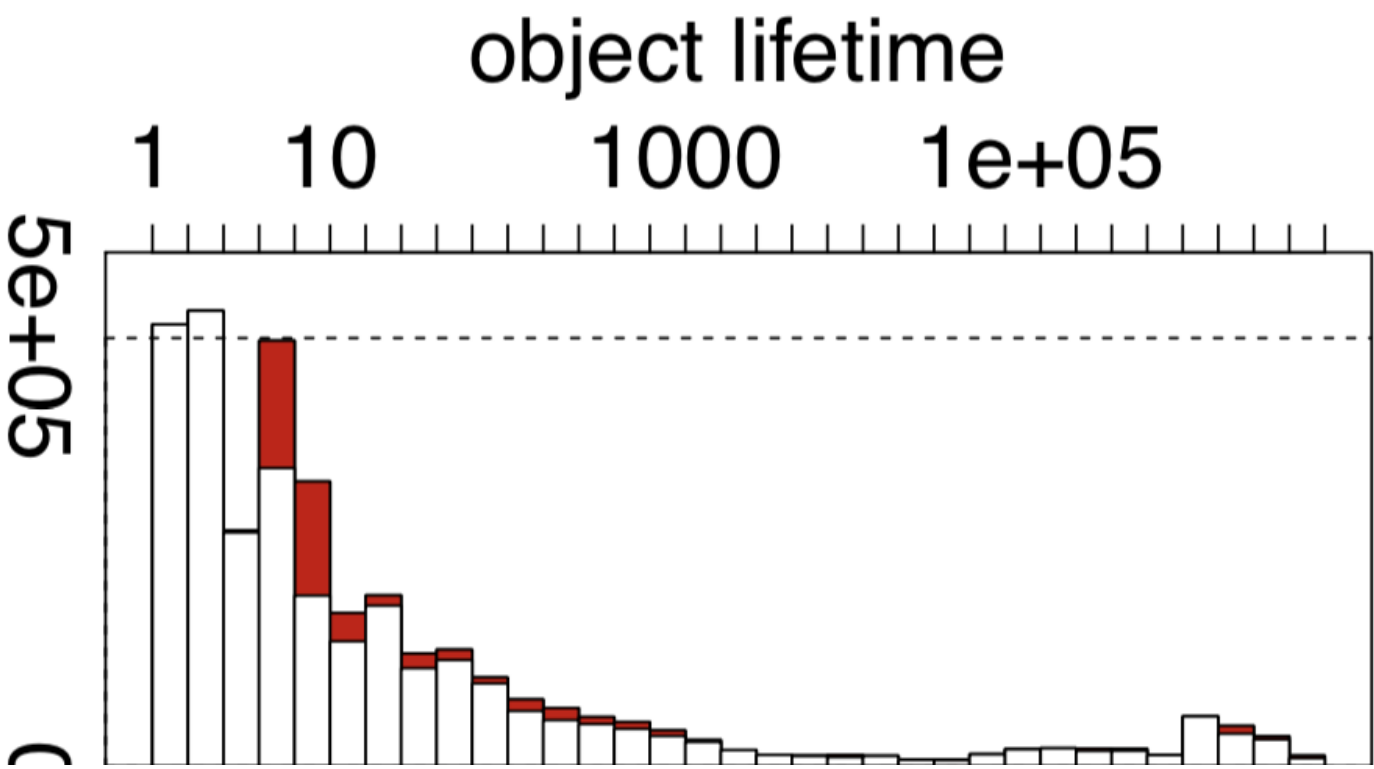


12 | 垃圾回收（下）

朗读人：郑雨迪 10'29" | 4.81M

在读博士的时候，我曾经写过一个统计 Java 对象生命周期的动态分析，并且用它来跑了一些基准测试。

其中一些程序的结果，恰好验证了许多研究人员的假设，即大部分的 Java 对象只存活一小段时间，而存活下来的小部分 Java 对象则会存活很长一段时间。



(pmd 中 Java 对象生命周期的直方图，红色的表示被逃逸分析优化掉的对象)

之所以要提到这个假设，是因为它造就了 Java 虚拟机的分代回收思想。简单来说，就是将堆空间划分为两代，分别叫做新生代和老年代。新生代用来存储新建的对象。当对象存活时间够长时，则将其移动到老年代。

Java 虚拟机可以给不同代使用不同的回收算法。对于新生代，我们猜测大部分的 Java 对象只存活一小段时间，那么便可以频繁地采用耗时较短的垃圾回收算法，让大部分的垃圾都能够在新生代被回收掉。

对于老年代，我们猜测大部分的垃圾已经在新生代中被回收了，而在老年代中的对象有大概率会继续存活。当真正触发针对老年代的回收时，则代表这个假设出错了，或者堆的空间已经耗尽了。

这时候，Java 虚拟机往往需要做一次全堆扫描，耗时也将不计成本。（当然，现代的垃圾回收器都在并发收集的道路上发展，来避免这种全堆扫描的情况。）

今天这一篇我们来关注一下针对新生代的 Minor GC。首先，我们来看看 Java 虚拟机中的堆具体是怎么划分的。

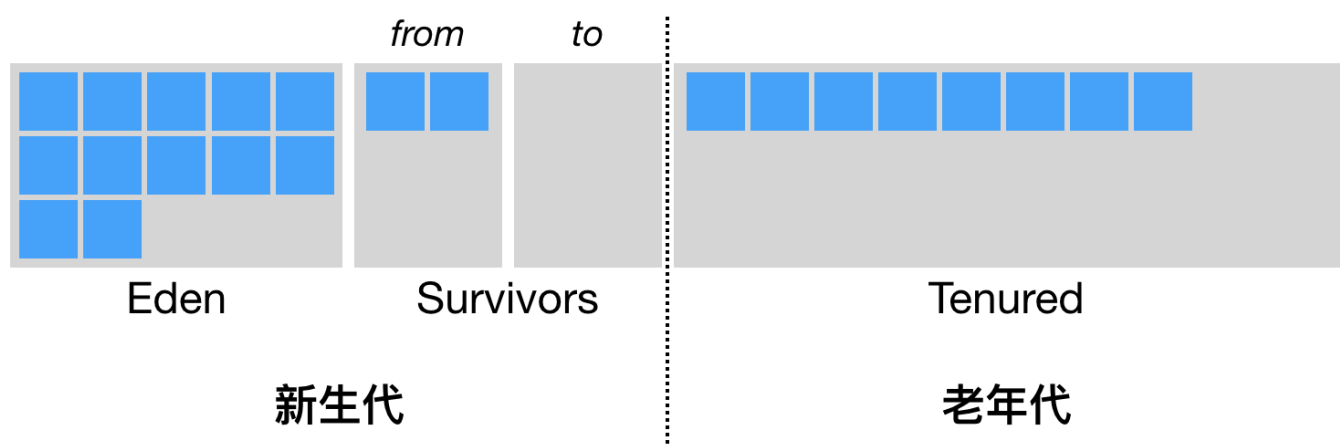
Java 虚拟机的堆划分

前面提到，Java 虚拟机将堆划分为新生代和老年代。其中，新生代又被划分为 Eden 区，以及两个大小相同的 Survivor 区。

默认情况下，Java 虚拟机采取的是一种动态分配的策略（对应 Java 虚拟机参数 -XX:+UsePSAdaptiveSurvivorSizePolicy），根据生成对象的速率，以及 Survivor 区的使用情况动态

调整 Eden 区和 Survivor 区的比例。

当然，你也可以通过参数 `-XX:SurvivorRatio` 来固定这个比例。但是需要注意的是，其中一个 Survivor 区会一直为空，因此比例越低浪费的堆空间将越高。



通常来说，当我们调用 `new` 指令时，它会在 Eden 区中划出一块作为存储对象的内存。由于堆空间是线程共享的，因此直接在这里边划空间是需要进行同步的。

否则，将有可能出现两个对象共用一段内存的事故。如果你还记得前两篇我用“停车位”打的比方的话，这里就相当于两个司机（线程）同时将车停入同一个停车位，因而发生剐蹭事故。

Java 虚拟机的解决方法是为每个司机预先申请多个停车位，并且只允许该司机停在自己的停车位上。那么当司机的停车位用完了该怎么办呢（假设这个司机代客泊车）？

答案是：再申请多个停车位便可以了。这项技术被称之为 TLAB（Thread Local Allocation Buffer，对应虚拟机参数 `-XX:+UseTLAB`，默认开启）。

具体来说，每个线程可以向 Java 虚拟机申请一段连续的内存，比如 2048 字节，作为线程私有的 TLAB。

这个操作需要加锁，线程需要维护两个指针（实际上可能更多，但重要也就两个），一个指向 TLAB 中空余内存的起始位置，一个则指向 TLAB 末尾。

接下来的 `new` 指令，便可以直接通过指针加法（bump the pointer）来实现，即把指向空余内存位置的指针加上所请求的字节数。

我猜测会有留言问为什么不把 bump the pointer 翻译成指针碰撞。这里先解释一下，在英语中我们通常省略了 bump up the pointer 中的 up。在这个上下文中 bump 的含义应为“提高”。另外一个例子是当我们发布软件的新版本时，也会说 bump the version number。

如果加法后空余内存指针的值仍小于或等于指向末尾的指针，则代表分配成功。否则，TLAB 已经

没有足够的空间来满足本次新建操作。这个时候，便需要当前线程重新申请新的 TLAB。

当 Eden 区的空间耗尽了怎么办？这个时候 Java 虚拟机便会触发一次 Minor GC，来收集新生代的垃圾。存活下来的对象，则会被送到 Survivor 区。

前面提到，新生代共有两个 Survivor 区，我们分别用 from 和 to 来指代。其中 to 指向的 Survivor 区是空的。

当发生 Minor GC 时，Eden 区和 from 指向的 Survivor 区中的存活对象会被复制到 to 指向的 Survivor 区中，然后交换 from 和 to 指针，以保证下一次 Minor GC 时，to 指向的 Survivor 区还是空的。

Java 虚拟机会记录 Survivor 区中的对象一共被来回复制了几次。如果一个对象被复制的次数为 15（对应虚拟机参数 `-XX:+MaxTenuringThreshold`），那么该对象将被晋升（promote）至老年代。另外，如果单个 Survivor 区已经被占用了 50%（对应虚拟机参数 `-XX:TargetSurvivorRatio`），那么较高复制次数的对象也会被晋升至老年代。

总而言之，当发生 Minor GC 时，我们应用了标记 - 复制算法，将 Survivor 区中的老存活对象晋升到老年代，然后将剩下的存活对象和 Eden 区的存活对象复制到另一个 Survivor 区中。理想情况下，Eden 区中的对象基本都死亡了，那么需要复制的数据将非常少，因此采用这种标记 - 复制算法的效果极好。

Minor GC 的另外一个好处是不用对整个堆进行垃圾回收。但是，它却有一个问题，那就是老年代的对象可能引用新生代的对象。也就是说，在标记存活对象的时候，我们需要扫描老年代中的对象。如果该对象拥有对新生代对象的引用，那么这个引用也会被作为 GC Roots。

这样一来，岂不是又做了一次全堆扫描呢？

卡表

HotSpot 给出的解决方案是一项叫做卡表（Card Table）的技术。该技术将整个堆划分为一个个大小为 512 字节的卡，并且维护一个卡表，用来存储每张卡的一个标识位。这个标识位代表对应的卡是否可能存有指向新生代对象的引用。如果可能存在，那么我们就认为这张卡是脏的。

在进行 Minor GC 的时候，我们便可以不用扫描整个老年代，而是在卡表中寻找脏卡，并将脏卡中的对象加入到 Minor GC 的 GC Roots 里。当完成所有脏卡的扫描之后，Java 虚拟机便会将所有脏卡的标识位清零。

由于 Minor GC 伴随着存活对象的复制，而复制需要更新指向该对象的引用。因此，在更新引用的同时，我们又会设置引用所在的卡的标识位。这个时候，我们可以确保脏卡中必定包含指向新生代对象的引用。

在 Minor GC 之前，我们并不能确保脏卡中包含指向新生代对象的引用。其原因和如何设置卡的标识位有关。

首先，如果想要保证每个可能有指向新生代对象引用的卡都被标记为脏卡，那么 Java 虚拟机需要截获每个引用型实例变量的写操作，并作出对应的写标识位操作。

这个操作在解释执行器中比较容易实现。但是在即时编译器生成的机器码中，则需要插入额外的逻辑。这也就是所谓的写屏障（write barrier，注意不要和 volatile 字段的写屏障混淆）。

写屏障需要尽可能地保持简洁。这是因为我们并不希望在每条引用型实例变量的写指令后跟着一大串注入的指令。

因此，写屏障并不会判断更新后的引用是否指向新生代中的对象，而是宁可错杀，不可放过，一律当成可能指向新生代对象的引用。

这么一来，写屏障便可精简为下面的伪代码 [1]。这里右移 9 位相当于除以 512，Java 虚拟机便是通过这种方式来从地址映射到卡表中的索引的。最终，这段代码会被编译成一条移位指令和一条存储指令。

```
CARD_TABLE [this address >> 9] = DIRTY;
```

虽然写屏障不可避免地带来一些开销，但是它能够加大 Minor GC 的吞吐率（应用运行时间 / (应用运行时间 + 垃圾回收时间) ）。总的来说还是值得的。不过，在高并发环境下，写屏障又带来了虚共享（false sharing）问题 [2]。

在介绍对象内存布局中我曾提到虚共享问题，讲的是几个 volatile 字段出现在同一缓存行里造成的虚共享。这里的虚共享则是卡表中不同卡的标识位之间的虚共享问题。

在 HotSpot 中，卡表是通过 byte 数组来实现的。对于一个 64 字节的缓存行来说，如果用它来加载部分卡表，那么它将对对应 64 张卡，也就是 32KB 的内存。

如果同时有两个 Java 线程，在这 32KB 内存中进行引用更新操作，那么也将造成存储卡表的同一部分的缓存行的写回、无效化或者同步操作，因而间接影响程序性能。

为此，HotSpot 引入了一个新的参数 -XX:+UseCondCardMark，来尽量减少写卡表的操作。其伪代码如下所示：

```
if (CARD_TABLE [this address >> 9] != DIRTY)
    CARD_TABLE [this address >> 9] = DIRTY;
```

总结与实践

今天我介绍了 Java 虚拟机中垃圾回收具体实现的一些通用知识。

Java 虚拟机将堆分为新生代和老年代，并且对不同代采用不同的垃圾回收算法。其中，新生代分为 Eden 区和两个大小一致的 Survivor 区，并且其中一个 Survivor 区是空的。

在只针对新生代的 Minor GC 中，Eden 区和非空 Survivor 区的存活对象会被复制到空的 Survivor 区中，当 Survivor 区中的存活对象复制次数超过一定数值时，它将被晋升至老年代。

因为 Minor GC 只针对新生代进行垃圾回收，所以在枚举 GC Roots 的时候，它需要考虑从老年代到新生代的引用。为了避免扫描整个老年代，Java 虚拟机引入了名为卡表的技术，大致地标出可能存在老年代到新生代引用的内存区域。

由于篇幅的原因，我没有讲解 Java 虚拟机中具体的垃圾回收器。我在文章的末尾附了一段简单的介绍，如果你有兴趣的话可以参阅一下。

今天的实践环节，我们来看看 Java 对象的生命周期对垃圾回收的影响。

前面提到，Java 虚拟机的分代垃圾回收是基于大部分对象只存活一小段时间，小部分对象却存活一大段时间的假设的。

然而，实际情况中并非每个程序都符合前面提到的假设。如果一个程序拥有中等生命周期的对象，并且刚移动到老年代便不再使用，那么将给默认的垃圾回收策略造成极大的麻烦。

下面这段程序将生成 64G 的 Java 对象。并且，我通过 ALIVE_OBJECT_SIZE 这一变量来定义同时存活的 Java 对象的大小。这也是一种对于垃圾回收器来说比较直观的生命周期。

当我们使用 Java 8 的默认 GC，并且将新生代的空间限制在 100M 时，试着估算当 ALIVE_OBJECT_SIZE 为多少时，这段程序不会触发 Full GC（提示一下，如果 Survivor 区没法存储所有存活对象，将发生什么。）。实际运行情况又是怎样的？

```
// Run with java -XX:+PrintGC -Xmn100M -XX:PretenureSizeThreshold=10000 LifetimeTest
// You may also try with -XX:+PrintHeapAtGC, -XX:-UsePSAdaptiveSurvivorSizePolicy or -
XX:SurvivorRatio=N

public class LifetimeTest {

    private static final int K = 1024;

    private static final int M = K * K;

    private static final int G = K * M;

    private static final int ALIVE_OBJECT_SIZE = 32 * M;
```

```
public static void main(String[] args) {  
    int length = ALIVE_OBJECT_SIZE / 64;  
    ObjectOf64Bytes[] array = new ObjectOf64Bytes[length];  
    for (long i = 0; i < G; i++) {  
        array[(int) (i % length)] = new ObjectOf64Bytes();  
    }  
}  
  
class ObjectOf64Bytes {  
    long placeholder0;  
    long placeholder1;  
    long placeholder2;  
    long placeholder3;  
    long placeholder4;  
    long placeholder5;  
}
```

附录：Java 虚拟机中的垃圾回收器

针对新生代的垃圾回收器共有三个：Serial，Parallel Scavenge 和 Parallel New。这三个采用的都是标记 - 复制算法。其中，Serial 是一个单线程的，Parallel New 可以看成 Serial 的多线程版本。Parallel Scavenge 和 Parallel New 类似，但更加注重吞吐率。此外，Parallel Scavenge 不能与 CMS 一起使用。

针对老年代的垃圾回收器也有三个：刚刚提到的 Serial Old 和 Parallel Old，以及 CMS。Serial Old 和 Parallel Old 都是标记 - 压缩算法。同样，前者是单线程的，而后者可以看成前者的多线程版本。

CMS 采用的是标记 - 清除算法，并且是并发的。除了少数几个操作需要 Stop-the-world 之外，它可以在应用程序运行过程中进行垃圾回收。在并发收集失败的情况下，Java 虚拟机会使用其他两个压缩型垃圾回收器进行一次垃圾回收。由于 G1 的出现，CMS 在 Java 9 中已被废弃 [3]。

G1 (Garbage First) 是一个横跨新生代和老年代的垃圾回收器。实际上，它已经打乱了前面所说的堆结构，直接将堆分成极其多个区域。每个区域都可以充当 Eden 区、Survivor 区或者老年代中的一个。它采用的是标记 - 压缩算法，而且和 CMS 一样都能够在应用程序运行过程中并发地进行垃圾回收。

G1 能够针对每个细分的区域来进行垃圾回收。在选择进行垃圾回收的区域时，它会优先回收死亡对

象较多的区域。这也是 G1 名字的由来。

即将到来的 Java 11 引入了 ZGC，宣称暂停时间不超过 10ms。如果你感兴趣的话，可参考 R 大的这篇文章 [4]。

[1] <http://psy-lob-saw.blogspot.com/2014/10/the-jvm-write-barrier-card-marking.html>

[2] <https://blogs.oracle.com/dave/false-sharing-induced-by-card-table-marking>

[3] <http://openjdk.java.net/jeps/291>

[4] <https://www.zhihu.com/question/287945354/answer/458761494>



版权归极客邦科技所有，未经许可不得转载

精选留言



阿杜

写得真好，搞清楚了之前没掌握透彻的概念：

- (1) TLAB是为了避免对象分配时对内存的竞争
- (2) 卡表是为了处理minor gc时老年代对新生代的引用，为了避免整堆扫描而提出了卡表的概念

提个问题：JVM中堆内存根系统的物理内存是如何对应的，这个可以从哪里看？

2018-08-17



饶绍泉

TLAB 和 工作内存是什么关系？

2

0

2018-08-19



赤云

0

请求方法为线程私有，上面那个问方法局部变量的，，，搞清楚了，，，n个线程会复制n份。不冲突，只有多线程写成员变量才会出现。

2018-08-18



郭俊杰

0

老师，你好，一直想问个问题，多线程访问共享变量会存在线程安全问题，那么方法内部通过new的局部变量是不是也存在安全问题呢？希望老师帮忙回答下感谢

2018-08-18



侯金彪

0

看某些资料介绍说tlab是将对象创建在栈内存上，并且与逃逸分析一起用，这样在释放栈内存时就直接将这部分内存一起释放了，降低了gc成本。您讲解的说tlab是提前在堆内存中分配空间，这样没有降低gc成本呀！

2018-08-18



code-artist

0

通过GC_roots探索存活对象的时候，有的对象在新生代，有的对象在老年代。如果当前对象处在老年代而且在脏卡中，需要继续深度遍历对象。若不在脏卡中，就没必要进一步深度遍历下去了。不知道我的理解对不？

2018-08-17



John(易筋)

0

第二条只能用这个通过

```
java -XX:+PrintHeapAtGC -XX:-UsePSAdaptiveSurvivorSizePolicy LifetimeTest
```

2018-08-17

讲堂 深入拆解 Java 虚拟机 文章详情

13 | Java内存模型

2018-08-20 郑雨迪



13 | Java内存模型

朗读人：郑雨迪 13'45" | 6.30M

我们先来看一个反常识的例子。

```
int a=0, b=0;

public void method1() {
    int r2 = a;
    b = 1;
}

public void method2() {
    int r1 = b;
    a = 2;
}
```

这里我定义了两个共享变量 `a` 和 `b`，以及两个方法。第一个方法将局部变量 `r2` 赋值为 `a`，然后将共享变量 `b` 赋值为 1。第二个方法将局部变量 `r1` 赋值为 `b`，然后将共享变量 `a` 赋值为 2。请问 (`r1`, `r2`) 的可能值都有哪些？

在单线程环境下，我们可以先调用第一个方法，最终 (`r1`, `r2`) 为 (1, 0)；也可以先调用第二个方法，最终为 (0, 2)。

在多线程环境下，假设这两个方法分别跑在两个不同的线程之上，如果 Java 虚拟机在执行了任一方法的第一条赋值语句之后便切换线程，那么最终结果将可能出现 (0, 0) 的情况。

除上述三种情况之外，Java 语言规范第 17.4 小节 [1] 还介绍了一种看似不可能的情况 (1, 2)。

造成这一情况的原因有三个，分别为即时编译器的重排序，处理器的乱序执行，以及内存系统的重排序。由于后两种原因涉及具体的体系架构，我们暂且放到一边。下面我先来讲一下编译器优化的重排序是怎么一回事。

首先需要说明一点，即时编译器（和处理器）需要保证程序能够遵守 `as-if-serial` 属性。通俗地说，就是在单线程情况下，要给程序一个顺序执行的假象。即经过重排序的执行结果要与顺序执行的结果保持一致。

另外，如果两个操作之间存在数据依赖，那么即时编译器（和处理器）不能调整它们的顺序，否则将会造成程序语义的改变。

```
int a=0, b=0;

public void method1() {
    int r2 = a;
    b = 1;
    .. // Code uses b
    if (r2 == 2) {
        ..
    }
}
```

在上面这段代码中，我扩展了先前例子中的第一个方法。新增的代码会先使用共享变量 `b` 的值，然后再使用局部变量 `r2` 的值。

此时，即时编译器有两种选择。

第一，在一开始便将 a 加载至某一寄存器中，并且在接下来 b 的赋值操作以及使用 b 的代码中避免使用该寄存器。第二，在真正使用 r2 时才将 a 加载至寄存器中。这么一来，在执行使用 b 的代码时，我们不再霸占一个通用寄存器，从而减少需要借助栈空间的情况。

```
int a=0, b=0;

public void method1() {
    for (..) {
        int r2 = a;
        b = 1;
        .. // Code uses r2 and rewrites a
    }
}
```

另一个例子则是将第一个方法的代码放入一个循环中。除了原本的两条赋值语句之外，我只在循环中添加了使用 r2，并且更新 a 的代码。由于对 b 的赋值是循环无关的，即时编译器很有可能将其移出循环之前，而对 r2 的赋值语句还停留在循环之中。

如果想要复现这两个场景，你可能需要添加大量有意义的局部变量，来给寄存器分配算法施加压力。

可以看到，即时编译器的优化可能将原本字段访问的执行顺序打乱。在单线程环境下，由于 as-if-serial 的保证，我们无须担心顺序执行不可能发生的情况，如 $(r1, r2) = (1, 2)$ 。

然而，在多线程情况下，这种数据竞争（data race）的情况是有可能发生的。而且，Java 语言规范将其归咎于应用程序没有作出恰当的同步操作。

Java 内存模型与 happens-before 关系

为了让应用程序能够免于数据竞争的干扰，Java 5 引入了明确定义的 Java 内存模型。其中最为重要的一个概念便是 happens-before 关系。happens-before 关系是用来描述两个操作的内存可见性的。如果操作 X happens-before 操作 Y，那么 X 的结果对于 Y 可见。

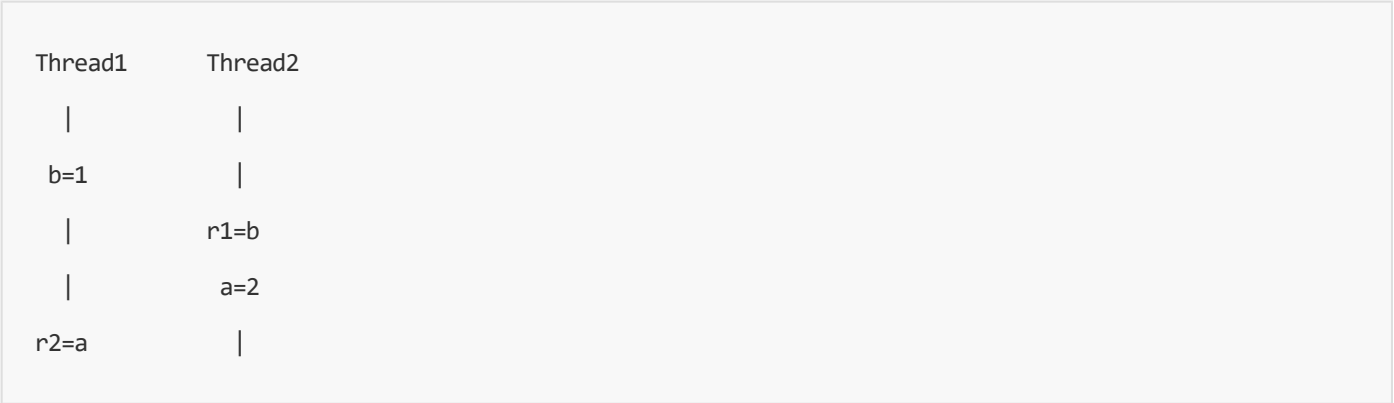
在同一个线程中，字节码的先后顺序（program order）也暗含了 happens-before 关系：在程序控制流路径中靠前的字节码 happens-before 靠后的字节码。然而，这并不意味着前者一定在后者之前执行。实际上，如果后者没有观测前者的运行结果，即后者没有数据依赖于前者，那么它们可能会被重排序。

除了线程内的 happens-before 关系之外，Java 内存模型还定义了下述线程间的 happens-before 关系。

- 1. 解锁操作 happens-before 之后（这里指时钟顺序先后）对同一把锁的加锁操作。
- 2. volatile 字段的写操作 happens-before 之后（这里指时钟顺序先后）对同一字段的读操作。
- 3. 线程的启动操作（即 Thread.starts()） happens-before 该线程的第一个操作。
- 4. 线程的最后一个操作 happens-before 它的终止事件（即其他线程通过 Thread.isAlive() 或 Thread.join() 判断该线程是否中止）。
- 5. 线程对其他线程的中断操作 happens-before 被中断线程所收到的中断事件（即被中断线程的 InterruptedException 异常，或者第三个线程针对被中断线程的 Thread.interrupted 或者 Thread.isInterrupted 调用）。
- 6. 构造器中的最后一个操作 happens-before 析构器的第一个操作。

happens-before 关系还具备传递性。如果操作 X happens-before 操作 Y，而操作 Y happens-before 操作 Z，那么操作 X happens-before 操作 Z。

在文章开头的例子中，程序没有定义任何 happens-before 关系，仅拥有默认的线程内 happens-before 关系。也就是 r2 的赋值操作 happens-before b 的赋值操作，r1 的赋值操作 happens-before a 的赋值操作。



拥有 happens-before 关系的两对赋值操作之间没有数据依赖，因此即时编译器、处理器都可能对其进行重排序。举例来说，只要将 b 的赋值操作排在 r2 的赋值操作之前，那么便可以按照赋值 b，赋值 r1，赋值 a，赋值 r2 的顺序得到 (1, 2) 的结果。

那么如何解决这个问题呢？答案是，将 a 或者 b 设置为 volatile 字段。

比如说将 b 设置为 volatile 字段。假设 r1 能够观测到 b 的赋值结果 1。显然，这需要 b 的赋值操作在时钟顺序上先于 r1 的赋值操作。根据 volatile 字段的 happens-before 关系，我们知道 b 的赋值操作 happens-before r1 的赋值操作。

```
int a=0;
volatile int b=0;

public void method1() {
```

```
    int r2 = a;

    b = 1;
}

public void method2() {
    int r1 = b;

    a = 2;
}
```

根据同一个线程中，字节码顺序所暗含的 happens-before 关系，以及 happens-before 关系的传递性，我们可以轻易得出 r2 的赋值操作 happens-before a 的赋值操作。

这也就意味着，当对 a 进行赋值时，对 r2 的赋值操作已经完成了。因此，在 b 为 volatile 字段的情况下，程序不可能出现 (r1, r2) 为 (1, 2) 的情况。

由此可以看出，解决这种数据竞争问题的关键在于构造一个跨线程的 happens-before 关系：操作 X happens-before 操作 Y，使得操作 X 之前的字节码的结果对操作 Y 之后的字节码可见。

Java 内存模型的底层实现

在理解了 Java 内存模型的概念之后，我们现在来看看它的底层实现。Java 内存模型是通过内存屏障（memory barrier）来禁止重排序的。

对于即时编译器来说，它会针对前面提到的每一个 happens-before 关系，向正在编译的目标方法中插入相应的读读、读写、写读以及写写内存屏障。

这些内存屏障会限制即时编译器的重排序操作。以 volatile 字段访问为例，所插入的内存屏障将不允许 volatile 字段写操作之前的内存访问被重排序至其之后；也将不允许 volatile 字段读操作之后的内存访问被重排序至其之前。

然后，即时编译器将根据具体的底层体系架构，将这些内存屏障替换成具体的 CPU 指令。以我们日常接触的 X86_64 架构来说，读读、读写以及写写内存屏障是空操作（no-op），只有写读内存屏障会被替换成具体指令 [2]。

在文章开头的例子中，method1 和 method2 之中的代码均属于先读后写（假设 r1 和 r2 被存储在寄存器之中）。X86_64 架构的处理器并不能将读操作重排序至写操作之后，具体可参考 Intel Software Developer Manual Volumn 3，8.2.3.3 小节。因此，我认为例子中的重排序必然是即时编译器造成的。

举例来说，对于 volatile 字段，即时编译器将在 volatile 字段的读写操作前后各插入一些内存屏障。

然而，在 X86_64 架构上，只有 volatile 字段写操作之后的写读内存屏障需要用具体指令来替代。

（HotSpot 所选取的具体指令是 lock add DWORD PTR [rsp],0x0，而非 mfence[3]。）

该具体指令的效果，可以简单理解为强制刷新处理器的写缓存。写缓存是处理器用来加速内存存储效率的一项技术。

在碰到内存写操作时，处理器并不会等待该指令结束，而是直接开始下一指令，并且依赖于写缓存将更改的数据同步至主内存（main memory）之中。

强制刷新写缓存，将使得当前线程写入 volatile 字段的值（以及写缓存中已有的其他内存修改），同步至主内存之中。

由于内存写操作同时会无效化其他处理器所持有的、指向同一内存地址的缓存行，因此可以认为其他处理器能够立即见到该 volatile 字段的最新值。

锁，volatile 字段，final 字段与安全发布

下面我来讲讲 Java 内存模型涉及的几个关键词。

前面提到，锁操作同样具备 happens-before 关系。具体来说，解锁操作 happens-before 之后对同一把锁的加锁操作。实际上，在解锁时，Java 虚拟机同样需要强制刷新缓存，使得当前线程所修改的内存对其他线程可见。

需要注意的是，锁操作的 happens-before 规则的关键字是同一把锁。也就意味着，如果编译器能够（通过逃逸分析）证明某把锁仅被同一线程持有，那么它可以移除相应的加锁解锁操作。

因此也就不再强制刷新缓存。举个例子，即时编译后的 synchronized (new Object()) {}，可能等同于空操作，而不会强制刷新缓存。

volatile 字段可以看成一种轻量级的、不保证原子性的同步，其性能往往优于（至少不亚于）锁操作。然而，频繁地访问 volatile 字段也会因为不断地强制刷新缓存而严重影响程序的性能。

在 X86_64 平台上，只有 volatile 字段的写操作会强制刷新缓存。因此，理想情况下对 volatile 字段的使用应当多读少写，并且应当只有一个线程进行写操作。

volatile 字段的另一个特性是即时编译器无法将其分配到寄存器里。换句话说，volatile 字段的每次访问均需要直接从内存中读写。

final 实例字段则涉及新建对象的发布问题。当一个对象包含 final 实例字段时，我们希望其他线程只能看到已初始化的 final 实例字段。

因此，即时编译器会在 final 字段的写操作后插入一个写写屏障，以防某些优化将新建对象的发布（即将实例对象写入一个共享引用中）重排序至 final 字段的写操作之前。在 X86_64 平台上，写写

屏障是空操作。

新建对象的安全发布（safe publication）问题不仅仅包括 final 实例字段的可见性，还包括其他实例字段的可见性。

当发布一个已初始化的对象时，我们希望所有已初始化的实例字段对其他线程可见。否则，其他线程可能见到一个仅部分初始化的新建对象，从而造成程序错误。这里我就不展开了。如果你感兴趣的话，可以参考这篇博客 [4]。

总结与实践

今天我主要介绍了 Java 的内存模型。

Java 内存模型通过定义了一系列的 happens-before 操作，让应用程序开发者能够轻易地表达不同线程的操作之间的内存可见性。

在遵守 Java 内存模型的前提下，即时编译器以及底层体系架构能够调整内存访问操作，以达到性能优化的效果。如果开发者没有正确地利用 happens-before 规则，那么将可能导致数据竞争。

Java 内存模型是通过内存屏障来禁止重排序的。对于即时编译器来说，内存屏障将限制它所能做的重排序优化。对于处理器来说，内存屏障会导致缓存的刷新操作。

今天的实践环节，我们来复现文章初始的例子。由于复现需要大量的线程切换事件，因此我借助了 OpenJDK CodeTools 项目的 jcstress 工具 [5]，来对该例子进行并发情况下的压力测试。具体的命令如下所示：

```
$ mvn archetype:generate -DinteractiveMode=false -DarchetypeGroupId=org.openjdk.jcstress -
DarchetypeArtifactId=jcstress-java-test-archetype -DarchetypeVersion=0.1.1 -DgroupId=org.sample -
DartifactId=test -Dversion=1.0
$ cd test
$ echo 'package org.sample;
import org.openjdk.jcstress.annotations.*;
import org.openjdk.jcstress.infra.results.IntResult2;
@JCStressTest
@Outcome(id = {"0", "0", "0", "2", "1", "0"}, expect = Expect.ACCEPTABLE, desc = "Normal outcome")
@Outcome(id = {"1", "2"}, expect = Expect.ACCEPTABLE_INTERESTING, desc = "Abnormal outcome")
@State
public class ConcurrencyTest {
    int a=0;
    int b=0; // 改成 volatile 试试？
```



```
@Actor
public void method1(IntResult2 r) {
    r.r2 = a;
    b = 1;
}

@Actor
public void method2(IntResult2 r) {
    r.r1 = b;
    a = 2;
}
}' > src/main/java/org/sample/ConcurrencyTest.java

$ mvn package

$ java -jar target/jcstress.jar
```

如果你想要复现非安全发布的情形，那么你可以试试这一测试用例 [6]。

[1] <https://docs.oracle.com/javase/specs/jls/se10/html/jls-17.html#jls-17.4>

[2] <http://gee.cs.oswego.edu/dl/jmm/cookbook.html>

[3] <https://blogs.oracle.com/dave/instruction-selection-for-volatile-fences--mfence-vs-lock:add>

[4] <http://vikan.com/blog/post/2014/02/14/java-safe-publication/>

[5] <https://wiki.openjdk.java.net/display/CodeTools/jcstress>

[6] <http://hg.openjdk.java.net/code-tools/jcstress/Ple/64f2cf32fa0a/tests-custom/src/main/java/org/openjdk/jcstress/tests/unsafe/UnsafePublication.java>





追梦

1

老师，求讲解下jvm中代码如何实现的内存屏障

2018-08-20

作者回复

即时编译器生成的代码里会使用CPU的内存屏障指令。HotSpot采用的lock前缀的指令，lock add DWORD PTR [rsp] 0。它也会刷缓存。

至于在即时编译器里禁止重排序所使用的“内存屏障”，就是一个特殊的编译器中间表达形式节点。

2018-08-20



黑崽

0

请教个问题。刚才有说不会把volatile放到寄存器，但是应该会在栈里面对吧。直接读取主内存，读写的是栈数据，然后利用堆内存和栈上数据是利用写缓存刷新同步的？

2018-08-22

作者回复

我可能没有在原文中讲清楚。这里指的是volatile变量不能被分配到寄存器中，但是计算还是加载到寄存器中来计算的。

所谓的分配到寄存器中，你可以理解为编译器将内存中的值缓存在寄存器中，之后一直用访问寄存器来代表对这个内存的访问的。假设我们要遍历一个数组，数组的长度是内存中的值。由于我们每次循环都要比较一次，因此编译器决定把它放在寄存器中，免得每次比较都要读一次内存。对于会更改的内存值，编译器也可以先缓存至寄存器，最后更新回内存即可。

Volatile会禁止上述优化。

2018-08-22



Jerry Chan

0

博客在哪里啊？

2018-08-21

作者回复

文末[4]所指向的链接。

2018-08-22



qpm

0

老师我提个问题。一个共享对象的变量是非volatile的，那么这个变量的写入会先写到寄存器上，再写回内存吗？那么jvm是不是无论如何都不保证啥时候变量的值会写回内存。假如另一个线程加锁访问这个变量，是不是jvm也不保证它能拿到最新数据。

2018-08-20

作者回复

对的！如果即时编译器把那个变量放在寄存器里维护，那么另一个线程也没辙。

2018-08-22



大场镇车王

0

老师 为什么volatile内存屏障不允许所有写操作之前的读操作被重排序到写操作之后？前面不是说volatile的写操作happens before对同一字段的读操作吗

2018-08-20

作者回复

volatile字段的happens-before关系指的是在两个不同线程中，volatile的写操作 happens-before 之后对同一字段的读操作。这里有个关键字之后，指的是时间上的先后。也就是我这边写，你之后再读就一定能读得到我刚刚写的值。普通字段则没有这个保证。

屏障不允许重排序是针对即时编译器的。写后对同一字段的读，属于数据依赖，本来也不可以重排序的。

2018-08-20



Alex 1984

0

老师，我在一些技术文章里看到说 volatile 的变量是存在工作内存，这个工作内存是一个什么概念？

2018-08-20

作者回复

工作内存是JMM抽象出来的一个概念。你可以映射到实际的CPU缓存。

2018-08-22



godtrue

2

恩，这节听了好几遍，也读了几遍，最后还是感觉蒙蒙的。

下面说下我的感受：

1:一图胜千言，尤其对于描述复杂的东西，这个建议其他同学也有提议的，希望雨迪采纳一下

2:感觉有些概念没有解释，比如：

2-1:Java内存模型，这节不就是要讲Java内存类型是什么？有什么特点？为什么这么设计嘛？不过我没看到这些内容，当然，特点是讲了的 happens-before 机制

2-2:内存屏蔽，这是什么意思？它怎么就能禁止重排序啦？还有有其引申出的各种屏蔽是怎么回事呢？也没完全明白

3:前面讲 as-if-serial 大概明白什么意思了，多处理器为了提高效率会采用流水线的方式来执行程序指令，但是同时要保证程序执行逻辑的正确性，所以，实际执行程序的指令和代码层面上会有不同，这个是由编译器来保证的，虽然执行逻辑不同但是程序逻辑是没变的，单线程没问题，但是多线程就变得复杂一些了，光靠这种方式保证不了啦，所以引出了下文

4:happens-before，这个概念和作用比较容易理解，线程内有这种关系，线程间更有，这个机制我

认为就是为了多线程环境下为了保证程序逻辑正确性的一种方式，不过它的具体实现细节感觉没理解，不清楚他是怎么办到的

5:volatile 这个关键字之前也学习过，作用是保证内容的修改对所有线程可见，原理是修改后同步更新所有的内容，这是因为内存和处理器直接还是存在距离的，比如：内存-一级缓存-二级缓存-各种寄存器-cpu，如果是内存-cpu，则不会有这种问题了，不过性能也就不行了

2018-08-23



熊猫酒仙

1

老师，既然有写缓存，是不是也有读缓存呢？强制刷新写缓存，无效化相应的内存数据，那么这些内存数据的读缓存也就失效了，需要重新加载最新数据，是否可以这样理解？

另外强制刷新写缓存是否也有粒度一说？就是我们加的锁也有粒度之分，那么所触发的强制刷新写缓存的区域可能不一样？

2018-08-22



啃yi嘴泥

0

老师你好，关于指令重排序有点不太理解，指令重排序的粒度是方法级别的，还是整个源文件级别的。文中说道，b 加了volatile后，能够保证 b=1 先于r1=b，这个我能理解，但是如何保证不会因为指令重排导致 b=1 先于r2=a发生呢？文中虽然说了，同一个线程中，字节码顺序暗含了r2=a happen before b=1，但是文中也提到了，拥有happen-before关系的两对赋值操作之间没有数据依赖，处理器可以指令重排序。r2=a 和b=1之间没有数据依赖呀！不好意思，这块有点迷糊，老师能给详细解答下不？

2018-08-24



梦想启动的蜗牛

0

再次复习下????

2018-08-20



agilejava

0

文字太多，最好能结合一些图形会更方便理解

2018-08-20

作者回复

多谢建议！

2018-08-20

讲堂 深入拆解 Java 虚拟机 文章详情

14 | Java虚拟机是怎么实现 synchronized的?

2018-08-22 郑雨迪



14 | Java虚拟机是怎么实现synchronized的?

朗读人：郑雨迪 14'26" | 6.62M

在 Java 程序中，我们可以利用 synchronized 关键字来对程序进行加锁。它既可以用来声明一个 synchronized 代码块，也可以直接标记静态方法或者实例方法。

当声明 synchronized 代码块时，编译而成的字节码将包含 monitorenter 和 monitorexit 指令。这两种指令均会消耗操作数栈上的一个引用类型的元素（也就是 synchronized 关键字括号里的引用），作为所要加锁解锁的锁对象。

```
public void foo(Object lock) {
    synchronized (lock) {
        lock.hashCode();
    }
}

// 上面的 Java 代码将编译为下面的字节码
public void foo(java.lang.Object);
```

Code:

```
0: aload_1
1: dup
2: astore_2
3: monitorenter
4: aload_1
5: invokevirtual java/lang/Object.hashCode:()I
8: pop
9: aload_2
10: monitorexit
11: goto          19
14: astore_3
15: aload_2
16: monitorexit
17: aload_3
18: athrow
19: return
```

Exception table:

from	to	target	type
4	11	14	any
14	17	14	any

我在文稿中贴了一段包含 `synchronized` 代码块的 Java 代码，以及它所编译而成的字节码。你可能会留意到，上面的字节码中包含一个 `monitorenter` 指令以及多个 `monitorexit` 指令。这是因为 Java 虚拟机需要确保所获得的锁在正常执行路径，以及异常执行路径上都能够被解锁。

你可以根据我在介绍异常处理时介绍过的知识，对照字节码和异常处理表来构造所有可能的执行路径，看看在执行了 `monitorenter` 指令之后，是否都有执行 `monitorexit` 指令。

当用 `synchronized` 标记方法时，你会看到字节码中方法的访问标记包括 `ACC_SYNCHRONIZED`。该标记表示在进入该方法时，Java 虚拟机需要进行 `monitorenter` 操作。而在退出该方法时，不管是正常返回，还是向调用者抛异常，Java 虚拟机均需要进行 `monitorexit` 操作。

```
public synchronized void foo(Object lock) {
    lock.hashCode();
}
```



```
// 上面的 Java 代码将编译为下面的字节码

public synchronized void foo(java.lang.Object);

    descriptor: (Ljava/lang/Object;)V
    flags: (0x0021) ACC_PUBLIC, ACC_SYNCHRONIZED
    Code:
        stack=1, locals=2, args_size=2

        0: aload_1
        1: invokevirtual java/lang/Object.hashCode:()I
        4: pop
        5: return
```

这里 `monitorenter` 和 `monitorexit` 操作所对应的锁对象是隐式的。对于实例方法来说，这两个操作对应的锁对象是 `this`；对于静态方法来说，这两个操作对应的锁对象是则是所在类的 `Class` 实例。

关于 `monitorenter` 和 `monitorexit` 的作用，我们可以抽象地理解为每个锁对象拥有一个锁计数器和一个指向持有该锁的线程的指针。

当执行 `monitorenter` 时，如果目标锁对象的计数器为 0，那么说明它没有被其他线程所持有。在这种情况下，Java 虚拟机会将该锁对象的持有线程设置为当前线程，并且将其计数器加 1。

在目标锁对象的计数器不为 0 的情况下，如果锁对象的持有线程是当前线程，那么 Java 虚拟机可以将其计数器加 1，否则需要等待，直至持有线程释放该锁。

当执行 `monitorexit` 时，Java 虚拟机则需将锁对象的计数器减 1。当计数器减为 0 时，那便代表该锁已经被释放掉了。

之所以采用这种计数器的方式，是为了允许同一个线程重复获取同一把锁。举个例子，如果一个 Java 类中拥有多个 `synchronized` 方法，那么这些方法之间的相互调用，不管是直接的还是间接的，都会涉及对同一把锁的重复加锁操作。因此，我们需要设计这么一个可重入的特性，来避免编程里的隐式约束。

说完抽象的锁算法，下面我们便来介绍 HotSpot 虚拟机中具体的锁实现。

重量级锁

重量级锁是 Java 虚拟机中最为基础的锁实现。在这种状态下，Java 虚拟机会阻塞加锁失败的线程，并且在目标锁被释放的时候，唤醒这些线程。

Java 线程的阻塞以及唤醒，都是依靠操作系统来完成的。举例来说，对于符合 `posix` 接口的操作系统（如 `macOS` 和绝大部分的 `Linux`），上述操作是通过 `pthread` 的互斥锁（`mutex`）来实现的。此外，这些操作将涉及系统调用，需要从操作系统的用户态切换至内核态，其开销非常之大。

为了尽量避免昂贵的线程阻塞、唤醒操作，Java 虚拟机会在线程进入阻塞状态之前，以及被唤醒后竞争不到锁的情况下，进入自旋状态，在处理器上空跑并且轮询锁是否被释放。如果此时锁恰好被释放了，那么当前线程便无须进入阻塞状态，而是直接获得这把锁。

与线程阻塞相比，自旋状态可能会浪费大量的处理器资源。这是因为当前线程仍处于运行状况，只不过跑的是无用指令。它期望在运行无用指令的过程中，锁能够被释放出来。

我们可以用等红绿灯作为例子。Java 线程的阻塞相当于熄火停车，而自旋状态相当于怠速停车。如果红灯的等待时间非常长，那么熄火停车相对省油一些；如果红灯的等待时间非常短，比如说我们在 `synchronized` 代码块里只做了一个整型加法，那么在短时间内锁肯定会被释放出来，因此怠速停车更加合适。

然而，对于 Java 虚拟机来说，它并不能看到红灯的剩余时间，也就没办法根据等待时间的长短来选择自旋还是阻塞。Java 虚拟机给出的方案是自适应自旋，根据以往自旋等待时是否能够获得锁，来动态调整自旋的时间（循环数目）。

就我们的例子来说，如果之前不熄火等到了绿灯，那么这次不熄火的时间就长一点；如果之前不熄火没等到绿灯，那么这次不熄火的时间就短一点。

自旋状态还带来另外一个副作用，那便是不公平的锁机制。处于阻塞状态的线程，并没有办法立刻竞争被释放的锁。然而，处于自旋状态的线程，则很有可能优先获得这把锁。

轻量级锁

你可能见到过深夜的十字路口，四个方向都闪黄灯的情况。由于深夜十字路口的车辆来往可能比较少，如果还设置红绿灯交替，那么很有可能出现四个方向仅有一辆车在等红灯的情况。

因此，红绿灯可能被设置为闪黄灯的情况，代表车辆可以自由通过，但是司机需要注意观察（个人理解，实际意义请咨询交警部门）。

Java 虚拟机也存在着类似的情形：多个线程在不同的时间段请求同一把锁，也就是说没有锁竞争。针对这种情形，Java 虚拟机采用了轻量级锁，来避免重量级锁的阻塞以及唤醒。

在介绍轻量级锁的原理之前，我们先来了解一下 Java 虚拟机是怎么区分轻量级锁和重量级锁的。

（你可以参照[HotSpot Wiki](#)里这张图阅读。）

在对象内存布局那一篇中我曾经介绍了对象头中的标记字段（mark word）。它的最后两位便被用来表示该对象的锁状态。其中，00 代表轻量级锁，01 代表无锁（或偏向锁），10 代表重量级锁，11 则跟垃圾回收算法的标记有关。

当进行加锁操作时，Java 虚拟机会判断是否已经是重量级锁。如果不是，它会在当前线程的当前栈

帧中划出一块空间，作为该锁的锁记录，并且将锁对象的标记字段复制到该锁记录中。

然后，Java 虚拟机会尝试用 CAS（compare-and-swap）操作替换锁对象的标记字段。这里解释一下，CAS 是一个原子操作，它会比较目标地址的值是否和期望值相等，如果相等，则替换为一个新的值。

假设当前锁对象的标记字段为 X...XYZ，Java 虚拟机会比较该字段是否为 X...X01。如果是，则替换为刚才分配的锁记录的地址。由于内存对齐的缘故，它的最后两位为 00。此时，该线程已成功获得这把锁，可以继续执行了。

如果不是 X...X01，那么有两种可能。第一，该线程重复获取同一把锁。此时，Java 虚拟机会将锁记录清零，以代表该锁被重复获取。第二，其他线程持有该锁。此时，Java 虚拟机会将这把锁膨胀为重量级锁，并且阻塞当前线程。

当进行解锁操作时，如果当前锁记录（你可以将一个线程的所有锁记录想象成一个栈结构，每次加锁压入一条锁记录，解锁弹出一条锁记录，当前锁记录指的便是栈顶的锁记录）的值为 0，则代表重复进入同一把锁，直接返回即可。

否则，Java 虚拟机会尝试用 CAS 操作，比较锁对象的标记字段的值是否为当前锁记录的地址。如果是，则替换为锁记录中的值，也就是锁对象原本的标记字段。此时，该线程已经成功释放这把锁。

如果不是，则意味着这把锁已经被膨胀为重量级锁。此时，Java 虚拟机会进入重量级锁的释放过程，唤醒因竞争该锁而被阻塞了的线程。

偏向锁

如果说轻量级锁针对的情况很乐观，那么接下来的偏向锁针对的情况则更加乐观：从始至终只有一个线程请求某一把锁。

这就好比你在私家庄园里装了个红绿灯，并且庄园里只有你在开车。偏向锁的做法便是在红绿灯处识别来车的车牌号。如果匹配到你的车牌号，那么直接亮绿灯。

具体来说，在线程进行加锁时，如果该锁对象支持偏向锁，那么 Java 虚拟机会通过 CAS 操作，将当前线程的地址记录在锁对象的标记字段之中，并且将标记字段的最后三位设置为 101。

在接下来的运行过程中，每当有线程请求这把锁，Java 虚拟机只需判断锁对象标记字段中：最后三位是否为 101，是否包含当前线程的地址，以及 epoch 值是否和锁对象的类的 epoch 值相同。如果都满足，那么当前线程持有该偏向锁，可以直接返回。

这里的 epoch 值是一个什么概念呢？

我们先从偏向锁的撤销讲起。当请求加锁的线程和锁对象标记字段保持的线程地址不匹配时（而且

epoch 值相等，如若不等，那么当前线程可以将该锁重偏向至自己），Java 虚拟机需要撤销该偏向锁。这个撤销过程非常麻烦，它要求持有偏向锁的线程到达安全点，再将偏向锁替换成轻量级锁。

如果某一类锁对象的总撤销数超过了一个阈值（对应 Java 虚拟机参数 -XX:BiasedLockingBulkRebiasThreshold，默认为 20），那么 Java 虚拟机会宣布这个类的偏向锁失效。

具体的做法便是在每个类中维护一个 epoch 值，你可以理解为第几代偏向锁。当设置偏向锁时，Java 虚拟机需要将该 epoch 值复制到锁对象的标记字段中。

在宣布某个类的偏向锁失效时，Java 虚拟机实则将该类的 epoch 值加 1，表示之前那一代的偏向锁已经失效。而新设置的偏向锁则需要复制新的 epoch 值。

为了保证当前持有偏向锁并且已加锁的线程不至于因此丢锁，Java 虚拟机需要遍历所有线程的 Java 栈，找出该类已加锁的实例，并且将它们标记字段中的 epoch 值加 1。该操作需要所有线程处于安全点状态。

如果总撤销数超过另一个阈值（对应 Java 虚拟机参数 -XX:BiasedLockingBulkRevokeThreshold，默认值为 40），那么 Java 虚拟机会认为这个类已经不再适合偏向锁。此时，Java 虚拟机会撤销该类实例的偏向锁，并且在之后的加锁过程中直接为该类实例设置轻量级锁。

总结与实践

今天我介绍了 Java 虚拟机中 synchronized 关键字的实现，按照代价由高至低可分为重量级锁、轻量级锁和偏向锁三种。

重量级锁会阻塞、唤醒请求加锁的线程。它针对的是多个线程同时竞争同一把锁的情况。Java 虚拟机采取了自适应自旋，来避免线程在面对非常小的 synchronized 代码块时，仍会被阻塞、唤醒的情况。

轻量级锁采用 CAS 操作，将锁对象的标记字段替换为一个指针，指向当前线程栈上的一块空间，存储着锁对象原本的标记字段。它针对的是多个线程在不同时间段申请同一把锁的情况。

偏向锁只会在第一次请求时采用 CAS 操作，在锁对象的标记字段中记录下当前线程的地址。在之后的运行过程中，持有该偏向锁的线程的加锁操作将直接返回。它针对的是锁仅会被同一线程持有的情况。

今天的实践环节，我们来验证一个坊间传闻：调用 Object.hashCode() 会关闭该对象的偏向锁 [1]。

你可以采用参数 -XX:+PrintBiasedLockingStatistics 来打印各类锁的个数。由于 C2 使用的是另外一个参数 -XX:+PrintPreciseBiasedLockingStatistics，因此你可以限制 Java 虚拟机仅使用 C1 来即时编译（对应参数 -XX:TieredStopAtLevel=1）。

1. 通过参数 `-XX:+UseBiasedLocking`，比较开关偏向锁时的输出结果。
2. 在 `main` 方法的循环前添加 `lock.hashCode` 调用，并查看输出结果。
3. 在 `Lock` 类中复写 `hashCode` 方法，并查看输出结果。
4. 在 `main` 方法的循环前添加 `System.identityHashCode` 调用，并查看输出结果。

```
// Run with -XX:+UnlockDiagnosticVMOptions -XX:+PrintBiasedLockingStatistics -XX:TieredStopAtLevel=1
public class SynchronizedTest {

    static Lock lock = new Lock();
    static int counter = 0;

    public static void foo() {
        synchronized (lock) {
            counter++;
        }
    }

    public static void main(String[] args) throws InterruptedException {
        // lock.hashCode(); // Step 2
        // System.identityHashCode(lock); // Step 4
        for (int i = 0; i < 1_000_000; i++) {
            foo();
        }
    }

    static class Lock {
        // @Override public int hashCode() { return 0; } // Step 3
    }
}
```

[1] <https://blogs.oracle.com/dave/biased-locking-in-hotspot>



版权归极客邦科技所有，未经许可不得转载

精选留言



齐帆

太抽象了，老师能画点儿图吗.....

2018-08-22

作者回复

不好意思哈，因为网上有很多图，忘了放个链接了。

你可以参考wiki.openjdk.java.net/display/HotSpot/Synchronization中的图。

2018-08-22



NEO

老师关于偏向锁有个疑问“它针对的是锁仅会被同一线程持有的情况。”如果只有一个线程持有锁还有必要加锁吗？

2018-08-22

作者回复

哈，这个属于应用程序的问题，JVM只是观察到这种情况，并尝试做出优化。

有一种可能，就是很长一段时间内，只有一个线程频繁加锁，后面换成另外的线程，这样前面那段时间可以用偏向锁。

2018-08-22



何yuan

一直认为synchronized是重量锁，是否也不一定？jvm处理的时候是先将当偏向锁处理，然后慢慢膨胀为重量级锁的是吗？

2018-08-22

作者回复

默认情况下是的。以前有个延缓毫秒数-XX:BiasedLockingStartupDelay，一开始用轻量级锁，在启动四秒之后才开始用偏向锁。我记得Java 9还是10默认值改为0了。

2018-08-22



顺子

0

对于轻量级锁，markword本质是一个32位的bit，无锁情况下，末尾是01
加锁过程只是通过CAS把这32位的bit替换为lockrecord的地址，由于lockrecord地址只有30位，所以末尾补齐32后，末尾是00
不知道理解是否正确

然后有一个疑问，升级为重量级锁的时候，10是什么时候修改的?看到的文档都只说把指针修改为mutex，没提到10是怎么来的，是mutex30位地址加上10后CAS替换进去的吗

2018-08-24

作者回复

在64位Java虚拟机里，地址都是64位，而且都有内存对齐，也就是最后几位是0。markword里的最后两位是用 按位或 操作并进去的。

(链接的图对应的应该是32位虚拟机，所以地址是32位)

2018-08-24



梦想启动的蜗牛

0

轻量级锁应该是先通过cas替换锁的标记字段，然后再把标记字段复制到栈中对应的锁记录中把？

2018-08-24

作者回复

都可以的。在Graal里是先复制再CAS的，可以减少一些临界区里的代码。

2018-08-24



阿杜

0

你好，例子运行了，没看出啥不同，只有“fast path lock entries”这个对应的数值不同，slow path lock entries一直是2，其他的都是0。

建议作者给出自己的实验截图，方便我们对比

2018-08-24



谢阳

0

如果不是 X...X01，那么有两种可能。第一，该线程重复获取同一把锁。此时，Java 虚拟机会将锁记录清零，以代表该锁被重复获取。第二，其他线程持有该锁。此时，Java 虚拟机会将这把锁膨胀为重量级锁，并且阻塞当前线程。

老师这段不太明白。1 锁记录清零怎么理解？改变锁对象的标记字段吗？2 锁膨胀的时候其他线程还持有锁对象吧，这个时候膨胀会具体做什么操作？如果操作了锁对象的标记字段会影响稍后释放

锁的cas吗

2018-08-23



贾智文

0

文中说轻量级锁因为内存对齐所以标识位是00，那么为什么重量级锁的时候，存储内容也是指针，却没有内存对齐呢？

2018-08-22



唯一

0

老师，问一下加锁实际上都是加在当前线程吗

2018-08-22

作者回复

这个说法有点歧义。

按我的理解，你应该在问是否为当前线程获得这把锁？那么答案是对的，一直是当前线程获得这把锁。

另外，锁是加在目标锁对象上的。

2018-08-24



Scott

0

请教一下，不是很明白epoch的作用，偏向锁默认被撤销20次使偏向锁失效才更新这个字段，没有想明白epoch的必要性

2018-08-22



(^o^)

0

“否则，Java 虚拟机会尝试用 CAS 操作，比较锁对象的标记字段的值是否为当前锁记录的地址。如果是，则替换为锁记录中的值，也就是锁对象原本的标记字段。此时，该线程已经成功释放这把锁。

如果不是，则意味着这把锁已经被膨胀为重量级锁。”升级为重量级锁的过程，会改变锁对象的什么信息呢？

2018-08-22



(^o^)

0

Synchronized某个对象时，这个对象的锁先是偏向锁，后根据具体竞争情况先升级为轻量级锁再升级为重量级锁吗？

2018-08-22



阿杜

0

想看下jvm是怎么实现stream和lambda表达式的

2018-08-22

作者回复

前几篇的invokedynamic(下)有讲

2018-08-22



林QC

0

当声明 synchronized 代码块时，编译而成的字节码将包含 monitorenter 和 monitorexit 指令。 ~ ~ 老师，标记在方法上也有这两个指令吗？

2018-08-22

作者回复

没有，方法有ACC_SYNCHRONIZED标志符

2018-08-22



贾智文

0

果不是 X...X01，那么有两种可能。第一，该线程重复获取同一把锁。此时，Java 虚拟机会将锁记录清零，以代表该锁被重复获取。第二，其他线程持有该锁。此时，Java 虚拟机会将这把锁膨胀为重量级锁，并且阻塞当前线程。这两个情况怎么区分呢？

2018-08-22

作者回复

看标记字段所指向的轻量级锁锁纪录是否在自己栈上。

2018-08-22



贾智文

0

假设当前锁对象的标记字段为 X...XYZ，Java 虚拟机会比较该字段是否为 X...X01。老师请问这个x.....x01是什么，根据什么来的呢？

2018-08-22

作者回复

这里可能没写清楚。我指的是标记字段的bits是否为X..X01，其中X..X是从原本的标记字段拷过来的

。

2018-08-22

讲堂 深入拆解 Java 虚拟机 文章详情

15 | Java语法糖与Java编译器

2018-08-24 郑雨迪



15 | Java语法糖与Java编译器

朗读人：郑雨迪 09'58" | 4.58M

在前面的篇章中，我们多次提到了 Java 语法和 Java 字节码的差异之处。这些差异之处都是通过 Java 编译器来协调的。今天我们便来列举一下 Java 编译器的协调工作。

自动装箱与自动拆箱

首先要提到的便是 Java 的自动装箱（auto-boxing）和自动拆箱（auto-unboxing）。

我们知道，Java 语言拥有 8 个基本类型，每个基本类型都有对应的包装（wrapper）类型。

之所以需要包装类型，是因为许多 Java 核心类库的 API 都是面向对象的。举个例子，Java 核心类库中的容器类，就只支持引用类型。

当需要一个能够存储数值的容器类时，我们往往定义一个存储包装类对象的容器。

对于基本类型的数值来说，我们需要先将其转换为对应的包装类，再存入容器之中。在 Java 程序中，这个转换可以是显式，也可以是隐式的，后者正是 Java 中的自动装箱。


```

public int foo() {
    ArrayList<Integer> list = new ArrayList<>();
    list.add(0);
    int result = list.get(0);
    return result;
}

```

以上图中的 Java 代码为例。我构造了一个 Integer 类型的 ArrayList，并且向其中添加一个 int 值 0。然后，我会获取该 ArrayList 的第 0 个元素，并作为 int 值返回给调用者。这段代码对应的 Java 字节码如下所示：

```

public int foo();
Code:
    0: new java/util/ArrayList
    3: dup
    4: invokespecial java/util/ArrayList."<init>":()V
    7: astore_1
    8: aload_1
    9: iconst_0
   10: invokestatic java/lang/Integer.valueOf:(I)Ljava/lang/Integer;
   13: invokevirtual java/util/ArrayList.add:(Ljava/lang/Object;)Z
   16: pop
   17: aload_1
   18: iconst_0
   19: invokevirtual java/util/ArrayList.get:(I)Ljava/lang/Object;
   22: checkcast java/lang/Integer
   25: invokevirtual java/lang/Integer.intValue:()I
   28: istore_2
   29: iload_2
   30: ireturn

```

当向泛型参数为 Integer 的 ArrayList 添加 int 值时，便需要用到自动装箱了。在上面字节码偏移量为 10 的指令中，我们调用了 Integer.valueOf 方法，将 int 类型的值转换为 Integer 类型，再存储至容器类中。

```
public static Integer valueOf(int i) {
    if (i >= IntegerCache.low && i <= IntegerCache.high)
        return IntegerCache.cache[i + (-IntegerCache.low)];
    return new Integer(i);
}
```

这是 `Integer.valueOf` 的源代码。可以看到，当请求的 `int` 值在某个范围内时，我们会返回缓存了的 `Integer` 对象；而当所请求的 `int` 值在范围之外时，我们则会新建一个 `Integer` 对象。

在介绍反射的那一篇中，我曾经提到参数 `java.lang.Integer.IntegerCache.high`。这个参数将影响这里的 `IntegerCache.high`。

也就是说，我们可以通过配置该参数，扩大 `Integer` 缓存的范围。Java 虚拟机参数 `-XX:+AggressiveOpts` 也会将 `IntegerCache.high` 调整至 20000。

奇怪的是，Java 并不支持对 `IntegerCache.low` 的更改，也就是说，对于小于 -128 的整数，我们无法直接使用由 Java 核心类库所缓存的 `Integer` 对象。

```
25: invokevirtual java/lang/Integer.intValue:()I
```

当从泛型参数为 `Integer` 的 `ArrayList` 取出元素时，我们得到的实际上也是 `Integer` 对象。如果应用程序期待的是一个 `int` 值，那么就会发生自动拆箱。

在我们的例子中，自动拆箱对应的是字节码偏移量为 25 的指令。该指令将调用 `Integer.intValue` 方法。这是一个实例方法，直接返回 `Integer` 对象所存储的 `int` 值。

泛型与类型擦除

你可能已经留意到了，在前面例子生成的字节码中，往 `ArrayList` 中添加元素的 `add` 方法，所接受的参数类型是 `Object`；而从 `ArrayList` 中获取元素的 `get` 方法，其返回类型同样也是 `Object`。

前者还好，但是对于后者，在字节码中我们需要进行向下转换，将所返回的 `Object` 强制转换为 `Integer`，方能进行接下来的自动拆箱。

```
13: invokevirtual java/util/ArrayList.add:(Ljava/lang/Object;)Z
...
19: invokevirtual java/util/ArrayList.get:(I)Ljava/lang/Object;
22: checkcast java/lang/Integer
```

之所以会出现这种情况，是因为 Java 泛型的类型擦除。这是个什么概念呢？简单地说，那便是 Java 程序里的泛型信息，在 Java 虚拟机里全部都丢失了。这么做主要是为了兼容引入泛型之前的代码。

当然，并不是每一个泛型参数被擦除类型后都会变成 Object 类。对于限定了继承类的泛型参数，经过类型擦除后，所有的泛型参数都将变成所限定的继承类。也就是说，Java 编译器将选取该泛型所能指代的所有类中层次最高的那个，作为替换泛型的类。

```
class GenericTest<T extends Number> {  
    T foo(T t) {  
        return t;  
    }  
}
```

举个例子，在上面这段 Java 代码中，我定义了一个 T extends Number 的泛型参数。它所对应的字节码如下所示。可以看到，foo 方法的方法描述符所接收参数的类型以及返回类型都为 Number。方法描述符是 Java 虚拟机识别方法调用的目标方法的关键。

```
T foo(T);  
descriptor: (Ljava/lang/Number;)Ljava/lang/Number;  
flags: (0x0000)  
Code:  
    stack=1, locals=2, args_size=2  
    0: aload_1  
    1: areturn  
Signature: (TT;)TT;
```

不过，字节码中仍存在泛型参数的信息，如方法声明里的 T foo(T)，以及方法签名（Signature）中的“(TT;)TT;”。这类信息主要由 Java 编译器在编译他类时使用。

既然泛型会被类型擦除，那么我们还有必要用它吗？

我认为是有必要的。Java 编译器可以根据泛型参数判断程序中的语法是否正确。举例来说，尽管经过类型擦除后，ArrayList.add 方法所接收的参数是 Object 类型，但是往泛型参数为 Integer 类型的 ArrayList 中添加字符串对象，Java 编译器是会报错的。

```
ArrayList<Integer> list = new ArrayList<>();
```

```
list.add("0"); // 编译出错
```

桥接方法

泛型的类型擦除带来了不少问题。其中一个便是方法重写。在第四篇的课后实践中，我留了这么一段代码：

```
class Merchant<T extends Customer> {
    public double actionPrice(T customer) {
        return 0.0d;
    }
}

class VIPOnlyMerchant extends Merchant<VIP> {
    @Override
    public double actionPrice(VIP customer) {
        return 0.0d;
    }
}
```

VIPOnlyMerchant 中的 actionPrice 方法是符合 Java 语言的方法重写的，毕竟都使用 @Override 来注解了。然而，经过类型擦除后，父类的方法描述符为 (LCustomer;)D，而子类的方法描述符为 (LVIP;)D。这显然不符合 Java 虚拟机关于方法重写的定义。

为了保证编译而成的 Java 字节码能够保留重写的语义，Java 编译器额外添加了一个桥接方法。该桥接方法在字节码层面重写了父类的方法，并将调用子类的方法。

```
class VIPOnlyMerchant extends Merchant<VIP>
...
public double actionPrice(VIP);
    descriptor: (LVIP;)D
    flags: (0x0001) ACC_PUBLIC
    Code:
        0: dconst_0
        1: dreturn

    public double actionPrice(Customer);
        descriptor: (LCustomer;)D
```

```
flags: (0x1041) ACC_PUBLIC, ACC_BRIDGE, ACC_SYNTHETIC
```

```
Code:
```

```
0: aload_0
```

```
1: aload_1
```

```
2: checkcast class VIP
```

```
5: invokevirtual actionPrice:(LVIP;)D
```

```
8: dreturn
```

```
// 这个桥接方法等同于
```

```
public double actionPrice(Customer customer) {
    return actionPrice((VIP) customer);
}
```

在我们的例子中，VIPOnlyMerchant 类将包含一个桥接方法 actionPrice(Customer)，它重写了父类的同名同方法描述符的方法。该桥接方法将传入的 Customer 参数强制转换为 VIP 类型，再调用原本的 actionPrice(VIP) 方法。

当一个声明类型为 Merchant，实际类型为 VIPOnlyMerchant 的对象，调用 actionPrice 方法时，字节码里的符号引用指向的是 Merchant.actionPrice(Customer) 方法。Java 虚拟机将动态绑定至 VIPOnlyMerchant 类的桥接方法之中，并且调用其 actionPrice(VIP) 方法。

需要注意的是，在 javap 的输出中，该桥接方法的访问标识符除了代表桥接方法的 ACC_BRIDGE 之外，还有 ACC_SYNTHETIC。它表示该方法对于 Java 源代码来说是不可见的。当你尝试通过传入一个声明类型为 Customer 的对象作为参数，调用 VIPOnlyMerchant 类的 actionPrice 方法时，Java 编译器会报错，并且提示参数类型不匹配。

```
Customer customer = new VIP();
new VIPOnlyMerchant().actionPrice(customer); // 编译出错
```

当然，如果你实在想要调用这个桥接方法，那么你可以选择使用反射机制。

```
class Merchant {
    public Number actionPrice(Customer customer) {
        return 0;
    }
}
```

```
class NaiveMerchant extends Merchant {
    @Override
    public Double actionPrice(Customer customer) {
        return 0.0D;
    }
}
```

除了前面介绍的泛型重写会生成桥接方法之外，如果子类定义了一个与父类参数类型相同的方法，其返回类型为父类方法返回类型的子类，那么 Java 编译器也会为其生成桥接方法。

```
class NaiveMerchant extends Merchant
    public java.lang.Double actionPrice(Customer);
        descriptor: (Ljava/lang/Double;)Ljava/lang/Double;
        flags: (0x0001) ACC_PUBLIC
        Code:
            stack=2, locals=2, args_size=2
                0: dconst_0
                1: invokestatic Double.valueOf:(D)Ljava/lang/Double;
                4: areturn

    public java.lang.Number actionPrice(Customer);
        descriptor: (Ljava/lang/Number;)Ljava/lang/Number;
        flags: (0x1041) ACC_PUBLIC, ACC_BRIDGE, ACC_SYNTHETIC
        Code:
            stack=2, locals=2, args_size=2
                0: aload_0
                1: aload_1
                2: invokevirtual actionPrice:(Ljava/lang/Double;)Ljava/lang/Double;
                5: areturn
```

我之前曾提到过，class 文件里允许出现两个同名、同参数类型但是不同返回类型的方法。这里的原方法和桥接方法便是其中一个例子。由于该桥接方法同样标注了 ACC_SYNTHETIC，因此，当在 Java 程序中调用 NaiveMerchant.actionPrice 时，我们只会调用到原方法。

其他语法糖

在前面的篇章中，我已经介绍过了变长参数、try-with-resources 以及在同一 catch 代码块中捕获多

种异常等语法糖。下面我将列举另外两个常见的语法糖。

foreach 循环允许 Java 程序在 for 循环里遍历数组或者 Iterable 对象。对于数组来说，foreach 循环将从 0 开始逐一访问数组中的元素，直至数组的末尾。其等价的代码如下面所示：

```
public void foo(int[] array) {
    for (int item : array) {
    }
}

// 等同于

public void bar(int[] array) {
    int[] myArray = array;
    int length = myArray.length;
    for (int i = 0; i < length; i++) {
        int item = myArray[i];
    }
}
```

对于 Iterable 对象来说，foreach 循环将调用其 iterator 方法，并且用它的 hasNext 以及 next 方法来遍历该 Iterable 对象中的元素。其等价的代码如下面所示：

```
public void foo(ArrayList<Integer> list) {
    for (Integer item : list) {
    }
}

// 等同于

public void bar(ArrayList<Integer> list) {
    Iterator<Integer> iterator = list.iterator();
    while (iterator.hasNext()) {
        Integer item = iterator.next();
    }
}
```

字符串 switch 编译而成的字节码看起来非常复杂，但实际上就是一个哈希桶。由于每个 case 所截获的字符串都是常量值，因此，Java 编译器会将原来的字符串 switch 转换为 int 值 switch，比较所输入的字符串的哈希值。

由于字符串哈希值很容易发生碰撞，因此，我们还需要用 `String.equals` 逐个比较相同哈希值的字符串。

如果你感兴趣的话，可以自己利用 `javap` 分析字符串 `switch` 编译而成的字节码。

总结与实践

今天我主要介绍了 Java 编译器对几个语法糖的处理。

基本类型和其包装类型之间的自动转换，也就是自动装箱、自动拆箱，是通过加入 `[Wrapper].valueOf`（如 `Integer.valueOf`）以及 `[Wrapper].[primitive]Value`（如 `Integer.intValue`）方法调用来实现的。

Java 程序中的泛型信息会被擦除。具体来说，Java 编译器将选取该泛型所能指代的所有类中层次最高的那个，作为替换泛型的具体类。

由于 Java 语义与 Java 字节码中关于重写的定义并不一致，因此 Java 编译器会生成桥接方法作为适配器。此外，我还介绍了 `foreach` 循环以及字符串 `switch` 的编译。

今天的实践环节，你可以探索一下 Java 10 的 `var` 关键字，是否保存了泛型信息？是否支持自动装箱拆箱？

```
public void foo() {  
    var value = 1;  
    var list = new ArrayList<Integer>();  
    list.add(value);  
    // list.add("1"); 这一句能够编译吗?  
}
```




版权归极客邦科技所有，未经许可不得转载

精选留言



三木子

0

从实现上说可以设计一个int类型的list，而jdk中arrayList是object类型，这样做是不是为了通用型考虑呢？

2018-08-24

作者回复

对的

2018-08-24

讲堂 深入拆解 Java 虚拟机 文章详情

16 | 即时编译（上）

2018-08-27 郑雨迪



16 | 即时编译（上）

朗读人：郑雨迪 10'41" | 4.90M

在专栏的第一篇中，我曾经简单地介绍过即时编译。这是一项用来提升应用程序运行效率的技术。通常而言，代码会先被 Java 虚拟机解释执行，之后反复执行的热点代码则会被即时编译成为机器码，直接运行在底层硬件之上。

今天我们便来详细剖析一下 Java 虚拟机中的即时编译。

分层编译模式

HotSpot 虚拟机包含多个即时编译器 C1、C2 和 Graal。

其中，Graal 是一个实验性质的即时编译器，可以通过参数 `-XX:+UnlockExperimentalVMOptions -XX:+UseJVMCICompiler` 启用，并且替换 C2。

在 Java 7 以前，我们需要根据程序的特性选择对应的即时编译器。对于执行时间较短的，或者对启动性能有要求的程序，我们采用编译效率较快的 C1，对应参数 `-client`。

对于执行时间较长的，或者对峰值性能有要求的程序，我们采用生成代码执行效率较快的 C2，对应参数 `-server`。

Java 7 引入了分层编译（对应参数 `-XX:+TieredCompilation`）的概念，综合了 C1 的启动性能优势和 C2 的峰值性能优势。

分层编译将 Java 虚拟机的执行状态分为了五个层次。为了方便阐述，我用“C1 代码”来指代由 C1 生成的机器码，“C2 代码”来指代由 C2 生成的机器码。五个层级分别是：

1. 解释执行；
2. 执行不带 profling 的 C1 代码；
3. 执行仅带方法调用次数以及循环回边执行次数 profling 的 C1 代码；
4. 执行带所有 profling 的 C1 代码；
5. 执行 C2 代码。

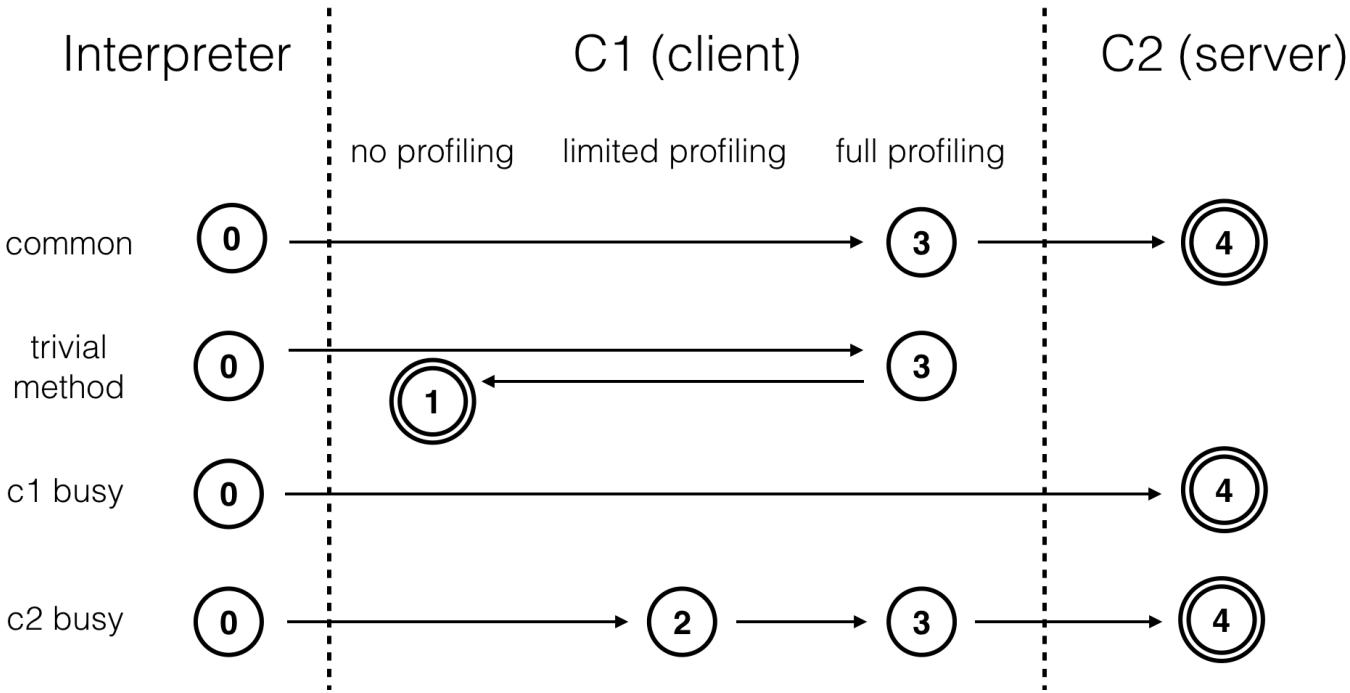
通常情况下，C2 代码的执行效率要比 C1 代码的高出 30% 以上。然而，对于 C1 代码的三种状态，按执行效率从高至低则是 1 层 > 2 层 > 3 层。

其中 1 层的性能比 2 层的稍微高一些，而 2 层的性能又比 3 层高出 30%。这是因为 profling 越多，其额外的性能开销越大。

这里解释一下，profling 是指在程序执行过程中，收集能够反映程序执行状态的数据。这里所收集的数据我们称之为程序的 profile。

你可能已经接触过许许多多的 profiler，例如 JDK 附带的 hprof。这些 profiler 大多通过注入（instrumentation）或者 JVMTI 事件来实现的。Java 虚拟机也内置了 profling。我会在下一篇中具体介绍 Java 虚拟机的 profling 都在做些什么。

在 5 个层次的执行状态中，1 层和 4 层为终止状态。当一个方法被终止状态编译过后，如果编译后的代码并没有失效，那么 Java 虚拟机是不会再次发出该方法的编译请求的。



不同的编译路径，图片来源于我之前一篇[介绍 Graal 的博客](#)。

这里我列举了 4 个不同的编译路径（[Igor 的演讲](#)列举了更多的编译路径）。通常情况下，热点方法会被 3 层的 C1 编译，然后再被 4 层的 C2 编译。

如果方法的字节码数目比较少（如 getter/setter），而且 3 层的 proPling 没有可收集的数据；那么，Java 虚拟机断定该方法对于 C1 代码和 C2 代码的执行效率相同。在这种情况下，Java 虚拟机会在 3 层编译之后，直接选择用 1 层的 C1 编译。由于这是一个终止状态，因此 Java 虚拟机不会继续用 4 层的 C2 编译。

在 C1 忙碌的情况下，Java 虚拟机在解释执行过程中对程序进行 proPling，而后直接由 4 层的 C2 编译。在 C2 忙碌的情况下，方法会被 2 层的 C1 编译，然后再被 3 层的 C1 编译，以减少方法在 3 层的执行时间。

Java 8 默认开启了分层编译。不管是开启还是关闭分层编译，原本用来选择即时编译器的参数 -client 和 -server 都是无效的。当关闭分层编译的情况下，Java 虚拟机将直接采用 C2。

如果你希望只是用 C1，那么你可以在打开分层编译的情况下使用参数 -XX:TieredStopAtLevel=1。在这种情况下，Java 虚拟机会在解释执行之后直接由 1 层的 C1 进行编译。

即时编译的触发

Java 虚拟机是根据方法的调用次数以及循环回边的执行次数来触发即时编译的。前面提到，Java 虚拟机在 0 层、2 层和 3 层执行状态时进行 proPling，其中就包含方法的调用次数和循环回边的执行次数。

这里的循环回边是一个控制流图中的概念。在字节码中，我们可以简单理解为来回跳转的指令。

(注意，这并不一定符合循环回边的定义。)

```
public static void foo(Object obj) {  
    int sum = 0;  
    for (int i = 0; i < 200; i++) {  
        sum += i;  
    }  
}
```

举例来说，上面这段代码将被编译为下面的字节码。其中，偏移量为 18 的字节码将往回跳至偏移量为 7 的字节码中。在解释执行时，每当运行一次该指令，Java 虚拟机便会将该方法的循环回边计数器加 1。

```
public static void foo(java.lang.Object);  
Code:  
    0: iconst_0  
    1: istore_1  
    2: iconst_0  
    3: istore_2  
    4: goto 14  
    7: iload_1  
    8: iload_2  
    9: iadd  
   10: istore_1  
   11: iinc 2, 1  
   14: iload_2  
   15: sipush 200  
   18: if_icmplt 7  
   21: return
```

在即时编译过程中，我们会识别循环的头部和尾部。在上面这段字节码中，循环的头部是偏移量为 14 的字节码，尾部为偏移量为 11 的字节码。

循环尾部到循环头部的控制流边就是真正意义上的循环回边。也就是说，C1 将在这个位置插入增加循环回边计数器的代码。

解释执行和 C1 代码中增加循环回边计数器的位置并不相同，但这并不会对程序造成影响。

实际上，Java 虚拟机并不会对这些计数器进行同步操作，因此收集而来执行次数也并非精确值。不管如何，即时编译的触发并不需要非常精确的数值。只要该数值足够大，就能说明对应的方法包含热点代码。

具体来说，在不启用分层编译的情况下，当方法的调用次数和循环回边的次数的和，超过由参数 `-XX:CompileThreshold` 指定的阈值时（使用 C1 时，该值为 1500；使用 C2 时，该值为 10000），便会触发即时编译。

当启用分层编译时，Java 虚拟机将不再采用由参数 `-XX:CompileThreshold` 指定的阈值（该参数失效），而是使用另一套阈值系统。在这套系统中，阈值的大小是动态调整的。

所谓的动态调整其实并不复杂：在比较阈值时，Java 虚拟机会将阈值与某个系数 s 相乘。该系数与当前待编译的方法数目成正相关，与编译线程的数目成负相关。

系数的计算方法为：

$$s = \text{queue_size_X} / (\text{TierXLoadFeedback} * \text{compiler_count_X}) + 1$$

其中 X 是执行层次，可取 3 或者 4；

`queue_size_X` 是执行层次为 X 的待编译方法的数目；

`TierXLoadFeedback` 是预设好的参数，其中 `Tier3LoadFeedback` 为 5，`Tier4LoadFeedback` 为 3；

`compiler_count_X` 是层次 X 的编译线程数目。

在 64 位 Java 虚拟机中，默认情况下编译线程的总数目是根据处理器数量来调整的（对应参数 `-XX:+CICompilerCountPerCPU`，默认为 true；当通过参数 `-XX:+CICompilerCount=N` 强制设定总编译线程数目时，`CICompilerCountPerCPU` 将被设置为 false）。

Java 虚拟机会将这些编译线程按照 1:2 的比例分配给 C1 和 C2（至少各为 1 个）。举个例子，对于一个四核机器来说，总的编译线程数目为 3，其中包含一个 C1 编译线程和两个 C2 编译线程。

对于四核及以上的机器，总的编译线程的数目为：

$$n = \log_2(N) * \log_2(\log_2(N)) * 3 / 2$$

其中 N 为 CPU 核心数目。

当启用分层编译时，即时编译具体的触发条件如下。

当方法调用次数大于由参数 `-XX:TierXInvocationThreshold` 指定的阈值乘以系数，或者当方法调用次数大于由参数 `-XX:TierXMINInvocationThreshold` 指定的阈值乘以系数，并且方法调用次数和循环回边次数之和大于由参数 `-XX:TierXCompileThreshold` 指定的阈值乘以系数时，便会触发 X 层即时编译。

触发条件为：

```
i > TierXInvocationThreshold * s || (i > TierXMinInvocationThreshold * s && i + b >
TierXCompileThreshold * s)
```

其中 i 为调用次数，b 为循环回边次数。

OSR 编译

可以看到，决定一个方法是否为热点代码的因素有两个：方法的调用次数、以及循环回边的执行次数。即时编译便是根据这两个计数器的和来触发的。为什么 Java 虚拟机需要维护两个不同的计数器呢？

实际上，除了以方法为单位的即时编译之外，Java 虚拟机还存在着另一种以循环为单位的即时编译，叫做 On-Stack-Replacement（OSR）编译。循环回边计数器便是用来触发这种类型的编译的。

OSR 实际上是一种技术，它指的是在程序执行过程中，动态地替换掉 Java 方法栈帧，从而使得程序能够在非方法入口处进行解释执行和编译后的代码之间的切换。事实上，去优化（deoptimization）采用的技术也可以称之为 OSR。

在不启用分层编译的情况下，触发 OSR 编译的阈值是由参数 `-XX:CompileThreshold` 指定的阈值的倍数。

该倍数的计算方法为：

$$(\text{OnStackReplacePercentage} - \text{InterpreterProfilePercentage}) / 100$$

其中 `-XX:InterpreterProfilePercentage` 的默认值为 33，当使用 C1 时 `-XX:OnStackReplacePercentage` 为 933，当使用 C2 时为 140。

也就是说，默认情况下，C1 的 OSR 编译的阈值为 13500，而 C2 的为 10700。

在启用分层编译的情况下，触发 OSR 编译的阈值则是由参数 `-XX:TierXBackEdgeThreshold` 指定的阈值乘以系数。

OSR 编译在正常的应用程序中并不多见。它只在基准测试时比较常见，因此并不需要过多了解。

总结与实践

今天我详细地介绍了 Java 虚拟机中的即时编译。

从 Java 8 开始，Java 虚拟机默认采用分层编译的方式。它将执行分为五个层次，分为为 0 层解释执行，1 层执行没有 proBling 的 C1 代码，2 层执行部分 proBling 的 C1 代码，3 层执行全部 proBling 的 C1 代码，和 4 层执行 C2 代码。

通常情况下，方法会首先被解释执行，然后被 3 层的 C1 编译，最后被 4 层的 C2 编译。

即时编译是由方法调用计数器和循环回边计数器触发的。在使用分层编译的情况下，触发编译的阈值是根据当前待编译的方法数目动态调整的。

OSR 是一种能够在非方法入口处进行解释执行和编译后代码之间切换的技术。OSR 编译可以用来解决单次调用方法包含热循环的性能优化问题。

今天的实践环节，你可以使用参数 `-XX:+PrintCompilation` 来打印你项目中的即时编译情况。

88	15	3	CompilationTest::foo (16 bytes)
88	16	3	java.lang.Integer::valueOf (32 bytes)
88	17	4	CompilationTest::foo (16 bytes)
88	18	4	java.lang.Integer::valueOf (32 bytes)
89	15	3	CompilationTest::foo (16 bytes) made not entrant
89	16	3	java.lang.Integer::valueOf (32 bytes) made not entrant
90	19 %	3	CompilationTest::main @ 5 (33 bytes)

简单解释一下该参数的输出：第一列是时间，第二列是 Java 虚拟机维护的编译 ID。

接下来是一系列标识，包括 %（是否 OSR 编译），s（是否 synchronized 方法），！（是否包含异常处理器），b（是否阻塞了应用线程，可了解一下参数 `-Xbatch`），n（是否为 native 方法）。再接下来则是编译层次，以及方法名。如果是 OSR 编译，那么方法名后面还会跟着 @以及循环所在的字节码。

当发生去优化时，你将看到之前出现过的编译，不过被标记了“made not entrant”。它表示该方法不能再被进入。

当 Java 虚拟机检测到所有的线程都退出该编译后的“made not entrant”时，会将该方法标记为“made zombie”，此时可以回收这块代码所占据的空间了。



深入拆解Java虚拟机

Oracle 高级研究员

手把手带你入门JVM

郑雨迪 Oracle Labs高级研究员，计算机博士



版权归极客邦科技所有，未经许可不得转载

精选留言



godtrue

3

有点费解，不过还好又明白了一些东西，小结如下：

1:即时编译-直接将Java字节码编译成机器码，运行在底层硬件之上，这么玩是为了提高代码的执行效率，通俗点就是能使代码跑的更快一些

2:即时编译的触发点是热点代码，即 即时编译仅针对热点代码来触发，热点代码是通过方法的调用次数或者回边循环的次数来标示的，这里也侧面反映出来即时编译是针对方法块的，有个疑问为什么不把所有代码都即时编译一下呢？这样程序的执行效率不是更快吗？为什么还分热点非热点呢？

3:分层编译的设计也有点奇怪，为什么要这样呢？使用最快的编译代码编译器编译的代码不是更好吗？

4:解释执行-将Java字节码一段一段的编译成机器码在底层硬件上运行，即时编译是一个相对解释执行而言的概念，它将热点代码先编译成机器码缓存起来，在解释执行字节码的时候判断出已经缓存起来了就不在编译直接获取执行就可以了

2018-08-28

作者回复

2. 即时编译是以方法为单位的。动态编译比较耗时，如果花了大量CPU资源编译出来的机器码运行不了几次，就很浪费了。

3. 机器码越快，需要的编译时间就越长。分层编译是一种折衷的方式，既能够满足部分不那么热

的代码能够在短时间内编译完成，也能满足很热的代码能够拥有最好的优化。

2018-08-30



大场镇车王

请问带所有proBling是什么意思 所有是个什么概念

2018-08-27

作者回复

下篇会讲。主要有分支跳转指令的branch prople，和类型相关指令的type prople。

2018-08-28

1



泰格杨

去优化是啥意思？

2018-08-27

作者回复

就是从执行机器码切换回解释执行，下一篇会详细介绍。

2018-08-27

1



code-artist

“前面提到，Java 虚拟机在 0 层、2 层和 3 层执行状态时进行 proBling”

第0层是解释执行，前面没说带proBling啊？ 前面只列出第2，3层带proBling. 这里会不会笔误？

2018-08-30

作者回复

多谢指出！

解释执行是带proBling的。最开始列2 3层时明确标注带proBling，是为了区分1，2，3层。

2018-08-30

0



Desire

请问打印出！表示什么意思 n表示native

2018-08-28

作者回复

是否包含异常处理器

2018-08-28

0



魏春河

我记得前面提到过CompileThreshold 默认是15，今天怎么成1500了？

2018-08-28

作者回复

15应该是反射调用从本地实现切换为动态实现吧。这里的1500是即时编译的。

2018-08-28

0



杨春鹏

Java中的即时编译器，是不是重新将java代码编译成机器码，并没有使用class文件？

2018-08-28

0

作者回复

确切地说，即时编译器将加载进VM中的类的某个方法所对应的byte数组编译为机器码。你也可以理解为从class文件到机器码。

2018-08-28



郑杰

0

能说下及时编译的结果是什么样的吗，然后他是怎么跟解释执行一起运行的

2018-08-27

作者回复

编译结果就是二进制码，下一篇有具体示例。解释执行遇到方法调用时，JVM会判断目标方法是否被编译。如果是，就跳转至编译好的二进制码中。

2018-08-27



公号-Java大后端

0

请问即时编译的对象是方法级还是语句块级？

2018-08-27

作者回复

一般都是方法级。

OSR编译会拿热循环作为编译对象。

2018-08-27

讲堂 深入拆解 Java 虚拟机 文章详情

17 | 即时编译（下）

2018-08-29 郑雨迪



17 | 即时编译（下）

朗读人：郑雨迪 10'37" | 4.87M

今天我们来继续讲解 Java 虚拟机中的即时编译。

Profiling

上篇提到，分层编译中的 0 层、2 层和 3 层都会进行 profiling，收集能够反映程序执行状态的数据。其中，最为基础的便是方法的调用次数以及循环回边的执行次数。它们被用于触发即时编译。

此外，0 层和 3 层还会收集用于 4 层 C2 编译的数据，比如说分支跳转字节码的分支 profile (branch profile)，包括跳转次数和不跳转次数，以及非私有实例方法调用指令、强制类型转换 checkcast 指令、类型测试 instanceof 指令，和引用类型的数组存储 astore 指令的类型 profile (receiver type profile)。

分支 profile 和类型 profile 的收集将给应用程序带来不少的性能开销。据统计，正是因为这部分额外的 profiling，使得 3 层 C1 代码的性能比 2 层 C1 代码的低 30%。

在通常情况下，我们不会在解释执行过程中收集分支 profile 以及类型 profile。只有在方法触发 C1

编译后，Java 虚拟机认为该方法有可能被 C2 编译，方才在该方法的 C1 代码中收集这些 profile。

只要在比较极端的情况下，例如等待 C1 编译的方法数目太多时，Java 虚拟机才会开始在解释执行过程中收集这些 profile。

那么这些耗费巨大代价收集而来的 profile 具体有什么作用呢？

答案是，C2 可以根据收集得到的数据进行猜测，假设接下来的执行同样会按照所收集的 profile 进行，从而作出比较激进的优化。

基于分支 profile 的优化

举个例子，下面这段代码中包含两个条件判断。第一个条件判断将测试所输入的 boolean 值。

如果为 true，则将局部变量 v 设置为所输入的 int 值。如果为 false，则将所输入的 int 值经过一番运算之后，再存入局部变量 v 之中。

第二个条件判断则测试局部变量 v 是否和所输入的 int 值相等。如果相等，则返回 0。如果不等，则将局部变量 v 经过一番运算之后，再将之返回。显然，当所输入的 boolean 值为 true 的情况下，这段代码将返回 0。

```
public static int foo(boolean f, int in) {
    int v;
    if (f) {
        v = in;
    } else {
        v = (int) Math.sin(in);
    }

    if (v == in) {
        return 0;
    } else {
        return (int) Math.cos(v);
    }
}
```

// 编译而成的字节码：

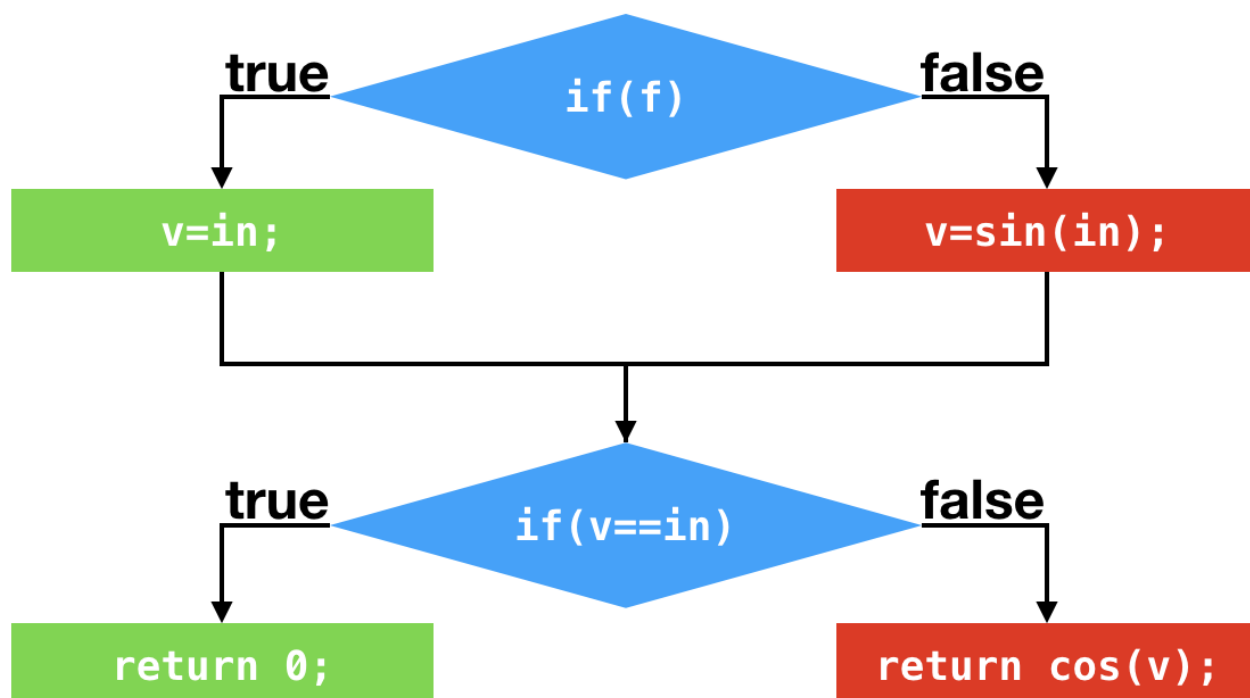
```
public static int foo(boolean, int);

Code:
    0: iload_0
    1: ifeq          9
```

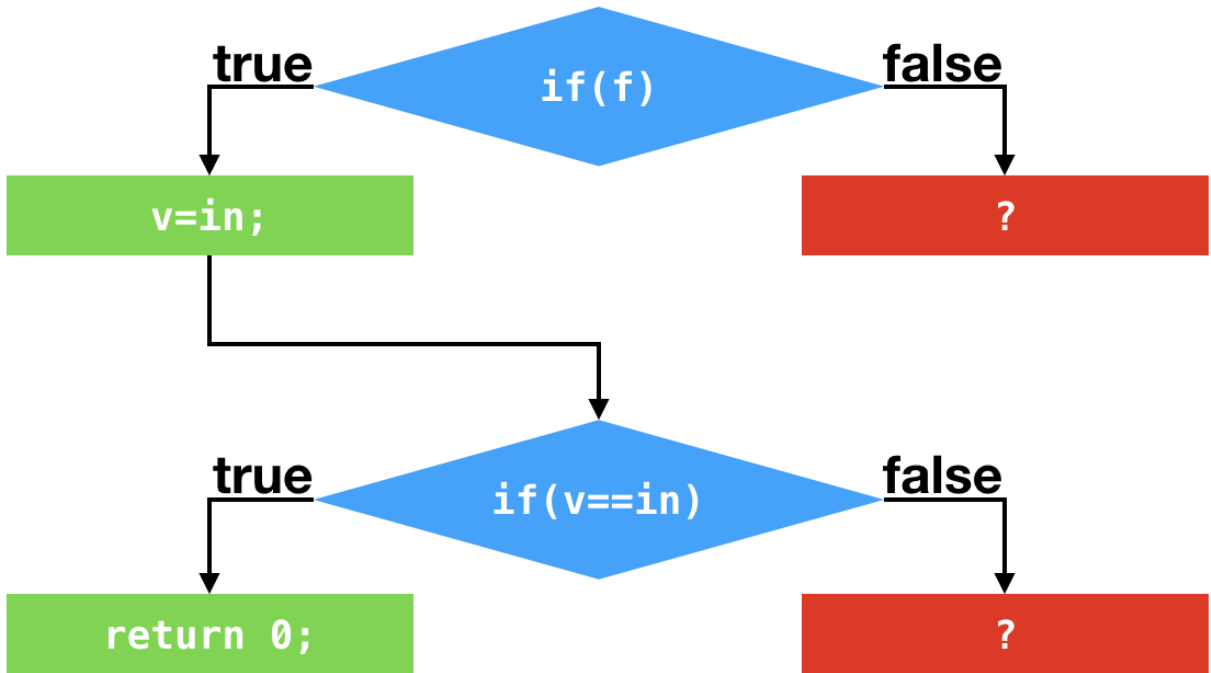
```

4: iload_1
5: istore_2
6: goto      16
9: iload_1
10: i2d
11: invokestatic  java/lang/Math.sin:(D)D
14: d2i
15: istore_2
16: iload_2
17: iload_1
18: if_icmpne    23
21: iconst_0
22: ireturn
23: iload_2
24: i2d
25: invokestatic  java/lang/Math.cos:(D)D
28: d2i
29: ireturn

```

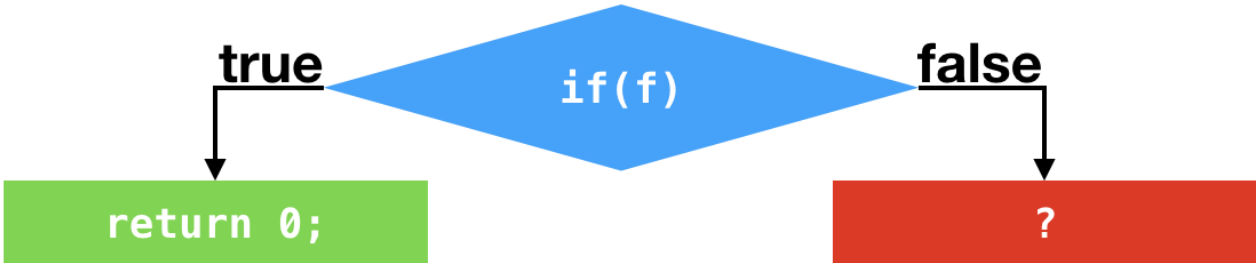


假设应用程序调用该方法时，所传入的 boolean 值皆为 true。那么，偏移量为 1 以及偏移量为 18 的条件跳转指令所对应的分支 prople 中，跳转的次数都为 0。



C2 可以根据这两个分支 profile 作出假设，在接下来的执行过程中，这两个条件跳转指令仍旧不会发生跳转。基于这个假设，C2 便不再编译这两个条件跳转语句所对应的 false 分支了。

我们暂且不管当假设错误的时候会发生什么，先来看一看剩下来的代码。经过“剪枝”之后，在第二个条件跳转处，v 的值只可能为所输入的 int 值。因此，该条件跳转可以进一步被优化掉。最终的结果是，在第一个条件跳转之后，C2 代码将直接返回 0。



这里我打印了 C2 的编译结果。可以看到，在地址为 2cee 的指令处进行过一次比较之后，该机器码便直接返回 0。

```
Compiled method (c2)      95   16      4      CompilationTest::foo (30 bytes)
...
CompilationTest.foo [0x0000000104fb2ce0, 0x0000000104fb2d38] 88 bytes
[Entry Point]
[Verified Entry Point]
[Constants]
# {method} {0x000000012629e380} 'foo' '(ZI)I' in 'CompilationTest'
```

```
# parm0:    rsi        = boolean
# parm1:    rdx        = int
#           [sp+0x30]   (sp of caller)
0x00000000104fb2ce0: mov     DWORD PTR [rsp-0x14000],eax
0x00000000104fb2ce7: push    rbp
0x00000000104fb2ce8: sub     rsp,0x20
0x00000000104fb2cec: test    esi,esi
0x00000000104fb2cee: je      0x00000000104fb2cfe // 跳转至?
0x00000000104fb2cf0: xor     eax,eax                // 将返回值设置为 0
0x00000000104fb2cf2: add     rsp,0x20
0x00000000104fb2cf6: pop     rbp
0x00000000104fb2cf7: test    DWORD PTR [rip+0xffffffffca32303],eax // safepoint
0x00000000104fb2cfd: ret
...

```

总结一下，根据条件跳转指令的分支 `prople`，即时编译器可以将从未执行过的分支剪掉，以避免编译这些很有可能不会用到的代码，从而节省编译时间以及部署代码所要消耗的内存空间。此外，“剪枝”将精简程序的数据流，从而触发更多的优化。

在现实中，分支 `prople` 出现仅跳转或者仅不跳转的情况并不多见。当然，即时编译器对分支 `prople` 的利用也不仅限于“剪枝”。它还会根据分支 `prople`，计算每一条程序执行路径的概率，以便某些编译器优化优先处理概率较高的路径。

基于类型 `profile` 的优化

另外一个例子则是关于 `instanceof` 以及方法调用的类型 `prople`。下面这段代码将测试所传入的对象是否为 `Exception` 的实例，如果是，则返回它的系统哈希值；如果不是，则返回它的哈希值。

```
public static int hash(Object in) {
    if (in instanceof Exception) {
        return System.identityHashCode(in);
    } else {
        return in.hashCode();
    }
}

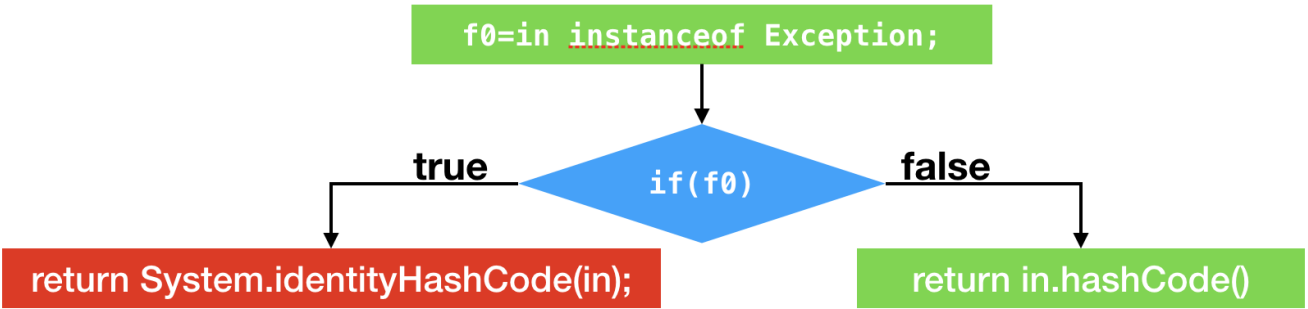
// 编译而成的字节码:
public static int hash(java.lang.Object);
Code:

```



```
0: aload_0
1: instanceof java/lang/Exception
4: ifeq      12
7: aload_0
8: invokestatic java/lang/System.identityHashCode:(Ljava/lang/Object;)I
11: ireturn
12: aload_0
13: invokevirtual java/lang/Object.hashCode:()I
16: ireturn
```

假设应用程序调用该方法时，所传入的 Object 皆为 Integer 实例。那么，偏移量为 1 的 instanceof 指令的类型 profile 仅包含 Integer，偏移量为 4 的分支跳转语句的分支 profile 中不跳转的次数为 0，偏移量为 13 的方法调用指令的类型 profile 仅包含 Integer。



在 Java 虚拟机中，instanceof 测试并不简单。如果 instanceof 的目标类型是 Pnal 类型，那么 Java 虚拟机仅需比较测试对象的动态类型是否为该 Pnal 类型。

在讲解对象的内存分布那一篇中，我曾经提到过，对象头存有该对象的动态类型。因此，获取对象的动态类型仅为单一的内存读指令。

如果目标类型不是 Pnal 类型，比如说我们例子中的 Exception，那么 Java 虚拟机需要从测试对象的动态类型开始，依次测试该类，该类的父类、祖先类，该类所直接实现或者间接实现的接口是否与目标类型一致。

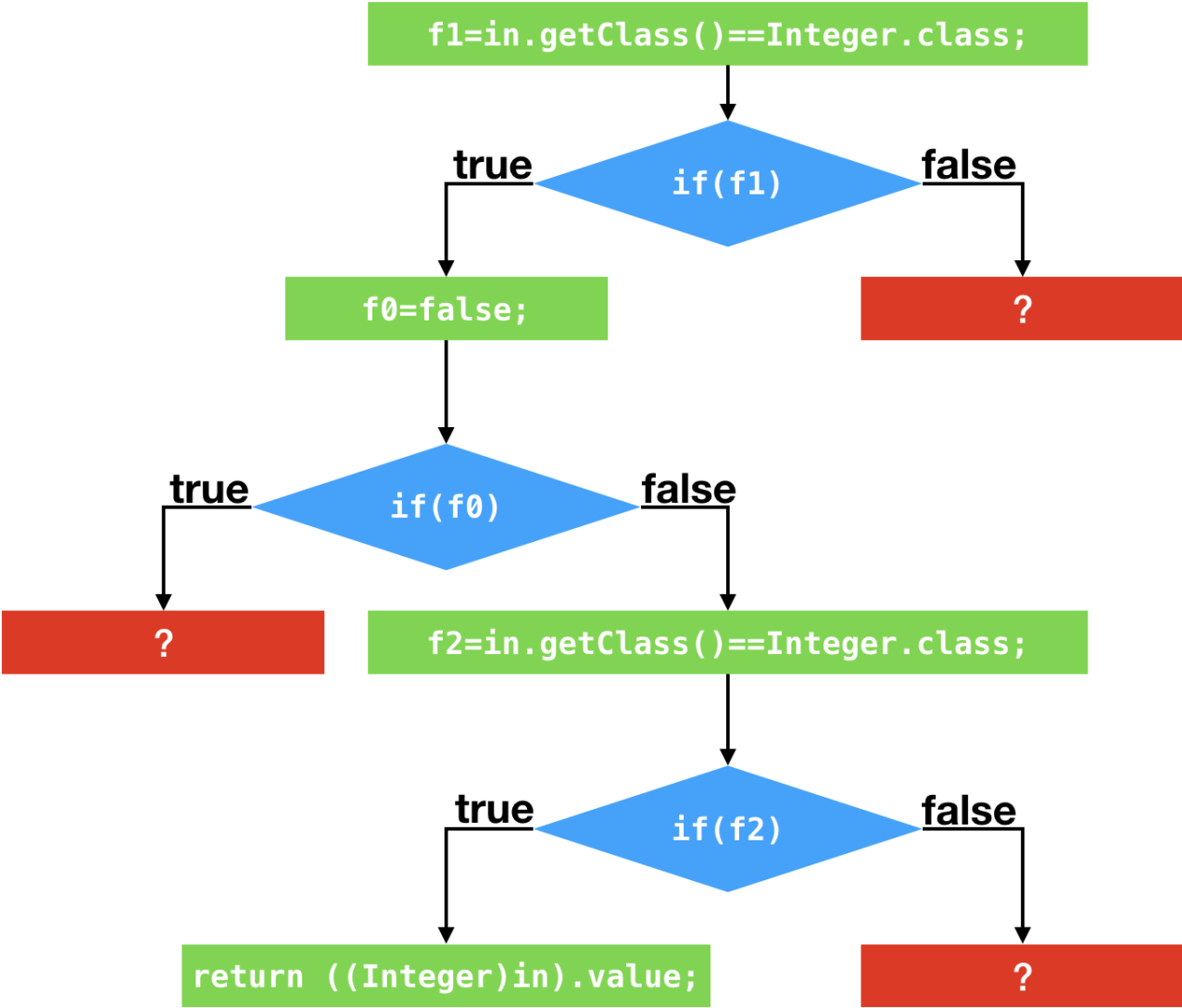
不过，在我们的例子中，instanceof 指令的类型 profile 仅包含 Integer。根据这个信息，即时编译器可以假设，在接下来的执行过程中，所输入的 Object 对象仍为 Integer 实例。

因此，生成的代码将测试所输入的对象动态类型是否为 Integer。如果是的话，则继续执行接下来的代码。（该优化源自 Graal，采用 C2 可能无法复现。）

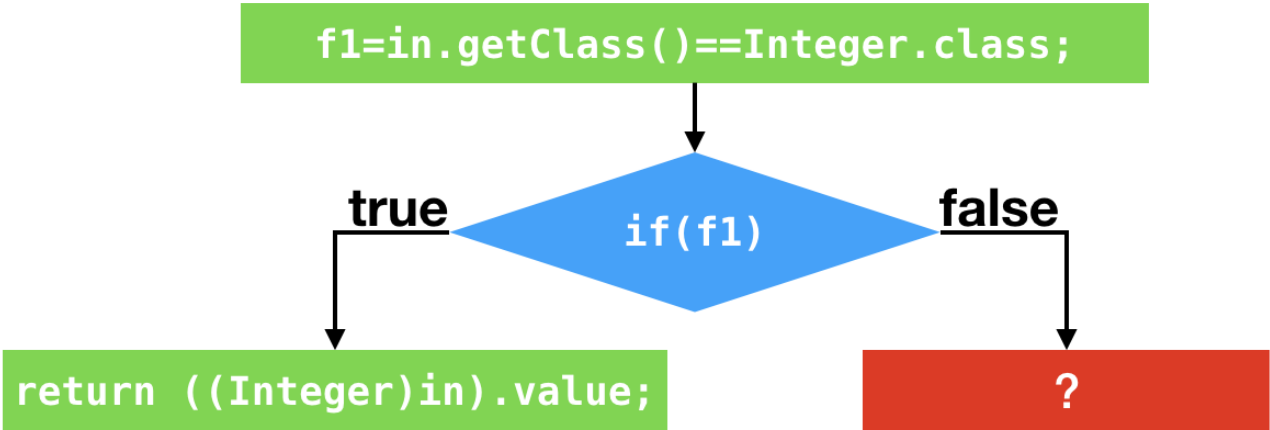
然后，即时编译器会采用和第一个例子中一致的针对分支 profile 的优化，以及对方法调用的条件去虚化内联。

我会在接下来的篇章中详细介绍内联，这里先说结果：生成的代码将测试所输入的对象动态类型是否为 Integer。如果是的话，则执行 Integer.hashCode() 方法的实质内容，也就是返回该 Integer 实例的 value 字段。

```
public final class Integer ... {  
    ...  
    @Override  
    public int hashCode() {  
        return Integer.hashCode(value);  
    }  
  
    public static int hashCode(int value) {  
        return value;  
    }  
    ...  
}
```



和第一个例子一样，根据数据流分析，上述代码可以最终优化为极其简单的形式。



这里我打印了 Graal 的编译结果。可以看到，在地址为 1ab7 的指令处进行过一次比较之后，该机器码便直接返回所传入的 Integer 对象的 value 字段。

```
Compiled method (JVMCI)      600   23    4
...
```

```
-----

CompilationTest.hash (CompilationTest.hash(Object)) [0x000000011d811aa0, 0x000000011d811b00] 96
bytes
[Entry Point]
[Verified Entry Point]
[Constants]

# {method} {0x00000001157053c8} 'hash' '(Ljava/lang/Object;)I' in 'CompilationTest'
# parm0:    rsi:rsi    = 'java/lang/Object'
#          [sp+0x20] (sp of caller)

0x000000011d811aa0: mov     DWORD PTR [rsp-0x14000],eax
0x000000011d811aa7: sub     rsp,0x18
0x000000011d811aab: mov     QWORD PTR [rsp+0x10],rbp

// 比较 [rsi+0x8], 也就是所传入的 Object 参数的动态类型, 是否为 Integer。这里 0xf80022ad 是
Integer 类的内存地址。

0x000000011d811ab0: cmp     DWORD PTR [rsi+0x8],0xf80022ad

// 如果不是, 跳转至?

0x000000011d811ab7: jne     0x000000011d811ad3

// 加载 Integer.value。在启用压缩指针时, 该字段的偏移量为 12, 也就是 0xc

0x000000011d811abd: mov     eax,DWORD PTR [rsi+0xc]
0x000000011d811ac0: mov     rbp,QWORD PTR [rsp+0x10]
0x000000011d811ac5: add     rsp,0x18
0x000000011d811ac9: test    DWORD PTR [rip+0xfffffffff272f537],eax
0x000000011d811acf: vzeroupper
0x000000011d811ad2: ret
```

和基于分支 prople 的优化一样，基于类型 prople 的优化同样也是作出假设，从而精简控制流以及数据流。这两者的核心都是假设。

对于分支 prople，即时编译器假设的是仅执行某一支；对于类型 prople，即时编译器假设的是对象的动态类型仅为类型 prople 中的那几个。

那么，当假设失败的情况下，程序将何去何从？我们继续往下看。

去优化

Java 虚拟机给出的解决方案便是去优化，即从执行即时编译生成的机器码切换回解释执行。

在生成的机器码中，即时编译器将在假设失败的位置上插入一个陷阱（trap）。该陷阱实际上是一条 call 指令，调用至 Java 虚拟机里专门负责去优化的方法。与普通的 call 指令不一样的是，去优

化方法将更改栈上的返回地址，并不再返回即时编译器生成的机器码中。

在上面的程序控制流图中，我画了很多红色方框的问号。这些问号便代表着一个个的陷阱。一旦踏入这些陷阱，便将发生去优化，并切换至解释执行。

去优化的过程相当复杂。由于即时编译器采用了许多优化方式，其生成的代码和原本的字节码的差异非常之大。

在去优化的过程中，需要将当前机器码的执行状态转换至某一字节码之前的执行状态，并从该字节码开始执行。这便要求即时编译器在编译过程中记录好这两种执行状态的映射。

举例来说，经过逃逸分析之后，机器码可能并没有实际分配对象，而是在各个寄存器中存储该对象的各个字段（标量替换，具体我会在之后的篇章中进行介绍）。在去优化过程中，Java 虚拟机需要还原出这个对象，以便解释执行时能够使用该对象。

当根据映射关系创建好对应的解释执行栈帧后，Java 虚拟机便会采用 OSR 技术，动态替换栈上的内容，并在目标字节码处开始解释执行。

此外，在调用 Java 虚拟机的去优化方法时，即时编译器生成的机器码可以根据产生去优化的原因来决定是否保留这一份机器码，以及何时重新编译对应的 Java 方法。

如果去优化的原因与优化无关，即使重新编译也不会改变生成的机器码，那么生成的机器码可以在调用去优化方法时传入 `Action_None`，表示保留这一份机器码，在下一次调用该方法时重新进入这一份机器码。

如果去优化的原因与静态分析的结果有关，例如类层次分析，那么生成的机器码可以在调用去优化方法时传入 `Action_Recompile`，表示不保留这一份机器码，但是可以不经过程序的 `prople`，直接重新编译。

如果去优化的原因与基于 `prople` 的激进优化有关，那么生成的机器码需要在调用去优化方法时传入 `Action_Reinterpret`，表示不保留这一份机器码，而且需要重新收集程序的 `prople`。

这是因为基于 `prople` 的优化失败的时候，往往代表这程序的执行状态发生改变，因此需要更正已收集的 `prople`，以更好地反映新的程序执行状态。

总结与实践

今天我介绍了 Java 虚拟机的 `proplng` 以及基于所收集的数据的优化和去优化。

通常情况下，解释执行过程中仅收集方法的调用次数以及循环回边的执行次数。

当方法被 3 层 C1 所编译时，生成的 C1 代码将收集条件跳转指令的分支 `prople`，以及类型相关指令的类型 `prople`。在部分极端情况下，Java 虚拟机也会在解释执行过程中收集这些 `prople`。

基于分支 prople 的优化以及基于类型 prople 的优化都将对程序今后的执行作出假设。这些假设将精简所要编译的代码的控制流以及数据流。在假设失败的情况下，Java 虚拟机将采取去优化，退回至解释执行并重新收集相关的 prople。

今天的实践环节，你可以使用参数

```
-XX:CompileCommand='print,*ClassName.methodName'
```

来打印程序运行过程中即时编译器生成的机器码。官方的 JDK 可能不包含反汇编器动态链接库，如 hsdls-amd64.dylib。你可能需要另外下载。

```
// java -XX:CompileCommand='print,CompilationTest.foo' CompilationTest.java -
XX:CompileCommand='print,CompilationTest.foo' CompilationTest
public class CompilationTest {
    public static int foo(boolean f, int in) {
        int v;
        if (f) {
            v = in;
        } else {
            v = (int) Math.sin(in);
        }
        if (v == in) {
            return 0;
        } else {
            return (int) Math.cos(v);
        }
    }
    public static void main(String[] args) throws InterruptedException {
        for (int i = 0; i < 500000; i++) {
            foo(true, 2);
        }
        Thread.sleep(2000);
    }
}
```

```
// java -XX:+UnlockExperimentalVMOptions -XX:+UseJVMCICompiler -
```

```
XX:CompileCommand='print,CompilationTest2.hash' CompilationTest2

public class CompilationTest2 {

    public static int hash(Object input) {

        if (input instanceof Exception) {

            return System.identityHashCode(input);

        } else {

            return input.hashCode();

        }

    }

    public static void main(String[] args) throws InterruptedException {

        for (int i = 0; i < 500000; i++) {

            hash(i);

        }

        Thread.sleep(2000);

    }

}
```



版权归极客邦科技所有，未经许可不得转载

精选留言



徐志毅

雨迪老师，请问有什么方式可以跟踪JMM里主内存与工作内存的交互，如什么时候加载到工作内存、什么时候同步到主内存～盼回复，谢谢

1

2018-08-29

作者回复

JMM的工作内存是个虚拟概念，映射到我们的体系架构就是CPU缓存。

要追踪缓存更新事件的话，估计得看perf，VTune等支持CPU performance counter的工具啦

2018-08-30



jimi

1

那么有什么规则，可以让代码尽可能被优化？

2018-08-29



godtrue

0

这节是即时编译器的有关优化、去优化、以及何时优化和为什么去优化的内容。可能比较底层，看不见摸不着，一句两句也说不清楚，所以，有点晦涩。

不过经过反复看有点感觉了，小结如下：

1:prople-是收集运行时状态信息，用于编译器优化，当然，收集信息也是耗性能的，所以，也是有前提条件的，当存在优化的可能性时才去费劲吧啦的收集相关信息

2:本节介绍的两种优化的方式思路，都是采用取巧少做事情的方式实现，是建立在假设有些事情不需要做的前提下采用的优化措施

3:如果假设失败，那就去优化呗！还用原来的方式老老实实的解释执行就完了

上述思路是理解了，不过具体实现还是蒙蒙的，应该是水平有限理解不到位吧！整体还是有收获的，所以，非常感谢！

2018-09-02



茶底

0

大佬我已经把graal拿到手了。目前用了下gu下语言挺好使的。但是这个怎么编译啊。。。

2018-08-30

作者回复

Java 10自带Graal编译器。Oracle OTN下的GraalVM也是编译好的版本，无需另外编译。

如果想要改Graal源代码后编译，可以参考github.com/oracle/graal/tree/master/compiler/README.md

2018-08-30



三木子

0

感觉好难，我想问下在现实情况下通过编译器调优的情况多吗？

2018-08-30

作者回复

调优的情况不多的，能做的即时编译器都帮你做了

2018-08-30

讲堂 深入拆解 Java 虚拟机 文章详情

18 | 即时编译器的中间表达形式

2018-08-31 郑雨迪



18 | 即时编译器的中间表达形式

朗读人：郑雨迪 10'40" | 4.90M

在上一章中，我利用了程序控制流图以及伪代码，来展示即时编译器中基于 prople 的优化。不过，这并非实际的优化过程。

1. 中间表达形式（IR）

在编译原理课程中，我们通常将编译器分为前端和后端。其中，前端会对所输入的程序进行词法分析、语法分析、语义分析，然后生成中间表达形式，也就是 IR（Intermediate Representation）。后端会对 IR 进行优化，然后生成目标代码。

如果不考虑解释执行的话，从 Java 源代码到最终的机器码实际上经过了两轮编译：Java 编译器将 Java 源代码编译成 Java 字节码，而即时编译器则将 Java 字节码编译成机器码。

对于即时编译器来说，所输入的 Java 字节码剥离了很多高级的 Java 语法，而且其采用的基于栈的计算模型非常容易建模。因此，即时编译器并不需要重新进行词法分析、语法分析以及语义分析，而是直接将 Java 字节码作为一种 IR。

不过，Java 字节码本身并不适合直接作为可供优化的 IR。这是因为现代编译器一般采用静态单赋值（Static Single Assignment, SSA）IR。这种 IR 的特点是每个变量只能被赋值一次，而且只有当变量被赋值之后才能使用。

```
y = 1;
y = 2;
x = y;
```

举个例子（[来源](#)），上面这段代码所对应的 SSA 形式伪代码是下面这段：

```
y1 = 1;
y2 = 2;
x1 = y2;
```

在源代码中，我们可以轻易地发现第一个对 y 的赋值是冗余的，但是编译器不能。传统的编译器需要借助数据流分析（具体的优化叫[reaching definition](#)），从后至前依次确认哪些变量的值被覆盖（kill）掉。

不过，如果借助了 SSA IR，编译器则可以通过查找赋值了但是没有使用的变量，来识别冗余赋值。

除此之外，SSA IR 对其他优化方式也有很大的帮助，例如常量折叠（constant folding）、常量传播（constant propagation）、强度削减（strength reduction）以及死代码删除（dead code elimination）等。

```
示例:
x1=4*1024 经过常量折叠后变为 x1=4096
x1=4; y1=x1 经过常量传播后变为 x1=4; y1=4
y1=x1*3 经过强度削减后变为 y1=(x1<<1)+x1
if(2>1){y1=1;}else{y2=1;}经过死代码删除后变为 y1=1
```

部分同学可能会手动进行上述优化，以期望能够达到更高的运行效率。实际上，对于这些简单的优化，编译器会代为执行，以便程序员专注于代码的可读性。

SSA IR 会带来一个问题，那便是不同执行路径可能会对同一变量设置不同的值。例如下面这段代码 if 语句的两个分支中，变量 y 分别被赋值为 0 或 1，并且在接下来的代码中读取 y 的值。此时，根据不同的执行路径，所读取到的值也很有可能不同。

```
x = ..;  
if (x > 0) {  
    y = 0;  
} else {  
    y = 1;  
}  
x = y;
```

为了解决这个问题，我们需要引入一个 Phi 函数的概念，能够根据不同的执行路径选择不同的值。于是，上面这段代码便可以转换为下面这段 SSA 伪代码。这里的 Phi 函数将根据前面两个分支分别选择 y1、y2 的值，并赋值给 y3。

```
x1 = ..;  
if (x1 > 0) {  
    y1 = 0;  
} else {  
    y2 = 1;  
}  
y3 = Phi(y1, y2);  
x2 = y3;
```

总之，即时编译器会将 Java 字节码转换成 SSA IR。更确切的说，是一张包含控制流和数据流的 IR 图，每个字节码对应其中的若干个节点（注意，有些字节码并没有对应的 IR 节点）。然后，即时编译器在 IR 图上面进行优化。

我们可以将每一种优化看成一个独立的图算法，它接收一个 IR 图，并输出经过转换后的 IR 图。整个编译器优化过程便是一个个优化串联起来的。

2. Sea-of-nodes

HotSpot 里的 C2 采用的是一种名为 Sea-of-Nodes 的 SSA IR。它的最大特点，便是去除了变量的概念，直接采用变量所指向的值，来进行运算。

在上面这段 SSA 伪代码中，我们使用了多个变量名 x1、x2、y1 和 y2。这在 Sea-of-Nodes 将不复存在。

取而代之的则是对应的值，比如说 Phi(y1, y2) 变成 Phi(0, 1)，后者本身也是一个值，被其他 IR 节点所依赖。正因如此，常量传播在 Sea-of-Nodes 中变成了一个 no-op。

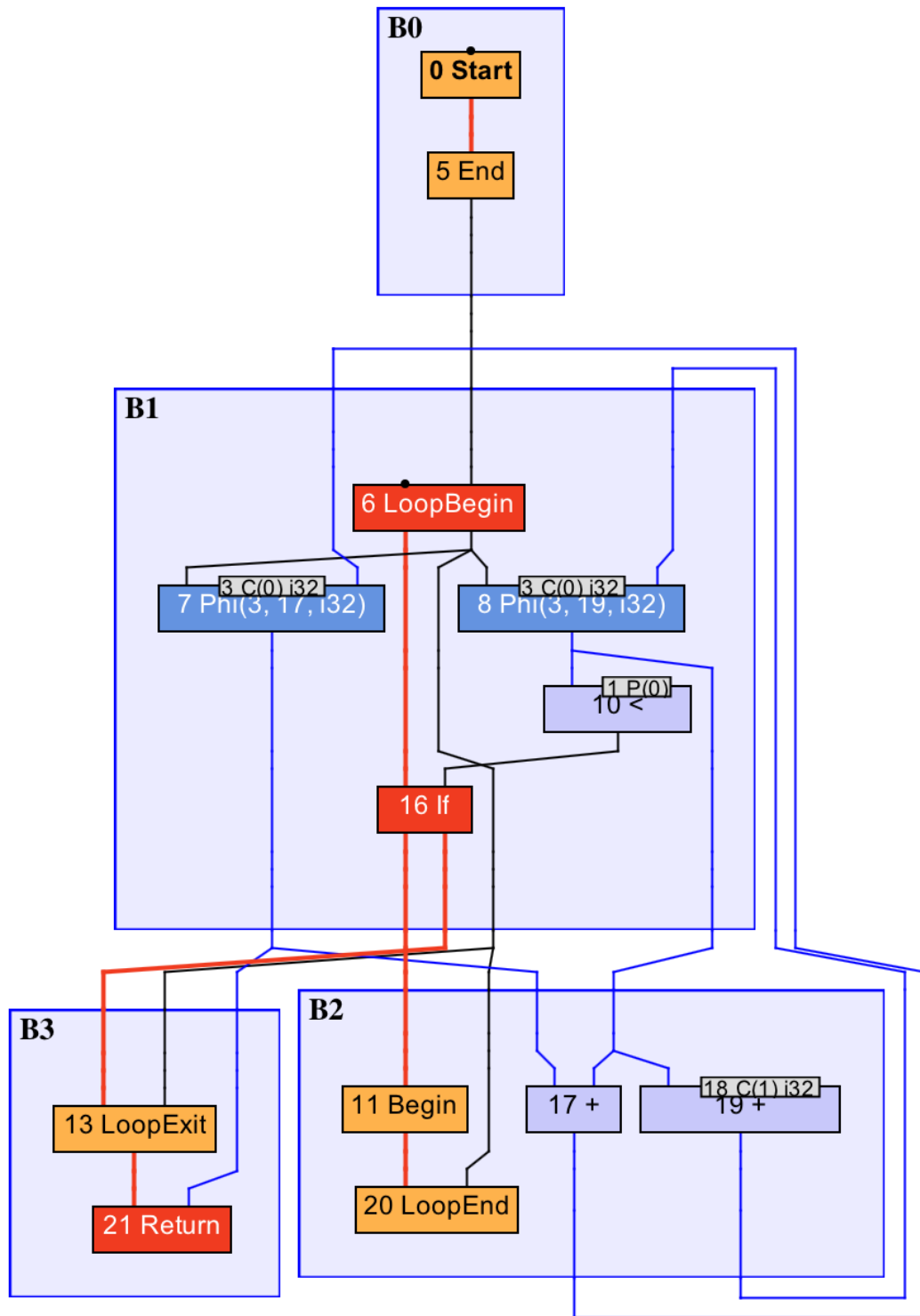
Graal 的 IR 同样也是 Sea-of-Nodes 类型的，并且可以认为是 C2 IR 的精简版本。由于 Graal 的 IR 系统更加容易理解，而且工具支持相对来说也比较全、比较新，所以下面我将围绕着 Graal 的 IR 系统来讲解。

尽管 IR 系统不同，C2 和 Graal 所实现的优化大同小异。对于那小部分不同的地方，它们也在不停地相互“借鉴”。所以你无须担心不通用的问题。

为了方便你理解今天的内容，我将利用 IR 可视化工具[Ideal Graph Visualizer](#) (IGV)，来展示具体的 IR 图。（这里 Ideal 是 C2 中 IR 的名字。）

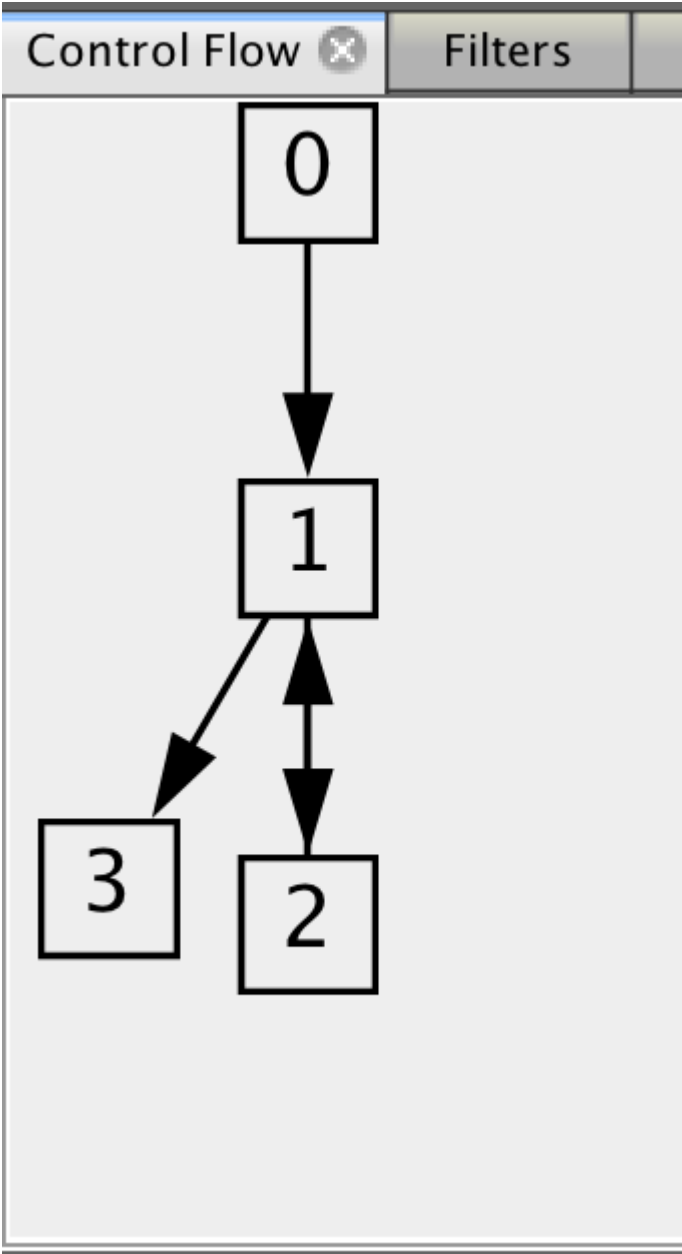
```
public static int foo(int count) {  
    int sum = 0;  
    for (int i = 0; i < count; i++) {  
        sum += i;  
    }  
    return sum;  
}
```

上面这段代码所对应的 IR 图如下所示：



IR 图

这里面，0 号 Start 节点是方法入口，21 号 Return 节点是方法出口。红色加粗线条为控制流，蓝色线条为数据流，而其他颜色的线条则是特殊的控制流或数据流。被控制流边所连接的是固定节点，其他的皆属于浮动节点。若干个顺序执行的节点将被包含在同一个基本块之中，如图中的 B0、B1 等。



基本块直接的控制流关系

基本块是仅有一个入口和一个出口的指令序列（IR 节点序列）。一个基本块的出口可以和若干个基本块的入口相连接，反之亦然。

在我们的例子中，B0 和 B2 的出口与 B1 的入口连接，代表在执行完 B0 或 B2 后可以跳转至 B1，并继续执行 B1 中的内容。而 B1 的出口则与 B2 和 B3 的入口连接。

可以看到，上面的 IR 图已经没有 sum 或者 i 这样的变量名了，取而代之的是一个一个的值，例如源程序中的 `i < count` 被转换为 10 号 < 节点，其接收两个值，分别为代表 i 的 8 号 Phi 节点，以及代表输入第 0 个参数的 1 号 P(0) 节点。

关于 8 号 Phi 节点，前面讲过，它将根据不同的执行路径选择不同的值。如果是从 5 号 End 节点进入的，则选择常量 0；如果是从 20 号 LoopEnd 节点跳转进入的，则选择 19 号 + 节点。

你可以自己分析一下代表 sum 的 7 号 Phi 节点，根据不同的执行路径都选择了哪些值。

浮动节点的位置并不固定。在编译过程中，编译器需要（多次）计算浮动节点具体的排布位置。这个过程我们称之为节点调度（node scheduling）。

节点调度是根据节点之间的依赖关系来进行的。举个例子，在前面的 IR 图中，10 号 < 节点是 16 号 if 节点用来判断是否跳转的条件，因此它需要排布在 16 号 if 节点（注意这是一个固定节点）之前。同时它又依赖于 8 号 Phi 节点的值以及 1 号 P(0) 节点的值，因此它需要排布在这两个节点之后。

需要注意的是，C2 没有固定节点这一概念，所有的 IR 节点都是浮动节点。它将根据各个基本块头尾之间的控制依赖，以及数据依赖和内存依赖，来进行节点调度。

这里的内存依赖是什么一个概念呢？假设一段程序往内存中存储了一个值，而后又读取同一内存，那么显然程序希望读取到的是所存储的值。即时编译器不能任意调度对同一内存地址的读写，因为它们之间存在依赖关系。

C2 的做法便是将这种时序上的先后记录为内存依赖，并让节点调度算法在进行调度时考虑这些内存依赖关系。Graal 则将内存读写转换成固定节点。由于固定节点存在先后关系，因此无须额外记录内存依赖。

3. Gloval Value Numbering

下面介绍一种因 Sea-of-Nodes 而变得非常容易的优化技术 —— Gloval Value Numbering (GVN) 。

GVN 是一种发现并消除等价计算的优化技术。举例来说，如果一段程序中出现了多次操作数相同的乘法，那么即时编译器可以将这些乘法并为一个，从而降低输出机器码的大小。如果这些乘法出现在同一执行路径上，那么 GVN 还将省下冗余的乘法操作。

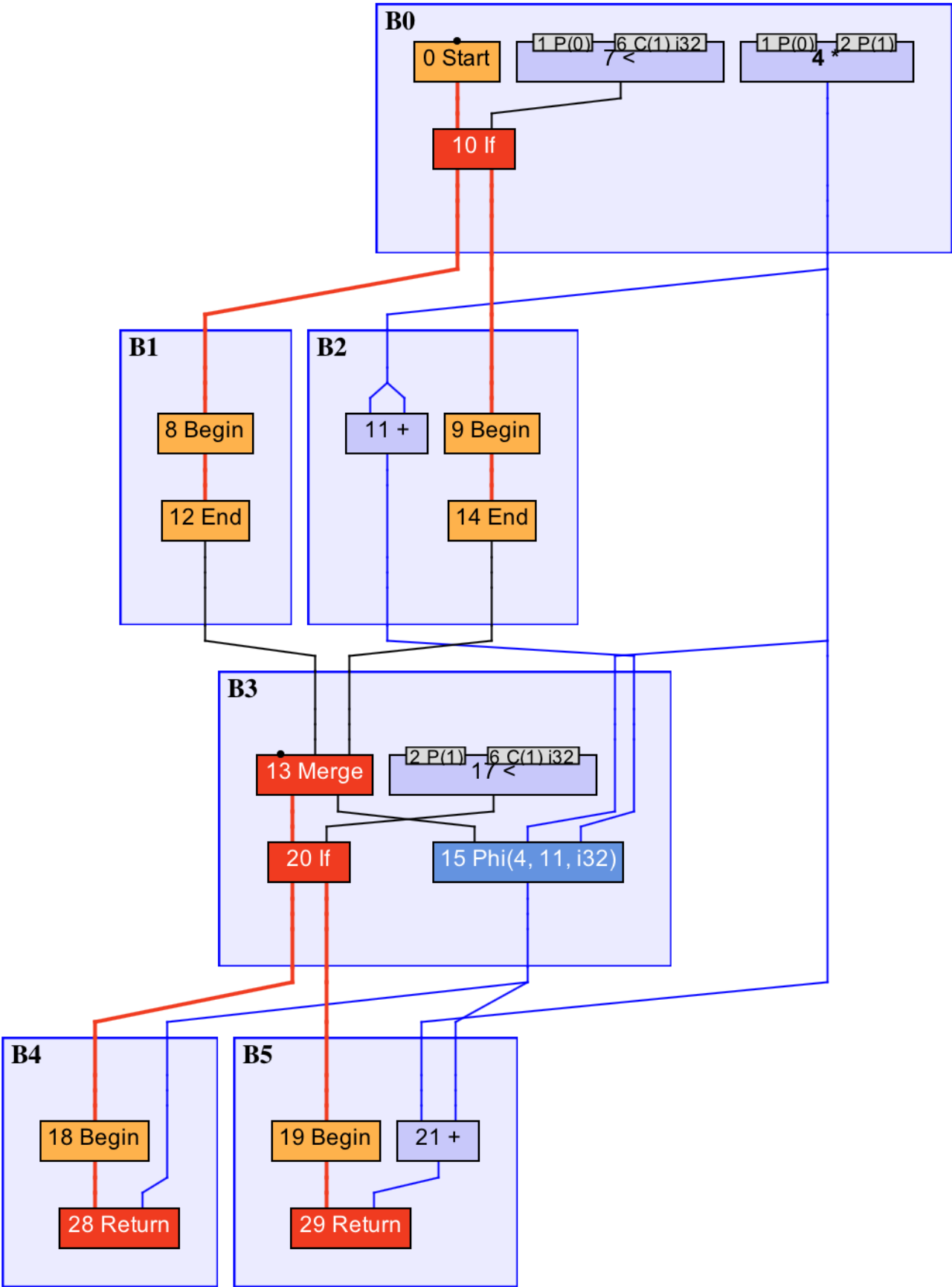
在 Sea-of-Nodes 中，由于只存在值的概念，因此 GVN 算法将非常简单：如果一个浮动节点本身不存在内存副作用（由于 GVN 可能影响节点调度，如果有内存副作用的话，那么将引发一些源代码中不可能出现的情况），那么即时编译器只需判断该浮动节点是否与已存在的浮动节点的类型相同，所输入的 IR 节点是否一致，便可以将这两个浮动节点归并成一个。

```
public static int foo(int a, int b) {  
    int sum = a * b;  
    if (a > 0) {  
        sum += a * b;  
    }  
    if (b > 0) {  
        sum += a * b;  
    }  
}
```



```
    }  
    return sum;  
}
```

我们来看一个实际的案例。在上面这段代码中，如果 a 和 b 都大于 0，那么我们需要做三次乘法。通过 GVN 之后，我们只会在 B0 中做一次乘法，并且在接下来的代码中直接使用乘法的结果，也就是 4 号 * 节点所代表的值。



我们可以将 GVN 理解为在 IR 图上的公共子表达式消除（Common Subexpression Elimination, CSE）。

这两者的区别在于，GVN 直接比较值的相同与否，而 CSE 则是借助词法分析器来判断两个表达式

相同与否。因此，在不少情况下，CSE 还需借助常量传播来达到消除的效果。

总结与实践

今天我介绍了即时编译器的内部构造。

即时编译器将所输入的 Java 字节码转换成 SSA IR，以便更好地进行优化。

具体来说，C2 和 Graal 采用的是一种名为 Sea-of-Nodes 的 IR，其特点用 IR 节点来代表程序中的值，并且将源程序中基于变量的计算转换为基于值的计算。

此外，我还介绍了 C2 和 Graal 的 IR 的可视化工具 IGV，以及基于 IR 的优化 GVN。

今天的实践环节，你可以尝试使用 IGV 来查看上一篇实践环节中的代码的具体编译过程。

你可以通过[该页面](#)下载当前版本的 IGV。解压后，可运行脚本位于 bin/idealgraphvisualizer 中。IGV 启动完成后，你可以通过下述指令将 IR 图打印至 IGV 中。（需附带 Graal 编译器的 Java 10 或以上版本。）

```
// java -XX:+UnlockExperimentalVMOptions -XX:+UseJVMCICompiler -
XX:CompileCommand='dontinline,CompilationTest::hash' -Dgraal.Dump=:3 -
Dgraal.MethodFilter='CompilationTest.hash' -Dgraal.OptDeoptimizationGrouping=false CompilationTest
public class CompilationTest {
    public static int hash(Object input) {
        if (input instanceof Exception) {
            return System.identityHashCode(input);
        } else {
            return input.hashCode();
        }
    }
}

public static void main(String[] args) throws InterruptedException {
    for (int i = 0; i < 500000; i++) {
        hash(i);
    }
    Thread.sleep(2000);
}
}
```



版权归极客邦科技所有，未经许可不得转载

精选留言



ext4

1

除了你上面提到的内存依赖，我看到C2的ideal graph里面还有一种依赖叫做I/O dependency，这个在Graal的graph里似乎也没有了。可以解释一下C2的这个I/O dependency是做什么的，以及Graal是如何替代这种依赖的表示的么？

2018-08-31